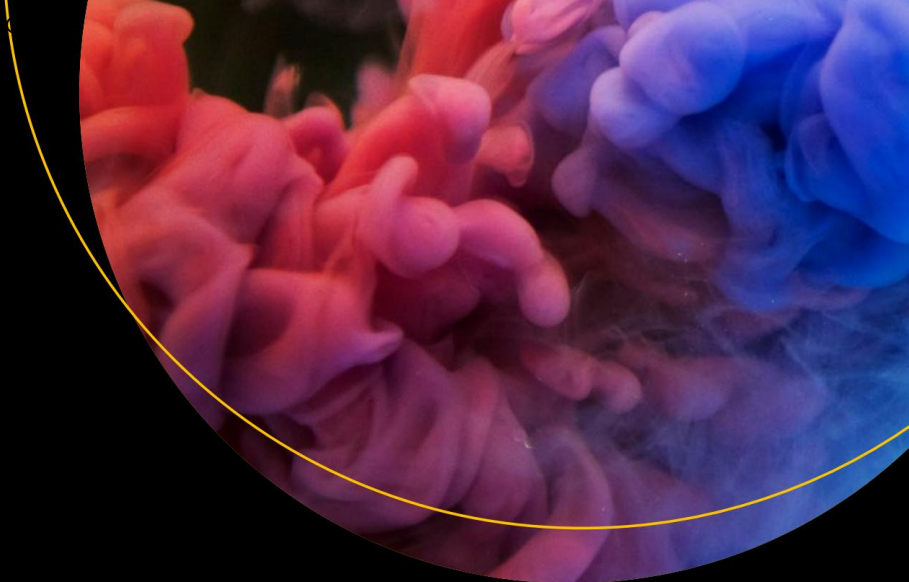




Представляем Мавен

Инструмент сборки для современной Java

Разработчики

Второе издание

Баладжи Варанаси

apress®

Представляем Мейвен

Инструмент сборки для сегодняшнего дня

Java-разработчики

Второе издание

Баладжи Варанаси

apress®

Представляем Maven: инструмент сборки для современных разработчиков Java

Баладжи Варанаси

Солт-Лейк-Сити, Юта, США

ISBN-13 (пбк): 978-1-4842-5409-7 <https://doi.org/10.1007/978-1-4842-5410-3>

ISBN-13 (электронный): 978-1-4842-5410-3

Copyright © 2019 Баладжи Варанаси

Эта работа является объектом авторского права. Все права сохраняются за Издателем, будь то весь материал или его часть, в частности права на перевод, перепечатку, повторное использование иллюстраций, декламацию, трансляцию, воспроизведение на микрофильмах или любым другим физическим способом, а также на передачу или хранение информации. Поиск, электронная адаптация, компьютерное программное обеспечение или аналогичная или отличающаяся методология, известная в настоящее время или разработанная в будущем.

В этой книге могут быть использованы названия торговых марок, логотипы и изображения. Вместо того, чтобы использовать символ товарного знака при каждом появлении имени, логотипа или изображения, зарегистрированного как товарный знак, мы используем имена, логотипы и изображения только в редакционных целях и в интересах владельца товарного знака, без намерения нарушить права на товарный знак.

Использование в данной публикации торговых наименований, товарных знаков, знаков обслуживания и аналогичных терминов, даже если они не идентифицированы как таковые, не должно рассматриваться как выражение мнения о том, являются ли они объектом прав собственности.

Хотя советы и информация, содержащиеся в этой книге, считаются верными и точными на дату публикации, ни авторы, ни редакторы, ни издатель не несут никакой юридической ответственности за любые ошибки или упущения, которые могут быть допущены. Издатель не дает никаких гарантий, явных или подразумеваемых, в отношении материалов, содержащихся здесь.

Управляющий директор ООО «Апресс Медиа»: Велмозд Шпар

Редактор по закупкам: Стив Энглин

Редактор разработки: Мэтью Муди

Координирующий редактор: Марк Пауэрс

Обложка разработана eStudioCalamar

Изображение на обложке разработано Freepik (www.freepik.com)

Распространяется в книжной торговле по всему миру компанией Springer Science+Business Media New York, 233 Spring Street, 6th Floor, New York, NY 10013. Телефон 1-800-SPRINGER, факс (201) 348-4505, электронная почта для заказов-ny@springer-sbm.com или посетите www.springeronline.com. Apress Media, LLC является калифорнийским ООО, а единственным участником (владельцем) является Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc — корпорация штата Делавэр.

Для получения информации о переводах, пожалуйста, пишите на editorial@apress.com; для прав на перепечатку, книгу в мягкой обложке или аудио, пожалуйста, отправьте электронное письмо по адресу bookpermissions@springernature.com.

Названия Apress можно приобрести оптом для академического, корпоративного или рекламного использования. Версии электронных книг и лицензии также доступны для большинства названий. Для получения дополнительной информации посетите нашу веб-страницу массовых продаж печатных и электронных книг по адресу <http://www.apress.com/bulk-sales>.

Любой исходный код или другой дополнительный материал, на который ссылается автор в этой книге, доступен читателям на странице продукта книги, расположенной по адресу www.apress.com/978-1-4842-5409-7.

Для получения более подробной информации посетите <http://www.apress.com/source-code>.

Напечатано на бескислотной бумаге.



Для моей Веды

Java

Программирование



ПРИСОЕДИНИТЬСЯ

Оглавление

Об автореix

О техническом рецензентеxi

Благодарностиxiii

Введениеxv

Глава 1: Начало работы с Maven1

 Стандартизированная структура каталогов2

 Декларативное управление зависимостями2

 Плагины3

 Унифицированная абстракция сборки3

 Инструменты Поддержка3

 Архитекты4

 Открытый исходный код4

 Альтернативы Maven5

 Муравей + Плоть5

 Грейда7

 Компоненты Maven7

 Maven SCM10

 Maven Ватон9

 Maven Доксия10

Резюме10

Оглавление

Глава 2: Настройка Maven11

Установка в Windows13

Установка на Mac13

Испытательная установка14

Получение помощи15

Дополнительные настройки16

Настройка прокси19

Защита паролей20

Поддержка IDE21

Резюме21

Глава 3. Управление зависимостями в Maven23

Использование новых репозиторий26

Идентификация зависимости28

Транзитивные зависимости29

Область зависимости32

Ручная установка зависимостей33

Резюме35

Глава 4: Основы проекта Maven37

Базовая организация проекта37

Понимание файла pom.xml40

Создание проекта42

Тестирование проекта44

Свойства в pom.xml49
Независимые свойства50
Пользовательские свойства50

Резюме51

Оглавление

Глава 5: Жизненный цикл Maven 53

Цели и платины 53

Жизненный цикл и этапы 57

Разработка плагинов 61

Рекомендации 68

Глава 6: Архетипы Maven 69

Встроенные архетипы 69

Создание веб-проекта 71

Многомодульный проект 74

Создание архетипа 80

Использование архетипа 86

Рекомендации 87

Глава 7: Документация и отчетность 89

Использование жизненного цикла сайта 89

Расширенная конфигурация сайта 94

Создание отчетов Javadoc 99

Генерация отчетов модульного тестирования 101

Создание отчетов о покрытии кода 102

Создание отчета SpotBugs 104

Рекомендации 105

Глава 8: Выпуск Maven 107

Интеграция с Nexus 107

Релиз проекта 113

Установка клиента Git 115

Создание репозитория GitHub 115

Проверка исходного кода 116

Оглавление

Выпуск Maven 118

Подготовить цель 118

Частый гол 123

Выполнить цель 123

Резюме 126

Глава 9: Непрерывная интеграция 127

Установка Дженкинса 128

Проект Maven 129

Настройка Jenkins 130

Запуск задания сборки 133

Резюме 135

Индекс 137

об авторе



Баладжи Варанаси — опытный лидер в области технологий, автор и оратор. Он имеет более чем 18-летний опыт управления, проектирования и предоставления высокопроизводительных масштабируемых корпоративных приложений. В этот период он работал в области безопасности, веб-доступности, поиска и корпоративных порталов. Он имеет степень магистра компьютерных наук Университета штата Юта. Он делится своими наблюдениями и экспериментами на <http://blog.inflinx.com>.

О техническом рецензенте

Херман Гонсалес-Моррис (Herman González-Morris) — полиглот, архитектор/инженер программного обеспечения с более чем 20-летним опытом работы в этой области, обладающий знаниями в области Java (EE), Spring, Haskell, C, Python и Javascript и других. Работает с распределенными веб-приложениями. Герман любит математические головоломки (включая чтение Кнута) и плавание. Он сделал технический обзор нескольких книг, в том числе книги о контейнерах приложений (WebLogic), а также книг, посвященных различным языкам программирования (Haskell, TypeScript, WebAssembly, Math для программистов и regex). Вы можете найти более подробную информацию в его блоге (<https://devwebcl.blogspot.com/>) или аккаунт в Твиттере (@devwebcl).

Благодарности

Эта книга была бы невозможна без поддержки нескольких человек, и я пользуюсь случаем, чтобы искренне поблагодарить их.

Спасибо замечательным людям из Apress: Стиву Энглину, Марку Пауэрсу, Мэтью Муди и многие другие. Я также хочу выразить огромную благодарность Херману Гонсалесу-Моррису за его технический обзор и за ценную отзыв, который он предоставил.

Наконец, я хочу поблагодарить мою жену Судху за ее постоянную поддержку и ободрение. Большое спасибо, дорогой!

Введение

Знакомство с Maven представляет собой краткое введение в Maven, фактически являющийся стандартом для создания, управления и автоматизации проектов на основе Java и JEE на предприятиях по всему миру. Книга начинается с объяснения основных концепций Maven и демонстрации того, как настроить и протестировать Maven на локальном компьютере. Затем он углубляется в такие концепции, как управление зависимостями, этапы жизненного цикла, подключаемые модули и цели. В нем также обсуждаются соглашения о структуре проекта, начало создания проекта с использованием архетипов, а также создание документации и отчетов. Наконец, он завершается обсуждением процесса выпуска Maven и интеграции с Jenkins.

Как устроена эта книга

Глава 1 начинается с краткого введения в Maven. В нем обсуждаются причины принятия Maven и дается обзор двух его альтернатив: Ant и Gradle.

Глава 2 посвящена настройке Maven на вашем компьютере и тестированию. установка. Он также предоставляет обзор настроек Maven. xml, и он показывает вам, как запустить Maven в прокси-сервере с поддержкой HTTP. Окружающая среда.

В главе 3 подробно рассматривается управление зависимостями в Maven. Затем обсуждаются координаты GAV, которые Maven использует для уникальной идентификации своих артефактов. Наконец, он охватывает транзитивные зависимости и их влияние на сборки.

Введение

В главе 4 обсуждается организация базового проекта Maven и рассматриваются важные элементы файла `pom.xml`. Затем вы узнаете о тестировании проекта с помощью JUnit.

Глава 5 подробно описывает жизненный цикл Maven, подключаемые модули, этапы сборки и цели. Затем он проведет вас через процесс создания и использования простого подключаемого модуля Maven.

В главе 6 представлены шаблоны проектов архетипов, которые позволяют для быстрой загрузки новых проектов. Встроенные архетипы используются для создания проекта Java, веб-проекта и многомодульного проекта. Затем вы создадите собственный архетип с нуля и используете его для создания нового проекта.

В главе 7 рассматриваются основы создания сайтов с помощью Maven. Затем обсуждается создание отчетов и документация, такая как Javadocs, отчеты о покрытии тестами и отчеты SpotBugs, а также способы их интеграции в систему. Сайт Мавен.

Глава 8 начинается с обсуждения менеджера репозитория Nexus. и показывает, как его можно интегрировать с Maven. Затем он обеспечивает полное освещение процесса выпуска Maven и его различных этапов.

Глава 9 знакомит с концепциями непрерывной интеграции (CI) и обсуждает установку и настройку Jenkins, инструмента с открытым исходным кодом для непрерывной интеграции.

Целевая аудитория

Знакомство с Maven предназначено для разработчиков и инженеров по автоматизации, которые хотели бы быстро начать работу с Apache Maven. Эта книга предполагает базовые знания Java. Предварительный опыт работы с Maven не требуется.

Загрузка исходного кода

Исходный код примеров из этой книги можно загрузить с помощью кнопки «Загрузить исходный код», расположенной на [странице www.apress.com/978-1-4842-5409-7](http://www.apress.com/978-1-4842-5409-7).

После загрузки разархивируйте код и поместите содержимое в папку C:\apress\gswm-book. Исходный код организован отдельными главами. Там, где это применимо, папки глав содержат проект gswm с минимальным набором файлов, чтобы вы могли начать работу с листингами кода этой главы. Папки глав также содержат папку с именем final, которая содержит ожидаемое конечное состояние проекта(ов).

Вопросы

Мы приветствуем отзывы читателей. Если у вас есть какие-либо вопросы или предложения, вы можете связаться с автором по адресу Balaji@inflinx.com.

ГЛАВА 1

Начало работы с Maven

Как и другие мастера, разработчики программного обеспечения полагаются на свои инструменты для создания приложений. Интегрированные среды разработки (IDE), инструменты отслеживания ошибок, инструменты сборки, фреймворки, контейнеры и инструменты отладки, такие как анализаторы памяти, играют жизненно важную роль в повседневной разработке и обслуживании качественного программного обеспечения. В этой книге обсуждаются и исследуются функции Maven, которые, как мы знаем, станут важным инструментом в вашем арсенале разработки программного обеспечения.

Apache Maven — это основанная на стандартах платформа управления проектами с открытым исходным кодом, которая упрощает создание, тестирование, создание отчетов и упаковку проектов. Первоначальные корни Maven уходят в проект Apache Jakarta Alexandria, который имел место в начале 2000 года. Впоследствии он использовался в проекте Apache Turbine. Как и многие другие проекты Apache того времени, проект Turbine имел несколько подпроектов, каждый со своей собственной системой сборки на основе Ant. В то время было сильное желание разработать стандартный способ создания проектов и легкого обмена сгенерированными артефактами между проектами. Это желание породило Мейвен. Версия Maven 1.0 была выпущена в 2004 г., за ней последовала версия 2.0 в 2005 г. и версия 3.0 в 2010 г. На момент написания этой книги текущей версией Maven была 3.6.1.

Maven стал одним из наиболее широко используемых программ с открытым исходным кодом. программы на предприятиях по всему миру. Давайте рассмотрим некоторые причины, по которым Maven так популярен.

Стандартизированная структура каталогов

Часто, когда мы начинаем работу над новым проектом, значительное количество времени тратится на выбор макета проекта и структуры папок, необходимых для хранения кода и файлов конфигурации. Эти решения могут сильно различаться в зависимости от проекта и команды, что может затруднить понимание и внедрение проектов других команд новыми разработчиками. Кроме того, существующим разработчикам может быть сложно переключаться между проектами и находить то, что они ищут.

Maven решает предыдущие проблемы, стандартизируя структуру папок и организацию проекта. Maven предоставляет рекомендации о том, где должны находиться различные части проекта, такие как исходный код, тестовый код и файлы конфигурации. Например, Maven предлагает разместить весь исходный код Java в папке `src\main\java`. Это упрощает понимание и навигацию по любому проекту Maven.

Кроме того, эти соглашения упрощают переход на новую среду IDE и ее использование. Исторически IDE различались структурой проекта и именами папок. Динамический веб-проект в Eclipse может использовать папку `WebContent` для хранения веб-активов, тогда как IntelliJ IDEA может использовать веб-папку для той же цели. С Maven ваши проекты имеют согласованную структуру и становятся независимыми от IDE.

Декларативное управление зависимостями

Большинство проектов Java полагаются на другие проекты и платформы с открытым исходным кодом для правильной работы. Загрузка этих зависимостей может быть обременительной. Вручную и отслеживать их версии по мере их использования в вашем проекте.

Maven предоставляет удобный способ объявить эти зависимости проекта в отдельном внешнем файле `pom.xml`. Затем он автоматически загружает эти зависимости и позволяет вам использовать их в своем проекте.

Это значительно упрощает управление зависимостями проекта. Важно отметить, что в файле `pom.xml` вы указываете что, а не как.

Глава 1 Начало работы с Maven

Файл `pom.xml` также может служить инструментом документации, в котором указаны зависимости вашего проекта и их версии.

Плагины

Maven следует архитектуре, основанной на подключаемых модулях, что позволяет легко расширять и настраивать его функциональность. Эти подключаемые модули инкапсулируют многократно используемую логику построения и задачи. Сегодня доступны сотни подключаемых модулей Maven, которые можно использовать для выполнения различных задач, от компиляции кода до упаковки и создания проектной документации.

Maven также упрощает создание собственных подключаемых модулей, что позволяет интегрировать задачи и рабочие процессы, характерные для вашей организации.

Унифицированная абстракция сборки

Maven предоставляет единый интерфейс для построения проектов. Вы можете создать проект Maven, используя всего несколько команд. Как только вы ознакомитесь с процессом сборки Maven, вы сможете легко понять, как создавать другие проекты Maven. Это освобождает разработчиков от необходимости изучать особенности сборки, чтобы они могли больше сосредоточиться на разработке.

Поддержка инструментов

Maven предоставляет мощный интерфейс командной строки для выполнения различных операций. Все основные IDE на сегодняшний день обеспечивают отличную инструментальную поддержку Maven. Кроме того, Maven полностью интегрирован с современными продуктами непрерывной интеграции, такими как Jenkins и Bamboo.

Архетипы

Как мы уже упоминали, Maven предоставляет стандартную структуру каталогов для своих проектов. Когда приходит время создавать новый проект Maven, вам нужно создавать каждый каталог вручную, и это может легко стать утомительным.

Здесь на помощь приходят архетипы Maven. Архетипы Maven — это предопределенные шаблоны проектов, которые можно использовать для создания новых проектов.

Проекты, созданные с использованием архетипов, будут содержать все папки и файлы, необходимые для работы.

Архетипы также являются ценным инструментом для объединения лучших практик и общих активов, которые вам понадобятся в каждом из ваших проектов. Рассмотрим команду, которая активно работает над веб-приложениями на базе Spring Framework.

Все веб-проекты на основе Spring имеют общие зависимости и требуют набора файлов конфигурации. Также весьма вероятно, что все эти веб-проекты имеют схожие файлы конфигурации Log4j/Logback, CSS/изображения и макеты страниц Thymeleaf. Maven позволяет этой команде объединить эти общие активы в архетип. Когда новые проекты создаются с использованием этого архетипа, в них автоматически включаются общие активы. Больше не нужно копировать и вставлять код или перетаскивать файлы.

Открытый исходный код

Maven имеет открытый исходный код, и его загрузка и использование не требуют никаких затрат. Он поставляется с богатой онлайн-документацией и поддержкой активного сообщества.

КОНВЕНЦИЯ ПО КОНФИГУРАЦИИ

Соглашение по конфигурации (CoC) или кодирование по соглашению — один из ключевых элементов Maven. Популяризированный сообществом Ruby on Rails, CoC делает упор на разумные значения по умолчанию, тем самым сокращая количество принимаемых решений. Это экономит время, а также приводит к более простому конечному продукту, поскольку объем требуемой конфигурации резко сокращается.

Глава 1 Начало работы с Maven

В рамках соблюдения CoC Maven предоставляет несколько разумных значений по умолчанию для своих проектов. Он устанавливает стандартную структуру каталогов и предоставляет значения по умолчанию для сгенерированных артефактов. Представьте, что вы смотрите на артефакт Maven с именем `log4j-core-2.11.2.jar`. С первого взгляда видно, что вы просматриваете JAR-файл `log4j-core` версии 2.11.2.

Одним из недостатков Maven CoC является жесткость, с которой сталкиваются конечные пользователи при его использовании. Чтобы решить эту проблему, вы можете настроить большинство значений Maven по умолчанию.

Например, можно изменить расположение исходного кода Java в вашем проекте.

Однако, как правило, такие изменения значений по умолчанию должны быть сведены к минимуму.

Альтернативы Мавену

Хотя основное внимание в этой книге уделяется Maven, давайте рассмотрим пару его альтернатив: Ant + Ivy и Gradle.

Муравей + Плющ

Apache Ant (<http://ant.apache.org>) — популярный инструмент с открытым исходным кодом для создания скриптов. Ant основан на Java и использует Extensible Markup Language (XML) для своей конфигурации. Файлом конфигурации по умолчанию для Ant является файл `build.xml`.

Использование Ant обычно включает в себя определение задач и целей. Как следует из названия, задача Ant — это единица работы, которую необходимо выполнить. Типичные задачи включают создание каталога, запуск теста, компиляцию исходного кода, создание файла архива веб-приложения (WAR) и т. д. цель _ это просто набор задач. Цель может зависеть от других целей. Эта зависимость позволяет нам упорядочить целевое выполнение. Листинг 1-1 демонстрирует простой файл `build.xml` с одной целью, называемой `compile`. Цель компиляции имеет две эхо-задачи и одну задачу `javac`.

Глава 1 Начало работы с Maven

Листинг 1-1. Пример файла Ant build.xml

```

<project name="Sample Build File" по умолчанию="compile" на
основе=".">

    <target name="compile" description="Компилировать исходный код">
        <echo message="Начало компиляции кода"/>
        <javac srcdir="src" destdir="dist"/>
        <echo message="Завершенная компиляция кода"/>
    </target>

</project>

```

Ant не накладывает никаких условностей или ограничений на ваш проект, и известно, что он чрезвычайно гибкий. Эта гибкость иногда приводила к сложной, трудной для понимания и трудной в обслуживании сборке. xml-файлы.

Apache Ivy (<http://ant.apache.org/ivy/>) обеспечивает автоматическое управление зависимостями, что делает использование Ant более приятным. В Ivy вы объявляете зависимости в XML-файле с именем ivy.xml, как показано в листинге 1-2. Интеграция Ivy с Ant включает в себя объявление новых целей в файле build.xml для извлечения и разрешения зависимостей.

Листинг 1-2. Пример листинга плюща

```

<плющ-модуль версия="2.0">
    <info Organization="com.apress" module="gswm-ivy" />

    <зависимости>
        <зависимость org="org.apache.logging.log4j" name="log4j-api"
            версия = "2.11.2" />
    </зависимости>
</плющ-модуль>

```

Грейдл

Gradle (<http://gradle.org/>) — это сборка с открытым исходным кодом, инструмент автоматизации проектов, который можно использовать для проектов Java и других проектов. В отличие от Ant и Maven, которые используют XML для настройки, Gradle использует доменный язык (DSL) на основе Groovy.

Gradle обеспечивает гибкость Ant и использует то же понятие задач. В листинге 1-3 показан файл build.gradle по умолчанию.

Листинг 1-3. Файл build.gradle по умолчанию

```
плагины {  
    идентификатор 'java'  
}  
  
версия = '1.0.0'  
  
репозитории {  
    mavenCentral()  
}  
  
зависимости {  
    Группа testCompileOnly: «junit», имя: «junit», версия: «4.10»  
}
```

DSL Gradle и его приверженность CoC приводят к компактным файлам сборки. Первая строка в листинге 1-3 включает подключаемый модуль Java для использования в сборке. Плагины в Gradle предоставляют предварительно настроенные задачи и зависимости для проекта. Подключаемый модуль Java, например, предоставляет задачи для создания исходных файлов, выполнения модульных тестов и установки артефактов. Раздел зависимостей в файле default.build указывает Gradle использовать зависимость JUnit во время компиляции тестовых исходных файлов. Гибкость и производительность Gradle способствовали его растущей популярности. Однако кривая обучения

Глава 1 Начало работы с Maven

Gradle и Groovy DSL могут быть крутыми. Поддержка IDE и подключаемых модулей для Gradle также менее развита по сравнению с Maven.

Несмотря на растущую конкуренцию со стороны других инструментов, Maven продолжает доминировать в пространстве инструментов сборки. Это видно на рис. 1-1, где показаны результаты опроса Synk.io 2018 года (<https://synk.io/blog/>).

[jvm-ecosystem-report-2018-tools/](#) об инструментах сборки, используемых разработчиками Java.

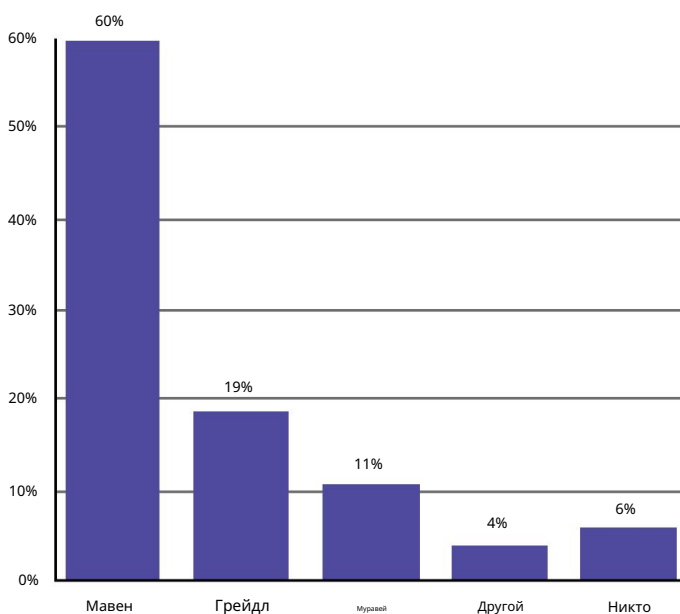


Рисунок 1-1. Результаты опроса об использовании инструментов сборки

Проект Polyglot для Maven (<https://github.com/takari/polyglot-maven>) позволяет создавать POM-файлы Maven на диалектах, отличных от XML. Поддерживаемые языки включают Groovy, Ruby, Scala, YAML и Java.

Компоненты Maven

Maven полагается на несколько компонентов для выполнения своей работы. Хотя вы можете не взаимодействовать с этими компонентами напрямую, обзор этих компонентов поможет понять внутреннюю работу Maven и лучше подготовиться к устранению ошибок Maven.

Мавен СКМ

Maven взаимодействует с несколькими системами контроля версий/управления кодом (SCM) для выполнения таких операций, как проверка кода или создание ветки или тега. Проект Maven SCM (<http://maven.apache.org/scm/>) предоставляет общий API для выполнения таких операций. Подключаемый модуль выпуска Maven, обсуждаемый в главе 8, в значительной степени зависит от компонентов Maven SCM. В настоящее время Maven SCM поддерживает несколько популярных систем управления кодом, таких как Git, Subversion и Perforce.

Мейвен Вагон

Как обсуждалось ранее, Maven автоматически загружает зависимости проекта, такие как файлы JAR, из разных мест, таких как FTP, файловые системы и веб-сайты. Проект Maven Wagon (<https://maven.apache.org/wagon/>) предоставляет транспортную абстракцию, которая позволяет Maven легко взаимодействовать с различными транспортными протоколами и извлекать зависимости. Maven Wagon поддерживает некоторые популярные протоколы, такие как

- а) Файл: позволяет извлекать зависимости с использованием протокола файловой системы.
- б) HTTP: позволяет извлекать зависимости с помощью HTTP/ протоколы HTTPS. Предоставляются два варианта реализации: один использует Apache HttpClient, а другой использует стандартную библиотеку Java.
- в) FTP: позволяет извлекать зависимости с помощью протокола передачи файлов.

Мейвен Доксия

Maven Doxia (<https://maven.apache.org/doxia/>) — это фреймворк для создания контента, который можно использовать для создания статического/динамического контента, такого как PDF-файлы и веб-страницы. Doxia поддерживает несколько популярных языков разметки, таких как Markdown, Apt, XHTML и Confluence. Maven в значительной степени полагается на Doxia для создания проектной документации и отчетов (подробнее об этом в главе 7).

Резюме

Apache Maven значительно упрощает процесс сборки и автоматизирует задачи управления проектами. В этой главе было представлено краткое введение в Maven и описаны основные причины его принятия. Мы также посмотрели на близких сверстников Maven: Ant + Ivy и Gradle.

В следующей главе вы узнаете о настройке, необходимой для и работает с Maven.

ГЛАВА 2

Настройка Maven

Установка Maven — простой и понятный процесс. В этой главе объясняется, как установить и настроить Maven в операционных системах Windows 10 и Mac. Вы можете выполнить аналогичную процедуру с другими операционными системами.

Примечание . Maven — это приложение на основе Java, для правильной работы которого требуется комплект Java Development Kit (JDK). Для Maven версии 3.6 требуется JDK 1.7 или выше. Прежде чем продолжить установку Maven, убедитесь, что у вас установлена Java. Если нет, установите JDK (а не только среду выполнения Java [JRE]) с [сайта `www.oracle.com/technetwork/java/javase/downloads/index.html`](http://www.oracle.com/technetwork/java/javase/downloads/index.html). Убедитесь, что у вас установлена переменная среды `JAVA_HOME`, указывающая на установку JDK. В этой книге мы будем использовать JDK 8.

Вы начнете процесс установки, загрузив последнюю версию Maven с веб-сайта Apache Maven (<http://maven.apache.org/>) . [скачать.html](#)). На момент написания этой статьи последняя версия — 3.6.1. Загрузите двоичный ZIP-файл Maven 3.6.1, как показано на рис. 2-1.

Глава 2. Настройка Maven

	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.6.1-bin.tar.gz	apache-maven-3.6.1-bin.tar.gz.sha512	apache-maven-3.6.1-bin.tar.gz.asc
Binary zip archive	apache-maven-3.6.1-bin.zip	apache-maven-3.6.1-bin.zip.sha512	apache-maven-3.6.1-bin.zip.asc
Source tar.gz archive	apache-maven-3.6.1-src.tar.gz	apache-maven-3.6.1-src.tar.gz.sha512	apache-maven-3.6.1-src.tar.gz.asc
Source zip archive	apache-maven-3.6.1-src.zip	apache-maven-3.6.1-src.zip.sha512	apache-maven-3.6.1-src.zip.asc

Рисунок 2-1. скачать архив maven

После завершения загрузки разархивируйте дистрибутив в локальную директорию на вашем компьютере. Будет создана папка с именем apache-maven 3.6.1-bin, содержимое которой показано на рис. 2-2.

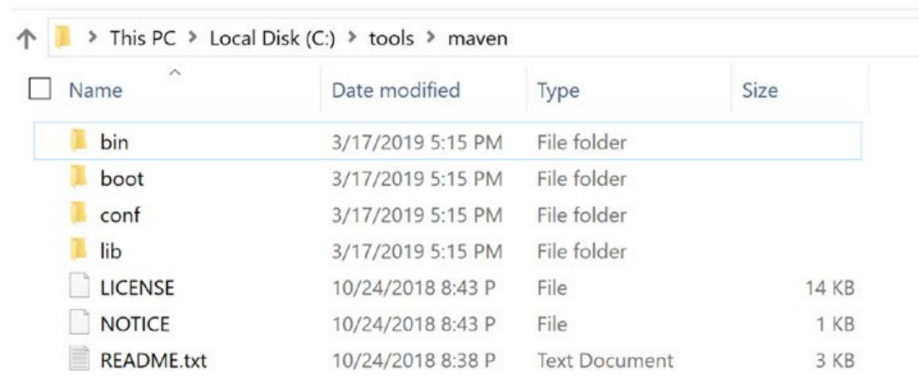


Рисунок 2-2. Содержимое каталога установки Maven

Папка bin содержит исполняемые файлы Maven для конкретной платформы — mvn.cmd для Windows и mvn.sh для платформ Mac и Unix/Linux, которые можно использовать для запуска Maven. Отладочные версии этих исполняемых файлов — mvnDebug.cmd и mvnDebug.sh — включают аргументы отладки, которые позволяют подключить IDE к запущенному процессу Maven для удаленной отладки.

Эта загрузочная папка содержит файл «plexus-classworlds-2.5.2.jar». Мавен использует фреймворк Plexus Classworlds (<https://codehaus-plexus.github.io/plexus-classworlds/>) для построения графа загрузчика классов.

Папка `conf` содержит файлы конфигурации, которые можно использовать для изменения поведения Maven по умолчанию. Важным файлом в этой папке является файл `settings.xml`, который мы рассмотрим позже в этой главе. Другой файл — это файл `simplelogger.properties`, который позволяет вам управлять ведением журнала Maven. Например, уровень журнала можно изменить на отладочный, задав для свойства `defaultLogLevel` значение отладки. Точно так же вы можете изменить свойство `logFile`, чтобы записать вывод журнала из «`System.out`» в файл.

Наконец, папка `lib` содержит библиотеки, необходимые для Maven, и его плагины для правильной работы.

Установка в Windows

Переместите содержимое `apache-maven-3.6.0-bin` в новый каталог `c:\tools\maven`. Следующим шагом является добавление исполняемого файла Maven в переменную среды `PATH`, чтобы вы могли запускать команды Maven из командной строки. В поле поиска Windows найдите «Переменная среды» и выберите «Редактировать системные переменные среды». В появившемся окне нажмите кнопку «Переменные среды», выберите переменную `PATH` и нажмите «Изменить». Нажмите «Создать» и введите значение «`C:/tools/maven/bin`» и нажмите «ОК».

Установка на Mac

Переместите содержимое папки `apache-maven-3.6.0-bin` в `$HOME/tools/maven`, где `$HOME` — ваш домашний каталог на Mac. Изменить `.bash_profile`, выполнив команду `nano ~/.bash_profile`. Добавьте исполняемый файл Maven в переменную `PATH`, добавив следующую строку в `.bash_profile`:

```
экспорт PATH = $ HOME/tools/maven/bin: $ PATH
```

Глава 2. Настройка Maven

На этом установка Maven завершена. Если у вас есть какие-либо открытые окна/терминалы командной строки, закройте их и снова откройте новый окно командной строки. Когда переменные среды добавляются или изменены, новые значения не распространяются на автоматическое открытие окон командной строки.

MAVEN_OPTS ПЕРЕМЕННАЯ СРЕДЫ

При использовании Maven, особенно в сложном проекте, есть вероятность, что вы столкнетесь с ошибками OutOfMemory. Это может произойти, например, при выполнении большого количества тестов JUnit или при создании большого количества отчетов. Чтобы устранить эту ошибку, увеличьте размер кучи виртуальной машины Java (JVM), используемой Maven. Это делается глобально путем создания новой переменной среды с именем MAVEN_OPTS. Для начала рекомендуем использовать значение -Xmx512m.

Тестовая установка

Теперь, когда Maven установлен, пришло время протестировать и проверить установку. Откройте командную строку и выполните следующую команду:

```
mvn -v
```

На компьютере с Windows эта команда должна выводить информацию похоже на следующее:

```
C:\> mvn -v
```

```
Apache Maven 3.6.1 (d66c9c0b3152b2e69ee9bac180bb8fcc8e6af555; 2019-04-04T13:00:29-06:00)
```

```
Дом Maven: C:\tools\maven\bin\..
```

```
Версия Java: 1.8.0_144, поставщик: Oracle Corporation, среда выполнения: C:
```

```
Java\jdk1.8.0_144\jre
```

Язык по умолчанию: en_US, кодировка платформы: Cp1252.

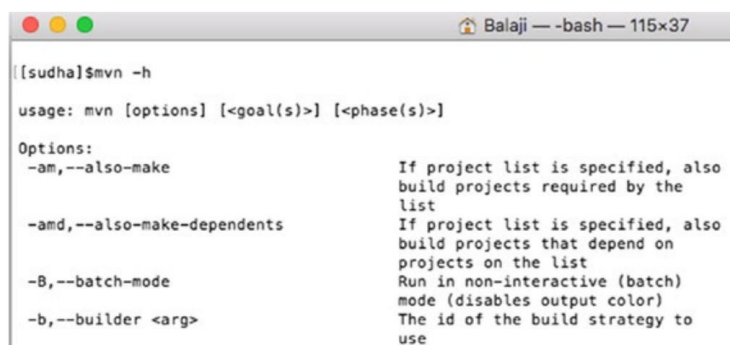
Название ОС: "windows 10", версия: "10.0", архитектура: "amd64", семейство: "windows"

Параметр командной строки `-v` сообщает путь, по которому установлен Maven и какую версию Java он использует. Вы также получите те же результаты, запустив расширенную команду `mvn --version`.

Получать помощь

Вы можете получить список параметров командной строки Maven, используя параметры `-h` или `--help`. Выполнение следующей команды приведет к выводу, аналогичному показанному на рис. 2-3.

МВН -Ч



```
[sudha]$mvn -h
usage: mvn [options] [<goal(s)>] [<phase(s)>]

Options:
  -am,--also-make                If project list is specified, also
                                build projects required by the
                                list
  -amd,--also-make-dependents    If project list is specified, also
                                build projects that depend on
                                projects on the list
  -B,--batch-mode                Run in non-interactive (batch)
                                mode (disables output color)
  -b,--builder <arg>            The id of the build strategy to
                                use
```

Рисунок 2-3. Результаты выполнения команды справки Maven

Дополнительные настройки

Шагов по установке, которые мы предоставили до сих пор, достаточно, чтобы начать работу с Maven. Однако для большинства корпоративных целей необходимо предоставить дополнительную информацию о конфигурации. Эта пользовательская конфигурация предоставляется в файле `settings.xml`. Maven ищет файл `settings.xml` в двух местах — в папке `conf` установки Maven и в папке `.m2` в

домашний каталог пользователя. Файл `settings.xml` в папке `conf` называется глобальными настройками, а файл в папке `.m2` называется пользовательскими настройками.

Если оба файла существуют, Maven объединит содержимое двух файлов, и пользовательские настройки будут иметь приоритет.

Примечание. Папка `.m2` важна для бесперебойной работы Maven.

Помимо прочего, в этой папке находится файл `settings.xml` и папка репозитория.

Папка репозитория содержит JAR-файлы подключаемых модулей и метаданные, необходимые Maven. Он также содержит файлы JAR, зависящие от проекта, которые Maven загрузил из Интернета. мы возьмем

присмотритесь к этой папке в главе 3.

По умолчанию папка `.m2` находится в вашем домашнем каталоге. В Windows

это обычно каталог `C:\Users\<<your_user_`

имя>>. На Mac это обычно каталог `/Users/<<your_`

имя_пользователя>>/.m2/репозиторий. Вы можете запустить команду `mvn`

`help:evaluate -Dexpression=settings.localRepository`, чтобы определить местоположение локального репозитория.

Когда вы запускаете команду Maven, Maven автоматически создает папку `.m2`.

Однако, если вы не видите эту папку на своем компьютере, создайте ее.

По умолчанию папка .m2 не содержит файла settings.xml. В папке .m2 на локальном компьютере создайте файл settings.xml и скопируйте содержимое скелета файла settings.xml, как показано в листинге 2-1. Мы рассмотрим некоторые из этих элементов в следующих главах. Краткое описание некоторых элементов представлено в таблице 2-1.

Листинг 2-1. Скелет Settings.xml Содержание

```
<настройки xmlns="http://maven.apache.org/SETTINGS/1.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0 http://
    maven.apache.org /xsd/настройки-- 1.0.0.xsd">

  <localRepository/>
  <interactiveMode/>
  <офлайн/>
  <pluginGroups/>
  <серверы/>
  <зеркала/>
  <прокси/>
  <профили/>
  <активные профили/>
</настройки>
```

Таблица 2-1. Детали элементов settings.xml

Имя элемента	Описание
localRepository	Maven хранит копии подключаемых модулей и зависимостей локально в папке <code>c:\Users\<<ваше_имя_пользователя>>\.m2\repository</code>. Элемент <code>localRepository</code> можно использовать для изменения пути к локальному репозиторию. Например, <code><localRepository>c:\mavenrepo </localRepository></code> изменит расположение репозитория на папку <code>mavenrepo</code>.
InteractiveMode	Как следует из названия, когда для этого значения установлено значение <code>true</code>, Maven взаимодействует с пользователем для ввода. Если значение равно <code>false</code>, Maven попытается использовать разумные значения по умолчанию. Значение по умолчанию верно.
не в сети	Если установлено значение <code>true</code>, эта конфигурация указывает Maven не подключаться к сети и работать в автономном режиме. Если для автономного режима установлено значение <code>true</code>, Maven не будет пытаться загружать новые зависимости или обновления для зависимостей. Значение по умолчанию — ложь.
серверы	Maven может взаимодействовать с различными серверами, такими как серверы Git, серверы сборки и серверы удаленных репозиториях. Этот элемент позволяет указать учетные данные безопасности, такие как имя пользователя и пароль, необходимые для подключения к этим серверам.
зеркала	Как следует из названия, зеркала позволяют указать альтернативные места для загрузки зависимостей из удаленных репозиториях. Например, ваша организация могла создать зеркальную копию общедоступного репозитория в своей внутренней сети. Зеркальный элемент позволяет заставить Maven использовать внутренний зеркальный репозиторий вместо общедоступного репозитория.
прокси	Прокси-серверы содержат информацию о прокси-сервере HTTP, необходимую для подключения к Интернету.

Настройка прокси

Как мы подробно обсудим в главе 3, Maven требует подключения к Интернету для загрузки подключаемых модулей и зависимостей. Некоторые компании используют прокси-серверы HTTP для ограничения доступа в Интернет. В этих сценариях запуск Maven приведет к ошибке «Невозможно загрузить артефакт». Чтобы решить эту проблему, отредактируйте файл settings.xml и добавьте информацию о прокси-сервере, относящуюся к вашей компании. Пример конфигурации показан в листинге 2-2.

Листинг 2-2. Settings.xml с прокси-контентом

```
<настройки xmlns="http://maven.apache.org/SETTINGS/1.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
    http://maven.apache.org/xsd/settings 1.0.0.xsd">

  <прокси>
    <прокси>
      <id>прокси компании</id>
      <active>истина</active>

      <протокол>http</протокол>
      <host>proxy.company.com</host>
      <порт>8080</порт>
      <username>имя пользователя-прокси</username>
      <пароль>прокси-пароль</пароль>
      <непрокси-хосты />
    </прокси>
  </прокси>
</настройки>
```

Защита паролей

Пароль для подключения к прокси-серверу в разделе 2-2 хранится открытым текстом в файле settings.xml. Если вы случайно поделились своими настройками.

xml, ваш пароль будет скомпрометирован. Чтобы решить эту проблему, Maven предоставляет механизм шифрования паролей, которые сохраняются в настройках. xml-файл.

Мы начинаем процесс шифрования, создавая мастер-пароль, используя следующий код:

```
mvn -emp мой мастер-пароль
{LCWw0+NAqw0HuYH9HNz+1D7aElXM242PtuyoDXDAuelxjwZC8MyXaACkHSy7 tZwU}
```

Maven требует сохранения нового сгенерированного мастер-пароля в файле settings-security.xml в папке .m2. Создайте новый файл settings-security.

xml в папке .m2 и скопируйте в этот файл следующее содержимое.

```
<НастройкиБезопасности>
<мастер>{LCWw0+NAqw0HuYH9HNz+1D7aElXM242PtuyoDXDAuelxjwZC8MyXaA
CkHSy7tZwU}</master>
</НастройкиБезопасности>
```

Выполните следующую команду, чтобы зашифровать пароль «proxypassword». После завершения команды скопируйте вывод и замените им открытый текстовый пароль в файле settings.xml:

```
mvn -ep прокси-пароль
{f4RnaIHgXqgHyKYySxor+cvshmhWeTAvNjuORNYyu5w=}
```

Хотя предыдущий процесс шифрует пароли и позволяет избежать необходимости сохранять пароли в открытом виде, важно помнить, что любой, у кого есть доступ к файлу settings-security.xml, может легко расшифровать пароль.

мастер-пароль и впоследствии расшифровать пароли в настройках.

xml-файл. Одним из способов решения этой проблемы является сохранение файла `settings-security.xml` на внешнем устройстве, например на USB-накопителе.

Поддержка IDE

В этой книге мы будем использовать командную строку для создания и сборки примеров приложений. Если вы заинтересованы в использовании IDE, хорошая новость заключается в том, что все современные IDE поставляются с полной интеграцией Maven без необходимости дополнительной настройки.

Резюме

В этой главе вы ознакомились с настройкой Maven на локальном компьютере.

Вы узнали, что Maven загружает плагины и артефакты, необходимые для его работы.

Эти артефакты хранятся в `.m2\repository`

папка. Папка `.m2` также содержит файл `settings.xml`, который можно использовать для настройки поведения Maven.

В следующей главе мы более подробно рассмотрим зависимость Maven. управление.

ГЛАВА 3

Зависимость от Maven Управление

Проекты корпоративного уровня обычно зависят от множества библиотек с открытым исходным кодом. Рассмотрим сценарий, в котором вы хотите использовать Log4J для ведения журнала вашего приложения. Для этого вы должны перейти к Log4J страницу загрузки, загрузите файл JAR и поместите его в библиотеку вашего проекта. папку или добавьте ее в путь к классам проекта. Есть несколько проблем с этим подходом:

1. Загруженный файл JAR может зависеть от нескольких других библиотек.
Теперь вам придется выследить
все эти зависимости (и их зависимости) и добавить их в свой
проект.
2. Когда придет время обновить файл JAR, вам нужно будет начать
процесс заново.
3. Вам необходимо добавить файлы JAR в систему управления
версиями вместе с исходным кодом, чтобы ваши проекты
можно было создавать на компьютере, отличном от вашего
собственного. Это увеличивает размер проекта, время проверки и сборки.
4. Совместное использование файлов JAR между командами внутри
вашей организации становится затруднительным.

Глава 3 Управление зависимостями Maven

Для решения этих проблем Maven предоставляет декларативное управление зависимостями. При таком подходе вы объявляете зависимости вашего проекта во внешнем файле с именем `pom.xml`. Maven автоматически загрузит эти зависимости и передаст их вашему проекту для сборки, тестирования или упаковки.

На рис . 3-1 показано высокоуровневое представление управления зависимостями Maven. Когда вы запускаете свой проект Maven в первый раз, Maven подключается к сети и загружает артефакты и связанные метаданные из удаленных репозиторий. Удаленный репозиторий по умолчанию называется Maven Central и находится по адресу `repo.maven.apache.org` и `uk.maven.org`. Maven помещает копию этих загруженных артефактов в свой локальный репозиторий. При последующих запусках Maven будет искать артефакт в своем локальном репозитории; и, не найдя артефакт, Maven попытается загрузить его из удаленного репозитория.

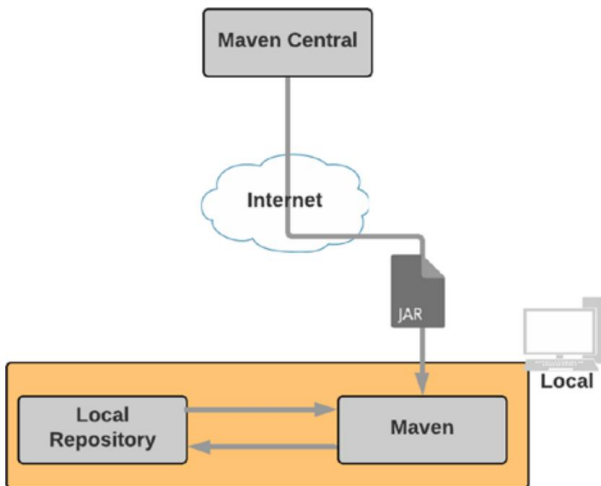


Рисунок 3-1. Управление зависимостями Maven

Хотя архитектура, показанная на рис. 3-1, работает в большинстве случаев, она создает несколько проблем в корпоративной среде. Первая проблема заключается в том, что обмен артефактами, связанными с компанией, между командами невозможен.

Глава 3 Управление зависимостями Maven

возможный. Из соображений безопасности и интеллектуальной собственности вы не захотите публиковать артефакты вашего предприятия на Maven Central.

Другая проблема касается юридических и лицензионных вопросов. Ваша компания может захотеть, чтобы команды использовали только официально одобренное программное обеспечение с открытым исходным кодом, и эта архитектура не подходит для этой модели. Последний вопрос касается пропускной способности и скорости загрузки. Во время большой нагрузки на Maven Central скорость загрузки артефактов Maven снижается, и это может негативно сказаться на ваших сборках. Следовательно, большинство предприятий используют архитектуру, показанную на рис. 3-2.

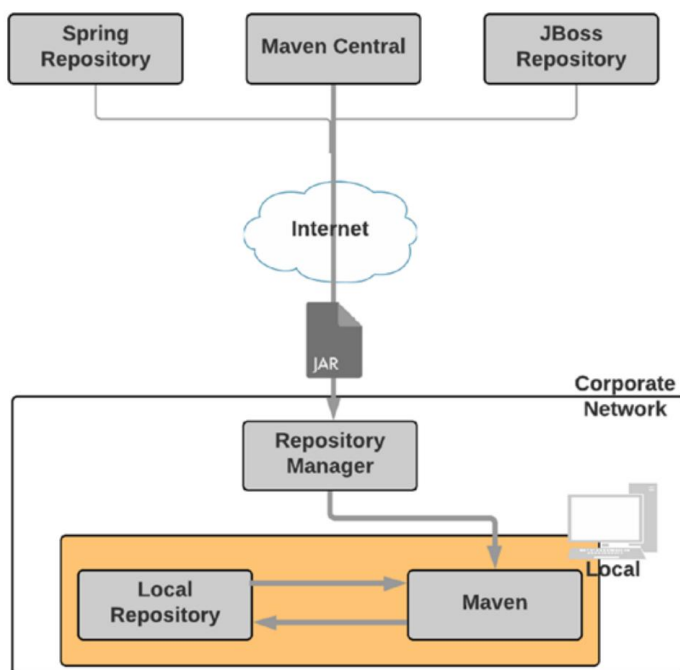


Рисунок 3-2. Архитектура корпоративного репозитория Maven

Внутренний менеджер репозитория действует как прокси для удаленных репозиториев. Это позволяет кэшировать артефакты из удаленных репозиториев, что ускоряет загрузку артефактов и повышает производительность сборки. Поскольку у вас есть полный контроль над внутренним хранилищем, вы можете регулировать

Глава 3 Управление зависимостями Maven

типы артефактов, разрешенных в вашей компании. Кроме того, вы также можете передавать артефакты вашей организации в менеджер репозитория, тем самым обеспечивая совместную работу. Существует несколько менеджеров репозитория с открытым исходным кодом, как показано в таблице 3-1.

Таблица 3-1. Менеджеры репозитория с открытым исходным кодом

Менеджер репозитория	URL-адрес
Репозиторий Nexus OSS	www.sonatype.com/nexus-repository-oss
Апач Архив	http://archiva.apache.org/
Artifactory с открытым исходным кодом	https://jfrog.com/c-открытым-исходным кодом/#artifactory

Использование новых репозиториях

Чтобы использовать новый репозиторий, вам необходимо изменить настройки. xml-файл. В листинге 3-1 показаны репозитории Spring и JBoss, добавленные в файл settings.xml. Таким же образом вы можете добавить менеджера репозитория вашей компании.

Примечание . Информация о репозиториях может быть предоставлена в файле settings.xml или pom.xml. У каждого подхода есть свои плюсы и минусы. Размещение информации о репозитории в файле pom.xml может сделать ваши сборки переносимыми. Это позволяет разработчикам загружать проекты и просто создавать их без каких-либо дополнительных изменений в их локальном файле settings.xml. Проблема с этим подходом заключается в том, что при выпуске артефактов соответствующий pom.xml файлы будут иметь информацию о репозитории, жестко закодированную в них. Если URL-адреса репозитория когда-либо изменятся, потребители этих артефактов

столкнется с ошибками из-за неверных путей к репозиторию. Помещение информации о репозитории в файл settings.xml решает эту проблему, и из-за обеспечиваемой гибкости подход settings.xml обычно рекомендуется в корпоративных условиях.

Листинг 3-1. Добавление репозиториев в settings.xml

```
<?xml версия="1.0" кодировка="UTF-8" ?>
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http:// /
maven.apache.org/SETTINGS/1.0.0 http://maven.apache.org/xsd/
settings-1.0.0.xsd">

    .....

    <профили>
        <профиль>
            <id>ваша_компания</id>
            <репозитории>
                <репозиторий>
                    <id>весеннее_репозиторий</id>
                    <url>http://repo.spring.io/release/</url>
                </репозиторий>
                <репозиторий>
                    <id>jboss_repo</id>
                    <url>https://repository.jboss.org/</url>
                </репозиторий>
            </репозитории>
        </профиль>
    </профили>
```


Глава 3 Управление зависимостями Maven

```
<активные профили>
<activeProfile>ваша_компания</activeProfile>
</активные профили>
```

```
.....
```

```
</настройки>
```

Идентификация зависимости

Зависимости Maven обычно представляют собой архивы, такие как JAR, WAR, корпоративный архив (EAR) и ZIP. Каждая зависимость Maven однозначно идентифицируется с использованием следующих координат группы, артефакта и версии (GAV):

groupId: идентификатор организации или группы, ответственной за этот проект. Примеры включают `org.hibernate`, `log4j`, `org.springframework` и `com.companyname`.

ArtifactId: идентификатор артефакта, создаваемого проектом. Он должен быть уникальным среди проектов, использующих один и тот же `groupId`. Примеры включают `hibernate-tools`, `log4j`, `spring core` и так далее.

версия: указывает номер версии проекта. Примеры включают `1.0.0`, `2.3.1-SNAPSHOT` и `5.4.2.Final`.

тип: указывает упаковку сгенерированного артефакта. Примеры включают JAR, WAR и EAR.

Артефакты, которые все еще находятся в разработке, помечены `SNAPSHOT` в их версии. Пример версии: `1.0-SNAPSHOT`. Это говорит Maven ежедневно искать обновленную версию артефакта в удаленных репозиториях.

Зависимости объявляются в файле `pom.xml` с помощью тега `dependencies`, как показано ниже:

```
<зависимости>
    <зависимость>
        <groupId>org.hibernate</groupId>
        <artifactId>инструменты гибернации</artifactId>
        <версия>5.4.2.Окончательная версия</версия>
    </зависимость>
</зависимости>
```

Транзитивные зависимости

Зависимости, объявленные в файле `pom.xml` вашего проекта, часто имеют свои собственные зависимости. Такие зависимости называются транзитивными зависимостями. Возьмем пример Hibernate Core. Для правильной работы требуется JBoss Logging, dom4j, javaassist и т. д. Ядро Hibernate, объявленное в вашем файле `pom.xml`, считается прямой зависимостью, а такие зависимости, как dom4j и javaassist, считаются транзитивными зависимостями вашего проекта. Ключевым преимуществом Maven является то, что он автоматически обрабатывает транзитивные зависимости и включает их в ваш проект.

На рис. 3-3 представлен пример транзитивных зависимостей. Заметьте транзитивные зависимости могут иметь свои собственные зависимости. Как вы понимаете, это может быстро усложниться, особенно когда несколько прямых зависимостей извлекают разные версии одного и того же JAR-файла.

Maven использует технику, известную как посредничество зависимостей, для решения конфликты версий. Проще говоря, посредничество в зависимости позволяет Maven извлекать зависимость, ближайшую к проекту в дереве зависимостей. На рис. 3-3 представлены две версии зависимости B: 0.0.8 и 1.0.0. В этом сценарии в проект включена версия 0.0.8 зависимости B, поскольку она является прямой зависимостью и ближе всего к дереву. Теперь посмотрим на три версии зависимости F: 0.1.3, 1.0.0 и 2.2.0. Все три

Глава 3 Управление зависимостями Maven

зависимости находятся на той же глубине. В этом случае Maven будет использовать первую найденную зависимость, то есть 0.1.3, а не последнюю версию 2.2.0. Если вы хотите, чтобы Maven использовал последнюю версию 2.2.0 артефакта F, вам необходимо явно добавить зависимость этой версии в файл `pom.xml`.

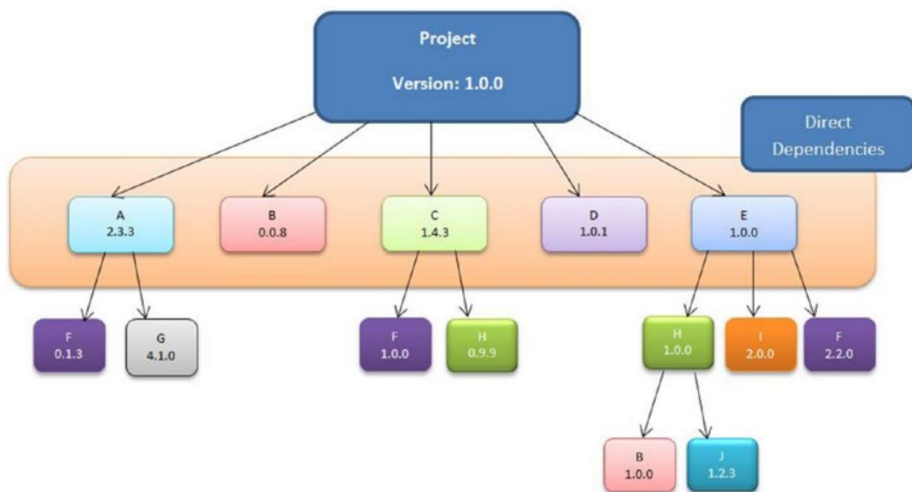


Рисунок 3-3. Транзитивные зависимости

Хотя транзитивные зависимости очень полезны, они могут вызвать проблемы и непредсказуемые побочные эффекты, так как вы можете в конечном итоге включить нежелательные файлы JAR или более старые версии файлов JAR. Maven предоставляет удобный подключаемый модуль зависимостей, который позволяет визуализировать дерево зависимостей проекта. Листинг 3-2 показывает результат выполнения цели дерева зависимостей в примере проекта. Вы можете видеть, что проект зависит от версии 4.11 JAR-файла JUnit. Сам JUnit JAR зависит от версии 1.3 JAR-файла hamcrest.

Листинг 3-2. Подключаемый модуль дерева зависимостей Maven

```
[sudha]$mvn зависимость:дерево
[INFO] Поиск проектов...
[INFO] --- maven-dependency-plugin:2.8:tree (default-cli) @ gswm
```

```
[ИНФО] com.apress.gswmbook:gswm:jar:1.0.0-SNAPSHOT
```

```
[INFO] \- junit:junit:jar:4.11:test
```

```
[INFO] \- org.hamcrest:hamcrest-core:jar:1.3:test
```

```
[ИНФОРМАЦИЯ] -----
```

```
[ИНФО] ДОСТИГАЙТЕ УСПЕХА
```

Бывают случаи, когда вы не хотите включать определенные JAR-файлы транзитивных зависимостей в окончательный архив. Например, при развертывании приложения внутри контейнера, такого как Tomcat или WebLogic, вы можете захотеть исключить определенные файлы JAR, такие как `servlet-api` или `javaee-api`, поскольку они будут конфликтовать с версиями, загружаемыми контейнерами. Maven предоставляет тег «исключает», чтобы исключить транзитивную зависимость. В листинге 3-3 показан код для исключения библиотеки hamcrest из зависимости JUnit. Как видите, элемент исключения принимает координаты groupId и ArtifactId той зависимости, которую вы хотите исключить.

Листинг 3-3. Зависимость JUnit с исключением

```
<зависимости>
  <зависимость>
    <groupId>подразделение</groupId>
    <artifactId>юнит</artifactId>
    <версия>${junit.version}</version>
    <scope>тест</scope>
    <исключения>
      <исключение>
        <groupId>org.hamcrest</groupId>
        <artifactId>подколенный гребень</artifactId>
      </исключение>
    </исключения>
  </зависимость>
</зависимости>
```

Область зависимости

Рассмотрим проект Java, который использует JUnit для модульного тестирования. Файл JAR JUnit, который вы включили в свой проект, нужен только во время тестирования. Вам действительно не нужно объединять JAR JUnit в ваш окончательный производственный архив. Точно так же рассмотрим драйвер базы данных MySQL, файл `mysql-connector-java.jar`. Вам нужен файл JAR, когда вы запускаете приложение внутри контейнера, такого как Tomcat, но не во время компиляции или тестирования кода. Maven использует концепцию области действия, которая позволяет указать, когда и где вам нужна конкретная зависимость.

Maven предоставляет следующие шесть областей:

компиляция: зависимости с областью компиляции доступны в пути к классу на всех этапах сборки, тестирования и запуска проекта. Это область действия по умолчанию.

при условии: зависимости с предоставленной областью действия доступны в пути к классу на этапах сборки и тестирования. Они не объединяются в сгенерированный артефакт. Примеры зависимостей, которые используют эту область, включают API-интерфейс Servlet, API-интерфейс JSP и т. д.

время выполнения: зависимости с областью выполнения недоступны в пути к классу на этапе сборки. Вместо этого они объединяются в сгенерированный артефакт и доступны во время выполнения.

test: Зависимости с областью тестирования доступны на этапе тестирования. JUnit и TestNG — хорошие примеры зависимостей с областью тестирования.

`system`: Зависимости с системной областью аналогичны зависимостям с предоставленной областью, за исключением того, что эти зависимости не извлекаются из репозитория. Вместо этого указывается жестко запрограммированный путь к файловой системе, от которой используются зависимости.

`import`: область импорта применима для `.pom` только файловые зависимости. Он позволяет включать информацию об управлении зависимостями из удаленного файла `.pom`. Область импорта доступна только в Maven 2.0.9 или более поздней версии.

Ручная установка зависимостей

В идеале вы будете извлекать зависимости в своих проектах из общедоступных репозиториях или менеджера корпоративного репозитория. Однако будут случаи, когда вам понадобится архив, доступный в вашем локальном репозитории, чтобы вы могли продолжить разработку. Например, вы можете ждать, пока ваши системные администраторы добавят требуемый файл JAR в менеджер вашего корпоративного репозитория.

Maven предоставляет удобный способ установки архива в локальный репозиторий с установочным плагином. Листинг 3-4 устанавливает файл `test.jar`, расположенный в папке `c:\apress\gswm-book\chapter3`.

Листинг 3-4. Установка зависимости вручную

```
C:\apress\gswm-book\chapter3>mvn install:install-file  
-DgroupId=com.apress.gswmbook -DartifactId=test -Dversion=1.0.0 -Dfile=C:\apress\gswm-  
book\chapter3\test.jar -Dpackaging=jar -DgeneratePom=true
```

[INFO] Поиск проектов...

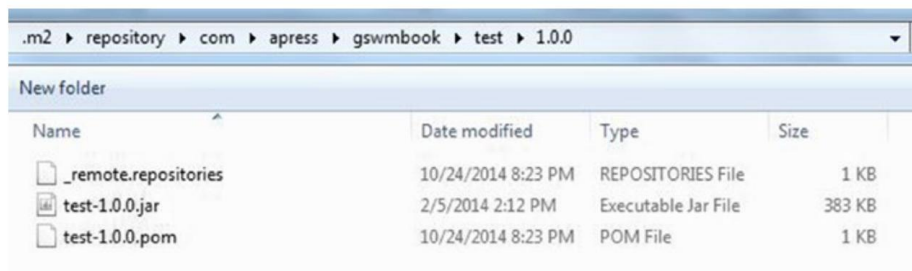
Глава 3 Управление зависимостями Maven

[ИНФОРМАЦИЯ]

[ИНФО] -----< org.apache.maven:standalone-pom >----- [ИНФО] Сборка проекта
 Maven Stub (без POM) 1 [ИНФО] -----[пом]----- --
 [ИНФОРМАЦИЯ]

[ИНФОРМАЦИЯ] --- maven-install-plugin:2.4:install-file (default-cli) @ standalone-pom
 --- [ИНФОРМАЦИЯ] Установка C:\apress\gswm-book\chapter3\test.jar на C:\
 Users\bavara\.m2\repository\com\apress\gswmbook\test\1.0.0\ test-1.0.0.jar [ИНФО]
 Установка C:\Users\bavara\AppData\Local\Temp\mvninstall5971068007426768105.pom
 в C:\Users\bavara\.m2\repository\com\apress\gswmbook\test\1.0.0\test-1.0.0.pom
 [INFO] ----- [ИНФОРМАЦИЯ] ПОСТРОИТЬ УСПЕХ
 [ИНФОРМАЦИЯ] -- ----- [ИНФО] Общее время: 0,439
 с [ИНФО] Завершено: 2019-09-01T00:05:21-06:00 [ИНФО] -----

Увидев сообщение BUILD SUCCESS, вы можете проверить установку
 перейдя в локальный репозиторий Maven, как показано на рисунке 3-4.



Name	Date modified	Type	Size
_remote.repositories	10/24/2014 8:23 PM	REPOSITORIES File	1 KB
test-1.0.0.jar	2/5/2014 2:12 PM	Executable Jar File	383 KB
test-1.0.0.pom	10/24/2014 8:23 PM	POM File	1 KB

Рисунок 3-4. Зависимость добавлена в репозиторий

Резюме

Управление зависимостями лежит в основе Maven. Каждый нетривиальный проект Java опирается на открытый исходный код или внешние артефакты, а управление зависимостями Maven автоматизирует процесс извлечения этих артефактов и включения их на нужных этапах процесса сборки. Вы также узнали, что Maven использует координаты GAV для идентификации своих артефактов.

В следующей главе вы узнаете об организации базового проекта Maven.

ГЛАВА 4

Основы проекта Maven

Maven предоставляет соглашения и стандартный макет каталога для всех своих проектов. Как обсуждалось в главе 1, эта стандартизация обеспечивает единый интерфейс сборки, а также облегчает разработчикам переход от одного проекта к другому. В этой главе объясняются основы проекта Maven и файла `pom.xml`.

Базовая организация проекта

Лучший способ понять структуру проекта Maven — посмотреть на него.

Рисунок 4-1 иллюстрирует простейший проект Java на основе Maven.



Рисунок 4-1. Структура проекта Maven Java

Глава 4. Основы проекта Maven

Теперь давайте рассмотрим каждый из компонентов в проекте:

- `Gswm` — это корневая папка проекта. Как правило, имя корневой папки совпадает с именем сгенерированный артефакт.
- Папка `src` содержит связанные с проектом артефакты, такие как исходный код или файлы свойств, которыми обычно требуется управлять в системе управления исходным кодом (SCM), такой как SVN или Git.
- Папка `src/main/java` содержит файлы Java. исходный код.
- Папка `src/test/java` содержит код модульного теста Java.
- Целевая папка содержит сгенерированные артефакты, такие как .класс файлы. Сгенерированные артефакты обычно не хранятся в SCM, поэтому вы не фиксируете целевую папку и ее содержимое в SCM.
- Файл `LICENSE.txt` содержит информацию о лицензии. связанные с проектом.
- Файл `README.txt` содержит информацию/инструкции о проекте.
- Файл `NOTICE.txt` содержит уведомления, необходимые для сторонних библиотек, используемых в этом проекте.
- Каждый проект Maven имеет файл `pom.xml` в корне проекта. Он содержит информацию о проекте и конфигурации, такую как зависимости и плагины.

Помимо каталогов `src/main` и `src/test`, Maven

рекомендует несколько других каталогов для хранения дополнительных файлов и ресурсов.

В Таблице 4-1 перечислены эти каталоги вместе с их содержимым.

Таблица 4-1. Каталоги Maven

Имя каталога	Описание
источник/основной/ресурсы	Содержит ресурсы, такие как файлы конфигурации Spring и шаблоны скорости, которые должны попасть в сгенерированный артефакт.
источник/основной/конфигурации	Содержит файлы конфигурации, такие как файлы контекста Tomcat, Файлы конфигурации James Mail Server и так далее. Эти файлы не попадут в сгенерированный артефакт.
источник/основной/скрипты	Содержит любые сценарии, необходимые системным администраторам и разработчикам для приложения.
источник/тест/ресурсы	Содержит файлы конфигурации, необходимые для тестирования.
источник/основной/веб-приложение	Содержит веб-ресурсы, такие как файлы .jsp, таблицы стилей и изображения.
источник/это	Проводит интеграционные тесты для приложения.
источник/основной/дб	Содержит файлы базы данных, такие как сценарии SQL.
источник/сайт	Содержит файлы, необходимые при создании сайта проекта.

Maven предоставляет архетипы (как обсуждалось в главе 6) для быстрой загрузки проектов. Однако в этой главе вы вручную соберете Java-проект на основе Maven. Используйте следующие инструкции для создания проекта:

1. С помощью командной строки перейдите в папку, в которой вы хотите создать проект. В этой книге мы предполагаем, что это каталог `c:\apress\gswm-book\`.

Глава 4.

2. Запустите команду `mkdir gswm`.

Глава 4. Основы проекта Maven

3. перейдите во вновь созданный каталог и создайте пустой файл `pom.xml`.
4. Создайте каталог `src` в `gswm`, затем создайте основной каталог в `src` и, наконец, создайте папку `java`. каталог под `main`.

Начальная структура проекта должна напоминать структуру, показанную на рис. 4-2.

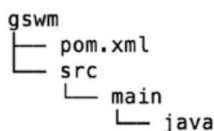


Рисунок 4-2. Стартовая структура проекта

Понимание файла `pom.xml`

Файл `pom.xml` — самый важный файл в проекте Maven. Как мы уже обсуждали в книге, файл `pom.xml` содержит информацию о конфигурации, необходимую Maven. В листинге 4-1 показан файл `pom.xml` с основной информацией о проекте. Запускаем файл `pom.xml` с проектом

элемент. Затем мы предоставляем идентификатор группы, идентификатор артефакта и версию.

координаты. Элемент упаковки сообщает Maven, что для этого проекта необходимо создать архив JAR. Наконец, мы используем элемент разработчиков, чтобы добавить информацию о разработчиках, которые работают над этим проектом.

Листинг 4-1. Конфигурация файла `pom.xml`

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

```

```

<groupId>com.apress.gswmbook</groupId>
<artifactId>gswm</artifactId> <version>1.0.0-
SNAPSHOT</version>
<packaging>банка</packaging>

<name>Начало работы с Maven</name>
<url>http://apress.com</url>

<developers>
  <developer>
    <id>balaji</id>
    <name>Balaji Varanasi</name>
    <email>balaji@inflinx.com</email>
    <properties> <active>true</active>

    </свойства>
  </developer>
  <developer>
    <id>судха</id>
    <name>Судха Белида</name>
    <email>sudha@inflinx.com</email>
    <properties> <active>true</active>

    </properties> </
  developer> </
</developers> </project>

```

Позже мы рассмотрим другие элементы в файле pom.xml. главе и во всей остальной части книги.

MAVEN ВЕРСИИ

В проектах Maven рекомендуется использовать следующие соглашения для управления версиями:

`<основная-версия>.<дополнительная-версия>.<дополнительная-версия>-квалификатор`

Основные, второстепенные и добавочные значения являются числовыми, а квалификатор может иметь такие значения, как RC, alpha, beta и SNAPSHOT.

Некоторые примеры, которые следуют этому соглашению: 1.0.0, 2.4.5-SNAPSHOT, 3.1.1-RC1 и т. д.

Классификатор SNAPSHOT в версии проекта имеет особое значение. Это говорит о том, что проект находится в стадии разработки. Когда в проекте используется зависимость SNAPSHOT, при каждой сборке проекта Maven будет извлекать и использовать последний артефакт SNAPSHOT.

Большинство менеджеров репозитория принимают выпускные сборки только один раз. Однако, когда вы разрабатываете приложение в среде непрерывной интеграции, вы хотите часто выполнять сборку и передавать последнюю сборку менеджеру репозитория.

Таким образом, рекомендуется добавлять к вашей версии суффикс SNAPSHOT во время разработки.

Создание проекта

Прежде чем мы рассмотрим создание проекта, давайте добавим класс Java HelloWorld в папку `src/main/java`. В листинге [4-2](#) показан код класса HelloWorld.

Листинг 4-2. Код для класса Java HelloWorld

```
открытый класс HelloWorld {  
    public void sayHello () {  
        System.out.print("Привет, мир");  
    }  
}
```

На рис. 4-3 показана структура проекта после добавления класса.

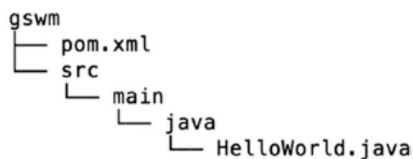


Рисунок 4-3. Структура проекта с добавленным классом Java

Теперь вы готовы собрать приложение, поэтому давайте запустим пакет mvn. от gswm. Вы должны увидеть вывод, аналогичный показанному в листинге 4-3.

Листинг 4-3. Выходные данные для команды пакета Maven для сборки приложения

```
C:\apress\gswm-book\chapter4\gswm>mvn package
```

```
[INFO] Поиск проектов...
```

```
[ИНФОРМАЦИЯ]
```

```
[ИНФОРМАЦИЯ] ----- [ИНФОРМАЦИЯ]
```

```
Начало работы с Maven 1.0.0-SNAPSHOT [ИНФОРМАЦИЯ] -----
```

```
-----
```

```
.....
```

```
[INFO] Компиляция 1 исходного файла в C:\apress\gswm-book\chapter4\
gswm\target\classes
```

```
.....
```

```
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ gswm --- [INFO] Сборка
jar: C:\apress\gswm-book\chapter4\gswm\target\ gswm-1.0. 0-SNAPSHOT.jar
```

```
[ИНФОРМАЦИЯ] ----- [ИНФО]
```

ДОСТИГАЙТЕ УСПЕХА

Глава 4. Основы проекта Maven

Примечание. Если вы запускаете Maven впервые, он загрузит подключаемые модули и зависимости, необходимые для его запуска. Таким образом, ваша первая сборка может занять больше времени, чем вы ожидаете.

Суффикс пакета после команды `mvn` — это фаза Maven, которая компилирует код Java и упаковывает его в файл JAR. Упакованный JAR-файл оказывается в папке `gswm\target`, как показано на рис. 4-4.

Local Disk (C:) > apress > gswm-book > chapter4 > gswm > target

Name	Type	Date
 classes	File folder	9/1/
 maven-archiver	File folder	9/1/
 maven-status	File folder	9/1/
 gswm-1.0.0-SNAPSHOT.jar	Executable Jar File	9/1/

Рисунок 4-4. Упакованный JAR, расположенный в целевой папке

Тестирование проекта

Теперь, когда вы завершили сборку проекта, давайте добавим тест JUnit, который проверяет метод `sayHello()`. Начнем этот процесс с добавления зависимости JUnit в файл `pom.xml`. Это достигается с помощью элемента зависимостей.

В листинге 4-4 показан обновленный файл `pom.xml` с зависимостью JUnit.

Листинг 4-4. Обновленный POM с зависимостью JUnit

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
```



```

<groupId>com.apress.gswmbook</groupId>
<artifactId>gswm</artifactId> <version>1.0.0-
SNAPSHOT</version>

<packaging>банка</packaging>

<name>Начало работы с Maven</name> <url>http://
apress.com</url>

<developers>
  <developer>
    <id>balaji</id>
    <name>Balaji Varanasi</name>
    <email>balaji@inflinx.com</email>
    <properties>
      <active>true</active>
    </свойства>
  </developer>
  <developer>
    <id>судха</id>
    <name>Судха Белида</name>
    <email>sudha@inflinx.com</email>
    <properties> <active>true</active>

    </properties> </
  developer> </
</developers>

<зависимости>
  <зависимость>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>

```

Глава 4. Основы проекта Maven

```

        <версия>4.12</версия>
        <scope>тест</scope>
    </зависимость>
</зависимости>

</проект>

```

Обратите внимание, что вы использовали тест области, указывающий, что файл JUnit.jar необходим только на этапе тестирования. Убедимся, что эта зависимость успешно добавлена, запустив `mvn dependency:tree` в командной строке. В листинге 4-5 показан результат этой операции.

Листинг 4-5. Вывод команды дерева Maven

```

C:\apress\gswm-book\chapter4\gswm> mvn зависимость: дерево
[INFO] --- maven-dependency-plugin:2.8:tree (default-cli) @ gswm ---

[ИНФО] com.apress.gswmbook:gswm:jar:1.0.0-SNAPSHOT
[INFO] \- junit:junit:jar:4.12:test
[INFO] \- org.hamcrest:hamcrest-core:jar:1.3:test
[ИНФОРМАЦИЯ] -----
[ИНФО] ДОСТИГАЙТЕ УСПЕХА

```

Цель дерева в подключаемом модуле зависимостей отображает зависимости проекта в виде дерева. Обратите внимание, что зависимость JUnit использует транзитивную зависимость с именем hamcrest, проект с открытым исходным кодом, упрощающий написание объектов сопоставления.

Теперь, когда у вас есть зависимость JUnit в пути к классу, давайте добавим в проект модульный тест HelloWorldTest.java. Создайте папки test/java под src и добавьте под ним HelloWorldTest.java. Обновленная структура проекта показана на рис. 4-5.



Рисунок 4-5. Структура Maven с тестовым классом

Исходный код HelloWorldTest показан в листинге [4-6](#).

Листинг 4-6. Код для класса Java HelloWorldTest

```
импортировать java.io.ByteArrayOutputStream;
```

```
импортировать java.io.PrintStream;
```

```
импортировать org.junit.After;
```

```
импортировать org.junit.Assert;
```

```
импортировать org.junit.Before;
```

```
импортировать org.junit.Test;
```

```
открытый класс HelloWorldTest {
```

```
    закрытый окончательный ByteArrayOutputStream outStream =
    new ByteArrayOutputStream();
```

```
    @Before
```

```
    public void setUp()
```

```
    { System.setOut(new PrintStream(outStream));
```

```
    }
```

```
    @Test
```

```
    public void testSayHello() { HelloWorld
```

```
        hw = new HelloWorld(); hw.sayHello();
```

Глава 4. Основы проекта Maven

```

        Assert.assertEquals("Hello World", outputStream.toString());

    }

    @After
    public void cleanUp()
    { System.setOut(null);
    }

}

```

Теперь у вас все настроено в этом проекте, так что вы можете еще раз запустить пакет mvn. После запуска вы увидите вывод, аналогичный показанному в листинге [4-7](#).

Листинг 4-7. Выходные данные для команды Maven для сборки проекта

```

C:\apress\gswm-book\chapter4\gswm>mvn package
[INFO] --- maven-compiler-plugin:2.5.1:compile (компиляция по
умолчанию) @ gswm --- [INFO] Нечего компилировать - все
классы обновлены [INFO]

[INFO] --- maven-resources-plugin:2.6:testResources (testResources по
умолчанию) @ gswm ---

[INFO] Каталог отчетов Surefire: C:\apress\gswm-book\chapter4\
gswm\target\surefire-reports

ИСПЫТАНИЯ

Запуск HelloWorldTest
Тесты запущены: 1, сбоев: 0, ошибок: 0, пропущено: 0, истекшее время:
0,038 с

```

```
Полученные результаты :

Запуск тестов: 1, сбоев: 0, ошибок: 0, пропущено: 0

[ИНФОРМАЦИЯ]

[INFO] --- maven-jar-plugin:2.4:jar (jar по умолчанию) @ gswm ---
[INFO] Сборка JAR: C:\apress\gswmbook\chapter4\gswm\target\
gswm-1.0.0-SNAPSHOT.jar
[ИНФОРМАЦИЯ] -----
[ИНФО] ДОСТИГАЙТЕ УСПЕХА
```

Обратите внимание на раздел «Тесты» в листинге 4-7. Это показывает, что Maven запустил тест и успешно завершил его.

Рисунок 4-6 показывает обновленную целевую папку. Вы можете видеть, что вы сейчас имеете папку тестовых классов с соответствующими отчетами в этой папке.

Local Disk (C:) > apress > gswm-book > chapter4 > gswm > target		
Name	Type	Date
 classes	File folder	9/1/:
 maven-archiver	File folder	9/1/:
 maven-status	File folder	9/1/:
 surefire-reports	File folder	9/1/:
 test-classes	File folder	9/1/:
 gswm-1.0.0-SNAPSHOT.jar	Executable Jar File	9/1/:

Рисунок 4-6. Целевая папка с тестовыми классами

Свойства в pom.xml

Maven предоставляет заполнители свойств АКА, которые можно использовать внутри файла pom.xml. Свойства Maven упоминаются в файле pom.xml с использованием нотации \${property_name}. Существует два типа свойств — неявные и определяемые пользователем свойства.

Глава 4. Основы проекта Maven

Неявные свойства

Неявные свойства — это свойства, доступные по умолчанию для любого проекта Maven. Например, Maven предоставляет свойства своей объектной модели проекта, используя «проект». префикс. Чтобы получить доступ к значению `ArtifactId` внутри файла `pom.xml`, вы можете использовать `${project.artifactId}`, как показано ниже:

```
<сборка>
    <finalName>${project.artifactId}</finalName>
</сборка>
```

Аналогично, для доступа к свойствам из файла `settings.xml` вы можете использовать настройки." префикс. Наконец, «env.» префикс может использоваться для доступа к значениям переменных среды. Например, `${env.PATH}` вернет значение переменной окружения `PATH`.

Пользовательские свойства

Maven позволяет объявлять пользовательские свойства в файле `pom.xml` с помощью элемента `<properties />`. Эти свойства очень полезны для объявления версий зависимостей. В листинге 4-8 показан обновленный файл `pom.xml`. файл с версией JUnit, объявленной как свойство. Это особенно полезно, когда `pom.xml` имеет много зависимостей и вам нужно узнать или изменить версию конкретной зависимости.

Листинг 4-8. Файл `pom.xml` со свойствами

```
<проект xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
    maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```

<groupId>com.apress.gswmbook</groupId> <!--
  Удалено для краткости -->

<properties>
  <junit.version>4.12</junit.version> </properties>

<developers>
  <!-- Удалено для краткости --> </
developers> <dependencies> <dependency>

  <groupId>junit</groupId>
  <artifactId>junit</artifactId> <version>$
  {junit.version}</version> <scope>test</scope>
  </dependency> </dependencies> </project>

```

Резюме

CoC Maven предписывает стандартный макет каталога для всех своих проектов. Он предоставляет несколько удобных каталогов, таких как `src\main\java` и `src\test`, а также рекомендации по содержимому каждого из них. Вы узнали об обязательном файле `pom.xml` и некоторых его элементах, которые используются для настройки поведения проекта Maven.

В следующей главе вы рассмотрите жизненный цикл Maven, подключаемые модули, этапы сборки, цели и способы их эффективного использования.

ГЛАВА 5

Жизненный цикл Мавена

Центральное место в Maven занимает его жизненный цикл, обеспечивающий единый интерфейс для создания и распространения проектов. В этой главе мы рассмотрим жизненный цикл и строительные блоки, составляющие жизненный цикл.

Цели и плагины

Процессы сборки, создающие такие артефакты, как файлы JAR или WAR, обычно требуют успешного выполнения нескольких шагов и задач в четко определенном порядке. Примеры таких задач включают компиляцию исходного кода, выполнение модульных тестов и упаковку артефакта. Maven использует концепцию целей для представления таких детальных задач.

Чтобы лучше понять, что такое цель, давайте рассмотрим пример.

В листинге 5-1 показана цель компиляции, выполняемая для кода проекта gswm в папке C:\apress\gswm-book\chapter5\gswm. Как следует из названия, компиляция цель компилирует исходный код. Цель компиляции идентифицирует класс Java HelloWorld.java в папке src/main/java, компилирует его и помещает скомпилированный файл класса в папку target\classes.

Листинг 5-1. Цель компиляции Maven

```
C:\apress\gswm-book\chapter5\gswm> компилятор mvn: скомпилировать
[INFO] Поиск проектов...
[INFO] --- maven-compiler-plugin:3.1:compile (default-cli) @ gswm ---
```


Глава 5 Жизненный цикл Maven

```
[INFO] Компиляция 1 исходного файла в C:\apress\gswm-book\chapter5\
gswm\цель\классы
```

```
[ИНФОРМАЦИЯ] -----
```

```
[ИНФО] ДОСТИГАЙТЕ УСПЕХА
```

```
[ИНФОРМАЦИЯ] -----
```

Цели в Maven упакованы в плагины, которые, по сути, представляют собой набор одной или нескольких целей. В листинге 5-1 компилятор — это подключаемый модуль, обеспечивающий цель компиляции.

В листинге 5-2 представлена довольно изящная цель, которая называется clean. Как уже упоминалось ранее целевая папка содержала временные файлы и артефакты, созданные Maven. Бывают случаи, когда целевая папка становится огромной или когда определенные файлы, которые были кэшированы, необходимо удалить из папки. Чистая цель достигает именно этого, поскольку пытается удалить целевую папку и все ее содержимое.

Листинг 5-2. Maven чистая цель

```
C:\apress\gswm-book\chapter5\gswm>mvn clean:clean
```

```
[INFO] Поиск проектов...
```

```
[INFO] --- maven-clean-plugin:2.5:clean (default-cli) @ gswm ---
```

```
[INFO] Удаление C:\apress\gswm-book\chapter5\gswm\target
```

```
[ИНФОРМАЦИЯ] -----
```

```
[ИНФО] ДОСТИГАЙТЕ УСПЕХА
```

```
[ИНФОРМАЦИЯ] -----
```

Обратите внимание на формат команды clean:clean в листинге 5-2. Чистый перед двоеточием (:) представляет чистый подключаемый модуль, а чистый следующее за двоеточием представляет чистую цель. К настоящему моменту должно быть очевидно, что для запуска цели в командной строке требуется следующий синтаксис:

```
mvn plugin_identifier:goal_identifier
```

Maven предоставляет готовый подключаемый модуль справки, который можно использовать для перечисления доступных целей в данном подключаемом модуле. В листинге 5-3 показана цель описания подключаемого модуля справки для отображения целей внутри подключаемого модуля компилятора.

Листинг 5-3. Подключаемый модуль справки Maven

```
mvn help:describe -Dplugin=компилятор
```

[INFO] Поиск проектов...

Название: Плагин компилятора Apache Maven

Описание: Плагин компилятора используется для компиляции исходников вашего проекта.

Идентификатор группы: org.apache.maven.plugins

Идентификатор артефакта: maven-compiler-plugin

Версия: 3.8.1

Префикс цели: компилятор

Этот плагин преследует 3 цели:

компилятор:компилировать

Описание: Компилирует исходники приложения.

компилятор: помощь

Описание: Показать справочную информацию о плагине maven-compiler.

Вызовите компилятор `mvn:help -Ddetail=true -Dgoal=<goal-name>`, чтобы отобразить сведения о параметрах.

компилятор: testCompile

Описание: Компилирует исходники тестов приложений.

Плагины и их поведение можно настроить с помощью раздела плагинов `pom.xml`. Рассмотрим случай, когда вы хотите, чтобы ваш проект был скомпилирован с помощью Java 8. Начиная с версии 3.8, подключаемый модуль компилятора Maven компилирует код для Java 1.6. Таким образом, вам потребуется изменить поведение этого подключаемого модуля в файле `pom.xml`, как показано в листинге 5-4.

Глава 5 Жизненный цикл Maven

Листинг 5-4. Подключаемый элемент в файле pom.xml

```

<проект xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-экземпляр"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd"> <modelVersion>4.0.0</
modelVersion >

<!-- Детали проекта для краткости опущены -->

<зависимости>
  <!-- Детали зависимости опущены для краткости -->
</зависимости>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId> <version>3.8.1</
version> <configuration>

        <источник>1.8</источник>
        <target>1.8</target> </
configuration> </plugin> </
plugins> </build> </project>

```

Теперь, если вы запустите команду `mvn compiler:compile`,
сгенерированные файлы классов будут иметь версию Java 1.8.

Примечание. У элемента `<build />` в `pom.xml` есть очень полезный дочерний элемент с именем `finalName`. По умолчанию имя сгенерированного Maven артефакта следует за `<<project_artifact_id>>-<<версия_проекта>>` формат. Однако иногда вы можете захотеть изменить имя сгенерированного артефакта без изменения артефакта. Этого можно добиться, объявив элемент `finalName` как `<finalName>new_name</finalName>`.

Жизненный цикл и этапы

Цели Maven детализированы и обычно выполняют одну задачу. Для выполнения сложных операций, таких как создание артефактов или документация, необходимо упорядоченное выполнение нескольких целей. Maven упрощает эти сложные операции с помощью абстракций жизненного цикла и фаз, так что операции, связанные со сборкой, можно выполнять с помощью нескольких команд.

Жизненный цикл сборки Maven представляет собой серию этапов, которые выполняются в том же порядке, независимо от создаваемого артефакта. Maven называет этапы жизненного цикла фазами. Каждый проект Maven имеет следующие три встроенных жизненных цикла:

- по умолчанию: этот жизненный цикл обрабатывает компиляцию, упаковку и развертывание проекта Maven.

- `clean`: этот жизненный цикл обрабатывает удаление временных файлов и сгенерированных артефактов из целевого каталога.

- `site`: этот жизненный цикл обрабатывает создание документации и создание сайта.

Глава 5 Жизненный цикл Maven

Чтобы лучше понять жизненный цикл сборки и его этапы, давайте рассмотрим некоторые фазы, связанные с жизненным циклом по умолчанию:

`validate`: запускает проверки, чтобы убедиться, что проект правильный и что все зависимости загружены и доступны.

`compile`: компилирует исходный код.

`test`: запускает модульные тесты с использованием фреймворков. Этот шаг не требует, чтобы приложение было упаковано.

`package`: собирает скомпилированный код в распространяемый формат, такой как JAR или WAR.

`install`: устанавливает упакованный архив в локальный репозиторий. Архив теперь доступен для использования любым проектом, работающим на этой машине.

`deploy`: отправляет созданный архив в удаленный репозиторий для использования другими командами и членами команды.

Жизненный цикл Maven является абстрактной концепцией и не может быть выполнен напрямую. Вместо этого вы выполняете одну или несколько фаз. Например, команда `mvn package` выполнит фазу пакета жизненного цикла по умолчанию. Помимо четкого определения порядка фаз в жизненном цикле, Maven также автоматически выполняет все фазы до запрошенной фазы. Таким образом, при запуске команды `mvn package` Maven запустит все предыдущие этапы, такие как компиляция и тестирование.

На каждом этапе необходимо выполнить ряд задач. Для этого каждая фаза связана с нулем или более целями. Фаза просто делегирует эти задачи связанным с ней целям. На рис. 5-1 показана связь между жизненным циклом, фазами, целями и подключаемыми модулями.

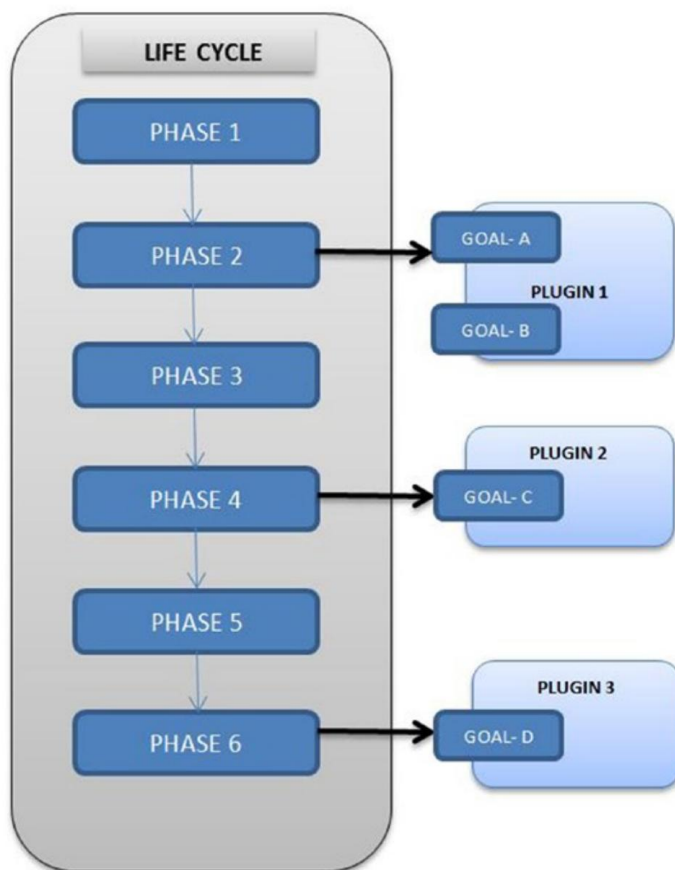


Рисунок 5-1. Связь между жизненным циклом, фазами, целями и подключаемыми модулями

Для фазы Maven допустимо отсутствие связанных с ней целей.

В этом случае Maven пропустит этап выполнения. Такие этапы служат заполнителями для пользователей и сторонних поставщиков, чтобы связать свои собственные созданные цели.

Глава 5 Жизненный цикл Maven

Элемент `<packaging />` в файле `pom.xml` будет автоматически назначить правильные цели для каждой из фаз без дополнительной настройки. Помните, что это преимущество CoS. Например, если элементом упаковки является `jar`, то фаза `package` будет привязана к `jar`. цель в плагине `jar`. Точно так же для артефакта `WAR` файл `pom.xml` привязывает пакет к цели войны в подключаемом модуле войны. На рис. 5-2 показана часть внутреннего жизненного цикла, связанная с проектом `WAR`.

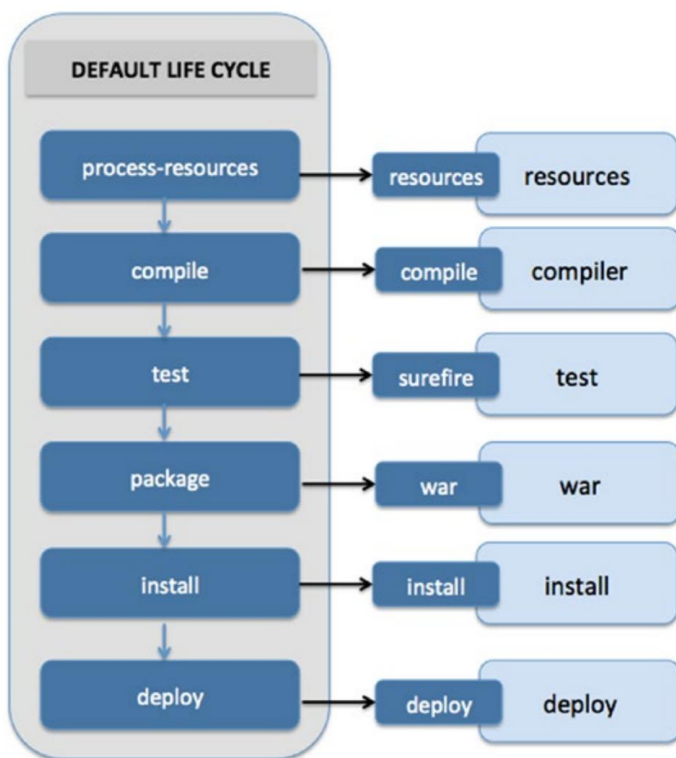


Рисунок 5-2. Жизненный цикл по умолчанию для проекта WAR

ПРОПУСК ИСПЫТАНИЙ

Как обсуждалось ранее, когда вы запускаете фазу пакета, также запускается фаза тестирования и выполняются все модульные тесты. Если на этапе тестирования возникают какие-либо сбои, сборка не выполняется. Это желаемое поведение. Однако бывают случаи, когда, например, при работе с унаследованным проектом вы хотели бы пропустить компиляцию и выполнение тестов, чтобы успешно построить проект. Вы можете добиться этого, используя свойство `maven.test.skip`. Вот пример использования этого свойства:

```
пакет mvn -Dmaven.test.skip=true
```

Разработка плагинов

Разработка пользовательских плагинов для Maven очень проста. Как обсуждалось ранее, плагин — это просто набор целей. Таким образом, когда мы говорим о разработке плагинов, мы, по сути, говорим о разработке целей. В Java эти цели реализуются с помощью MOJO, что означает Maven Old Java Object и похоже на Plain Old Java Object (POJO) в Java.

В этом разделе объясняется, как разработать подключаемый модуль `SystemInfoPlugin`, отображающий системную информацию, такую как версия Java, операционная система и т. п., на консоли с запущенной командой Maven.

Давайте начнем разработку этого подключаемого модуля с создания Java-проекта Maven с именем `gswm-maven-plugin`, как показано на рис. 5-3.

Глава 5 Жизненный цикл Maven

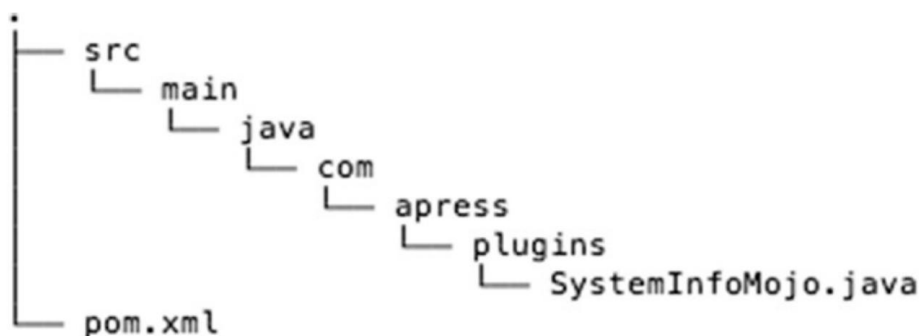


Рисунок 5-3. Проект Maven для разработки плагинов

Примечание. В этой главе мы вручную создаем проект подключаемого модуля.

Maven предоставляет `maven-archetype-mojo`, который поможет ускорить разработку вашего плагина. Мы узнаем об архетипах Maven в главе 6.

Содержимое файла `pom.xml` показано в листинге 5-5. Обратите внимание, что тип упаковки — `maven-plugin`. Мы добавили зависимости `maven-plugin-api` и `maven-plugin-annotations`, потому что они нужны для разработки плагинов. Мы будем использовать Apache Commons Lang для получения информации о системе. Следовательно, мы также добавили зависимость Apache Commons Lang 3.

Листинг 5-5. `pom.xml` с зависимостями

```

<?xml версия="1.0" кодировка="UTF-8"?>
<проект>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.apress.plugins</groupId>
  <artifactId>gswm-maven-plugin</artifactId>
  <версия>1.0.0</версия>
  <packaging>maven-плагин</packaging>
  <description>Плагин информации о системе</description>

```

```

<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
</свойства>
<dependencies>
  <dependency>
    <groupId>org.apache.maven</groupId>
    <artifactId>maven-plugin-api</artifactId>
    <version>3.6.1</version>
  </dependency>
  <dependency>
    <groupId>org.apache.maven.plugin-tools</groupId>
    <artifactId>maven-plugin-annotations</artifactId>
    <version>3.6.0</version> <scope>provided< /scope> </
  dependency> <dependency> <groupId>org.apache.commons</
groupId> <artifactId>commons-lang3</artifactId>
<version>3.9</version>

</dependency>
</dependencies>
<!-- Используйте последнюю версию плагина -->
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-plugin-plugin</artifactId>
      <version>3.6.0</version>
    </плагин>
  </plugins>
</build>

```

Глава 5 Жизненный цикл Maven

```

        </плагины>
    </сборка>

</проект>

```

Следующим шагом в процессе разработки является создание МОЈО. В листинге 5-6 показан код для SystemInfoMojo. Аннотация @Mojo помечает класс SystemInfoMojo как Mojo с «systeminfo» в качестве имени цели. Метод execute содержит эту логику цели. В SystemInfoMojo мы просто выводим на консоль несколько фрагментов системной информации.

Листинг 5-6. Java-класс SystemInfoMojo

```

пакет com.apress.plugins;

импортировать org.apache.commons.lang3.SystemUtils;
импортировать org.apache.maven.plugin.AbstractMojo;
импортировать org.apache.maven.plugin.MojoExecutionException;
импортировать org.apache.maven.plugin.MojoFailureException;
импортировать org.apache.maven.plugins.annotations.Mojo;

@Mojo (имя = "информация о системе")
открытый класс SystemInfoMojo расширяет AbstractMojo {

    @Override
    public void execute() выдает исключение MojoExecutionException,
    исключение MojoFailureException {
        getLog().info( "Домашняя          + SystemUtils.JAVA_HOME);
        страница Java: getLog().info( "Версия Java: "+ SystemUtils.JAVA_
        ВЕРСИЯ);
        getLog().info( "Имя ОС: + SystemUtils.OS_NAME");
        getLog().info( "Версия ОС: + SystemUtils.OS_
        ВЕРСИЯ);
        getLog().info("Имя пользователя:      + SystemUtils.ИМЯ_ПОЛЬЗОВАТЕЛЯ );
    }
}

```

Последним шагом в этом процессе является установка подключаемого модуля в репозиторий Maven. Запустите команду `mvn install` в корне каталога, и вы должны получить результат, показанный в листинге 5-7.

Листинг 5-7. Команда установки Maven

```
C:\apress\gswm-book\chapter5\gswm-maven-plugin>установить mvn
[INFO] Поиск проектов...
[ИНФОРМАЦИЯ]
[ИНФОРМАЦИЯ] -----< com.apress.plugins:gswm-maven-plugin>-----
[INFO] Сборка gswm-maven-plugin 1.0.0
[ИНФОРМАЦИЯ] -----[ maven-plugin ]-----
[ИНФОРМАЦИЯ]
[INFO] --- maven-resources-plugin:2.6:resources (ресурсы по умолчанию) @
gswm-maven-plugin

[INFO] Средство извлечения java-аннотаций mojo обнаружило 1 дескриптор mojo.

[INFO] --- maven-install-plugin:2.4:install (установка по умолчанию) @ gswm-maven-
plugin ---
[INFO] Установка C:\apress\gswm-book\chapter5\gswm-maven
plugin\target\gswm-maven-plugin-1.0.0.jar в C:\Users\<<USER_
ИМЯ>>\.m2\repository\com\apress\plugins\gswm-plugin\1.0.0\
gswm-maven-плагин-1.0.0.jar
[INFO] Установка C:\apress\gswm-book\chapter5\gswm-maven plugin\pom.xml
в C:\Users\<<ИМЯ_ПОЛЬЗОВАТЕЛЯ>>\.m2\repository\com\
apress\plugins\gswm-maven-plugin\1.0.0\gswm-maven-plugin--
1.0.0.пом
[ИНФОРМАЦИЯ] -----
[ИНФО] ДОСТИГАЙТЕ УСПЕХА
```

Глава 5 Жизненный цикл Maven

Теперь вы готовы начать использовать этот плагин. Помните, что синтаксис для запуска любой цели: `mvn pluginId:goal-name`. В листинге 5-8 показан этот подключаемый модуль в действии. Обратите внимание на системную информацию, отображаемую на консоли.

Листинг 5-8. Запуск подключаемого модуля `SystemInfoMojo`

```
C:\apress\gswm-book\chapter5\gswm-plugin>mvn com.apress.plugins:gswm-
maven-plugin:systeminfo [INFO] Сканирование проектов...
```

```
[ИНФОРМАЦИЯ] --- gswm-maven-plugin:1.0.0:systeminfo (default-cli) @ gswm-maven-
plugin --- [ИНФОРМАЦИЯ] Домашняя страница Java: C:\java\jdk-11 [ИНФОРМАЦИЯ]
Версия Java : 11.0.1 [INFO] Имя ОС: Windows [INFO] Версия ОС: 10 [INFO] Имя
пользователя: Balaji [INFO] -----
```

Недавно разработанный подключаемый модуль также готов к использованию в других проектах Maven. В листинге 5-9 показана часть POM-файла, которая связывает цель `systeminfo` с фазой проверки.

Листинг 5-9. Файл POM с использованием цели `systeminfo`

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.apress.plugins</groupId>
  <artifactId>gswm-plugin-test</artifactId> <version>1.0.0</
version> <packaging> jar</packaging> <description>Тест
  плагина</description>
```

```

<свойства>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target> </properties>

<зависимости/>

<сборка>
    <plugins>
        <plugin>
            <groupId>com.apress.plugins</groupId>
            <artifactId>gswm-maven-plugin </artifactId>

            <версия>1.0.0</версия>
            <выполнения>
                <исполнение>
                    <фаза>проверка</фаза>
                    <цели> <цель>информация о
                        системе </цель> </
                        цели> </выполнение>

                </исполнение>
            </выполнения>
        </plugin>
    </плагины> </
<сборка>

</проект>

```

Когда вызывается фаза Maven, такая как компиляция или пакет, вы см. вывод цели systeminfo, как показано в листинге [5-10](#).

Глава 5 Жизненный цикл Maven

Листинг 5-10. Вывод фазы компиляции

компиляция mvn

[INFO] Поиск проектов...

[INFO] Сборка gswm-plugin-test 1.0.0

[INFO] Главная страница Java: C:\java\jdk-11

[ИНФОРМАЦИЯ] Версия Java: 11.0.1

[ИНФОРМАЦИЯ] Название ОС: Windows

[ИНФОРМАЦИЯ] Версия ОС: 10

[INFO] Имя пользователя: Баладжи

[INFO] --- maven-resources-plugin:2.6:resources (ресурсы по умолчанию) @ gswm-plugin-test ---

Резюме

Maven использует архитектуру на основе подключаемых модулей, что позволяет легко расширять его функциональность. Каждый подключаемый модуль представляет собой набор одной или нескольких целей, которые можно использовать для выполнения задач, таких как компиляция исходного кода или выполнение тестов.

Maven связывает цели с фазами. Фазы обычно выполняются в последовательности как часть жизненного цикла сборки. Вы также изучили основы создания плагина.

В следующей главе вы познакомитесь с архетипами и узнаете о многомодульных проектах.

ГЛАВА 6

Архетипы Мавен

До этого момента в книге вы создавали проекты Maven вручную, генерируя папки и создавая файлы `pom.xml` с нуля. Это может стать утомительным, особенно если вам часто приходится создавать проекты.

Для решения этой проблемы Maven предоставляет архетипы. Архетипы Maven — это шаблоны проектов, которые позволяют пользователям легко создавать новые проекты.

Архетипы Maven также предоставляют прекрасную платформу для обмена передовым опытом. и обеспечить согласованность за пределами стандартной структуры каталогов Maven. Например, предприятие может создать архетип с фирменными каскадными таблицами стилей (CSS), утвержденными библиотеками JavaScript и повторно используемыми компонентами. Разработчики, использующие этот архетип для создания проектов, автоматически будут соответствовать стандартам компании.

Встроенные архетипы

Maven предоставляет разработчикам сотни готовых архетипов. Кроме того, многие проекты с открытым исходным кодом предоставляют дополнительные архетипы, которые вы можете скачать и использовать. Maven также предоставляет подключаемый модуль архетипов, предназначенный для создания новых архетипов и создания проектов на основе существующих архетипов.

Цель создания подключаемого модуля архетипа позволяет просматривать и выбирать архетип для использования. В листинге 6-1 показаны результаты запуска `generate` цель в командной строке. На момент написания этой книги можно было выбирать из нескольких тысяч архетипов. В этой главе мы рассмотрим использование некоторых из этих архетипов.

Глава 6. Архетипы Maven

Листинг 6-1. Maven генерирует цель

Архетип \$mvn: сгенерировать

[INFO] Поиск проектов...

[INFO] Генерация проекта в интерактивном режиме

[INFO] Архетип не определен. Использование maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)

Выберите архетип:

1: remote -> am.ik.archetype:elm-spring-boot-blank-archetype (пустой мультипроект для Spring Boot + Elm)

2: удаленный -> am.ik.archetype:maven-reactjs-blank-archetype (пустой проект для React.js)

3: удаленный -> am.ik.archetype:msgpack-rpc-jersey-blank-archetype (пустой проект для Spring Boot + Jersey)

.....

.....

2460: удаленный -> ws.osiris:osiris-archetype (Архетип Maven для Осириса)

2461: удаленный -> xyz.luan.generator:xyz-gae-generator (-)

2462: удаленный -> xyz.luan.generator:xyz-генератор (-)

2463: локальный -> com.inflinx.book.ldap:practical-ldap-пустой архетип (-)

2464: локальный -> com.inflinx.book.ldap:practical-ldap-архетип (-)

2465: локальный -> com.apress.gswm:gswm-web-archetype (архетип gswm-web)

Выберите число или примените фильтр (формат: [groupId:]artifactId, содержит с учетом регистра): 1398:

Создание веб-проекта

Maven предоставляет архетип `maven-archetype-webapp` для создания веб-приложения.

Давайте сгенерируем приложение, выполнив следующую команду в папке C:

```
\apress\gswm-book\chapter6:
```

Архетип mvn: сгенерировать -DarchetypeArtifactId = веб-приложение `maven-archetype`

Команда работает в интерактивном режиме. Введите следующую информацию для запрошенных входных данных:

Определите значение для свойства 'groupId': `com.apress.gswmbook`

Определить значение свойства 'artifactId': `gswm-web`

Определить значение свойства «версия»: `1.0-SNAPSHOT`: : <<Нажмите Enter>>

Определить значение свойства «пакет»: `com.apress.gswmbook`: : `war`

Подтвердите настройку свойств:

идентификатор группы: `com.apress.gswmbook`

идентификатор артефакта: `gswm-web`

версия: `1.0-СНИМОК`

пакет: война

Y: <<Нажмите Enter>>

Сгенерированная структура каталогов должна напоминать ту, что показана на Рисунок 6-1.

Глава 6. Архетипы Maven

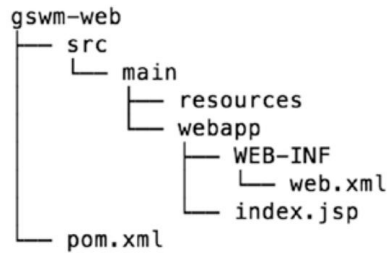


Рисунок 6-1. Структура веб-проекта Maven

Файл `pom.xml` минимален и имеет только зависимость JUnit. Maven упрощает запуск вашего веб-приложения с помощью встроенных веб-серверов, таких как Tomcat и Jetty. В листинге 6-2 показан измененный файл `pom.xml` с добавленным подключаемым модулем Jetty.

Листинг 6-2. Модифицированный файл `pom.xml` со встроенным подключаемым модулем Jetty

```

<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.apress.gswmbook</groupId>
  <artifactId>gswm-web</artifactId> <packaging>war</
  packaging> <version>1.0-SNAPSHOT</version>

  <name>gswm-web Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies> <dependency> <groupId>junit</
    groupId> <artifactId>junit</artifactId>
    <version>3.8.1</version> <scope>тест</
    scope> </dependency> </dependencies>

```

```

<сборка>
  <finalName>gswm-web</finalName> <plugins>
    <plugin> <groupId>org.eclipse.jetty</groupId>
      <artifactId>jetty-maven-plugin</artifactId>
        <version>9.4.12.RC2</ версия>

    </плагин> </
  плагины> </
<сборка>
</проект>

```

Чтобы запустить веб-приложение с помощью сервера Jetty, запустите следующую команду в корневом каталоге проекта:

mvn пристань: беги

Вы увидите развернутый проект и вывод, аналогичный показанному в листинге 6-3.

Листинг 6-3. Выходные данные команды запуска причала

```

[INFO] Started oejmpJettyWebAppContext@e38f0b7{Веб-приложение, созданное по
архетипу,/file: C:/apress/gswm-book/chapter6/ gswm-web/src/main/webapp/,AVAILABLE}
{file:///C:/ apress/gswm book/chapter6/gswm-web/src/main/webapp/}

```

```

[INFO] Started ServerConnector@5a0e0886{HTTP/1.1,[http/1.1]} {0.0.0.0:8080}

```

```

[INFO] Запущен @5120ms

```

```

[INFO] Запущен Jetty Server

```

Теперь запустите браузер и перейдите по адресу <http://localhost:8080/>. Вы должны увидеть веб-страницу, как показано на рисунке 6-2.

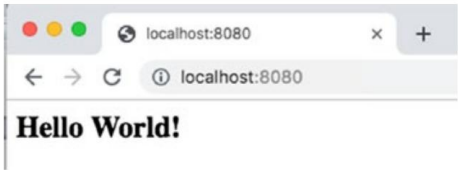


Рисунок 6-2. Веб-проект запущен в браузере

Многомодульный проект

Проекты Java Enterprise Edition (JEE) часто разбивают на несколько модулей для упрощения разработки и сопровождения. Каждый из этих модулей создает такие артефакты, как Enterprise JavaBeans (EJB), веб-службы, веб-проекты и клиентские JAR-файлы. Maven поддерживает разработку таких крупных проектов JEE, позволяя вкладывать несколько проектов Maven в один проект Maven. Схема такого многомодульного проекта показана на рис. 6-3. Родительский проект содержит файл `pom.xml` и отдельные проекты Maven внутри него.

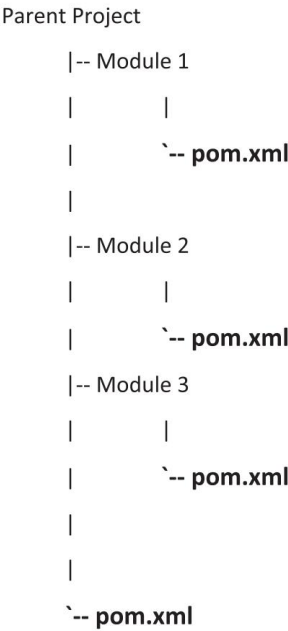


Рисунок 6-3. Многомодульная структура проекта

В оставшейся части этого раздела мы объясним, как построить мультимодуль-проект для сценария, в котором вам нужно разделить большой проект на веб-приложение (артефакт WAR), которое предоставляет пользовательский интерфейс, проект службы (артефакт JAR), который содержит код уровня службы, и проект сохраняемости, который содержит код уровня вашего репозитория. На рис. 6-4 показано визуальное представление этого сценария.

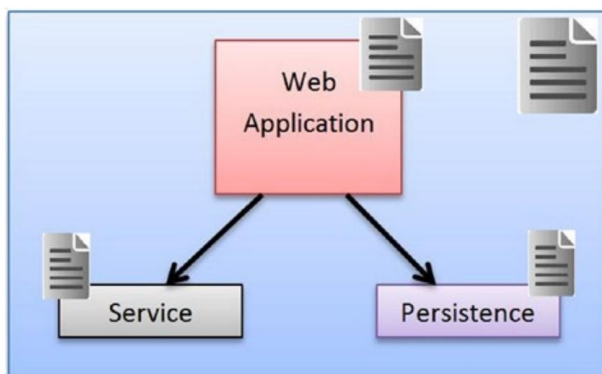


Рисунок 6-4. Многомодульный проект Maven

Начнем процесс с создания родительского проекта. Для этого запустите следующую команду в командной строке в папке `C:\apress\gswm book\chapter6`:

```
Архетип mvn: сгенерировать -DgroupId=com.apress.gswmbook
-DartifactId=gswm-parent -Dversion=1.0.0-SNAPSHOT
-DarchetypeGroupId=org.codehaus.mojo.archetypes -DarchetypeArtifactId=pom-
root
```

Архетип `pom-root` создает папку `gswm-parent` и файл `pom.xml`. файл под ним. Как видно из листинга 6-4, сгенерированный файл `pom.xml` имеет минимальное содержимое. Обратите внимание, что для упаковки родительского проекта задан тип `pom`.

Глава 6. Архетипы Maven

Листинг 6-4. Родительский файл pom.xml

```

<проект xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-экземпляр"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.apress.gswmbook</groupId>
  <artifactId>gswm-parent</artifactId>
  <версия>1.0.0-SNAPSHOT</версия>

  <packaging>бумага</packaging>
  <name>gswm-родитель</name>
</проект>

```

Затем создайте веб-проект, выполнив следующую команду в C:\apress\gswm-book\chapter6\gswm-parent папка:

```

Архетип mvn: сгенерировать -DgroupId=com.apress.gswmbook
-DartifactId=gswm-web -Dversion=1.0.0-SNAPSHOT -Dpackage=war
-DarchetypeArtifactId=maven-archetype-webapp

```

Во время создания этого веб-проекта вы предоставляете координаты Maven, такие как groupId, версия и т. д., в качестве параметров цели создания. Это создало проект gswm-web.

Следующим шагом является создание проекта службы. Запустите следующее команда в папке C:\apress\gswm-book\chapter6\gswm-parent:

```

Архетип mvn: сгенерировать -DgroupId=com.apress.gswmbook
-DartifactId=gswm-service -Dversion=1.0.0-SNAPSHOT
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```

Обратите внимание, что вы не указали параметр пакета, так как maven archetype-quickstart по умолчанию создает проект JAR. Также обратите внимание на

использование параметра `InteractiveMode`. Это указывает Maven просто запустить команду, не запрашивая ввода пользователя.

Аналогично предыдущему шагу создайте другой репозиторий `gswm` проекта Java, выполнив следующую команду в папке `C:\apress\gswmbook\chapter6\gswm-parent`:

```
Архетип mvn: сгенерировать -DgroupId=com.apress.gswmbook
-DartifactId=gswm-repository -Dversion=1.0.0-SNAPSHOT
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Теперь, когда у вас есть все сгенерированные проекты, давайте посмотрим на файл `pom.xml` в папке `gswm-parent`. В листинге 6-5 показан файл `pom.xml`.

Листинг 6-5. Родительский файл `pom.xml` с модулями

```
<?xml версия="1.0" кодировка="UTF-8"?>
<проект xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/
maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.apress.gswmbook</groupId>
  <artifactId>gswm-parent</artifactId>
  <версия>1.0.0-SNAPSHOT</версия>
  <packaging>бумара</packaging>
  <name>gswm-родитель</name>
  <модули>
    <module>gswm-web</module>
    <module>gswm-сервис</module>
    <module>gswm-репозиторий</module>
  </модули>
</проект>
```


Глава 6. Архетипы Maven

Элемент `modules` позволяет объявлять дочерние модули в многомодульном проекте. Когда вы создавали каждый модуль, Maven разумно регистрировал их как дочерний модуль. Кроме того, он изменил файл `pom.xml` отдельного модуля и добавил родительскую информацию `pom`. В листинге 6-6 показан файл `pom.xml` проекта `gswm-web` с родительскими элементами `pom`.

Листинг 6-6. Файл `pom.xml` для веб-модуля

```
<?xml версия="1.0"?>

<проект xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd" xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр">

  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>com.apress.gswmbook</groupId>
    <artifactId>gswm-parent</artifactId> <version>1.0.0-
    SNAPSHOT</version> </parent>

  <groupId>com.apress.gswmbook< /groupId>
  <artifactId>gswm-web</artifactId> <version>1.0.0-
  SNAPSHOT</version>

  <packaging>war</packaging>
  <name>gswm-web Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies> <dependency> <groupId>junit</
  groupId> <artifactId>junit</artifactId>
    <version>3.8.1</version> <scope>test</
  scope> </dependency> </dependencies>
```

```

<сборка>
  <finalName>gswm-web</finalName>
</build>
</проект>

```

Когда вся инфраструктура настроена, вы готовы к созданию следующего проекта. Для этого просто запустите команду `mvn package` в `gswm-project`, как показано в листинге 6-7.

Листинг 6-7. Запуск пакета Maven в родительском проекте

```

C:\apress\gswm-book\chapter6\gswm-parent>mvn package [INFO]
Поиск проектов...
[ИНФОРМАЦИЯ] ----- [ИНФОРМАЦИЯ]
Порядок сборки реактора: [ИНФОРМАЦИЯ]

[ИНФОРМАЦИЯ] gswm-
parent [ИНФОРМАЦИЯ] gswm-web
Maven Webapp [ИНФОРМАЦИЯ] gswm-
service [ИНФОРМАЦИЯ] gswm-
repository [ИНФОРМАЦИЯ] ----- [ИНФОРМАЦИЯ]
Обзор реактора: [ИНФОРМАЦИЯ]

[ИНФОРМАЦИЯ] gswm-родитель ..... УСПЕХ [0,001 с]
[INFO] gswm-web Maven Webapp ..... УСПЕХ [1,033 с]
[ИНФО] gswm-service ..... УСПЕХ [0,552 с]
[INFO] gswm-репозиторий ..... УСПЕХ [0,261 с]
[ИНФОРМАЦИЯ] ----- [ИНФОРМАЦИЯ]
ПОСТРОИТЬ УСПЕХ [ИНФОРМАЦИЯ] -----

```

Создание архетипа

Maven предоставляет несколько способов создания нового архетипа. Здесь мы будем использовать существующий проект для создания архетипа.

Давайте начнем с создания проекта-прототипа, который вы будете использовать в качестве основы для создания архетипа. Этот проект будет совместим с Servlet 4.0 и имеет сервлет состояния, который возвращает код состояния HTTP 200. Вместо создания веб-проекта с нуля скопируйте ранее сгенерированный код проекта gswm-web и создайте gswm-web-prototype в папке C:\apress. \gswm

Глава 6. Внесите следующие изменения во вновь скопированный проект:

1. Удалите целевую папку и другие ресурсы, такие как файлы интегрированной среды разработки (IDE) (.project, .classpath и т. д.), которые вы не хотите использовать в архетипе.
2. Замените содержимое файла web.xml под папку webapp/WEB-INF со следующим кодом. Это обновит веб-приложение для использования Servlet 4.0.

```
<?xml версия="1.0" кодировка="UTF-8"?>
<версия веб-приложения = "4.0" xmlns = "http://xmlns.jcp.org/xml/ns/
javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

  <display-name>Веб-приложение, созданное по архетипу </
display-name>

</веб-приложение>
```

3. Добавьте зависимость Servlet 4.0 в файл pom.xml. Обновленный pom.xml показан в листинге [6-8](#).

Листинг 6-8. pom.xml с зависимостью от сервлета

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/maven-v4_0_0.xsd"> <modelVersion>4.0.0</
modelVersion> <groupId> com.apress.gswmbook</groupId>
<artifactId>gswm-web</artifactId> <packaging>war</packaging>
<version>1.0-SNAPSHOT</version> <name>gswm-web Maven
WebApp</name> < url>http://maven.apache.org</url>
<dependencies> <dependency> <groupId>javax.servlet</
groupId> <artifactId>javax.servlet-api</artifactId> <version>4.0.1< /
версия> <scope>при условии</scope> </dependency> </
dependencies> <build>

<finalName>gswm-web</finalName>
<plugins> <plugin>
<groupId>org.eclipse.jetty</
groupId> <artifactId>jetty-maven-plugin</
artifactId> <version>9.4.12.RC2</ версия> </плагин>
</плагины> </сборка>

</project>

```

Глава 6. Архетипы Maven

4. Поскольку вы будете заниматься веб-разработкой на Java, создайте папку с именем java в папке src/main. Точно так же создайте test/java и test/resources папки в src.
5. Создайте файл AppStatusServlet.java в папке com. Пакет `apress.gswmbook.web.servlet` в папке `src/main/java`. Пакет `com.apress.gswmbook.web.servlet` преобразуется в структуру папок `com\apress\gswmbook\web\servlet`. Исходный код `AppStatusServlet.java` показан в листинге 6-9.

Листинг 6-9. Исходный код класса Java `AppStatusServlet`

```
пакет com.apress.gswmbook.web.servlet;
```

```
импортировать javax.servlet.annotation.WebServlet;
```

```
импортировать javax.servlet.*; импортировать
```

```
javax.servlet.http.*; импортировать java.io.*;
```

```
@WebServlet("/status")
```

```
открытый класс AppStatusServlet расширяет HttpServlet {
```

```
    public void doGet (запрос HttpServletRequest,
```

```
    Ответ HttpServletResponse) выдает IOException {
```

```
        Writer writer = response.getWriter(); писатель.println("OK");
```

```
        ответ.setStatus(ответ.SC_OK);
```

```
    }
```

```
}
```

Проект-прототип будет похож на структуру, показанную на Рисунок 6-5.

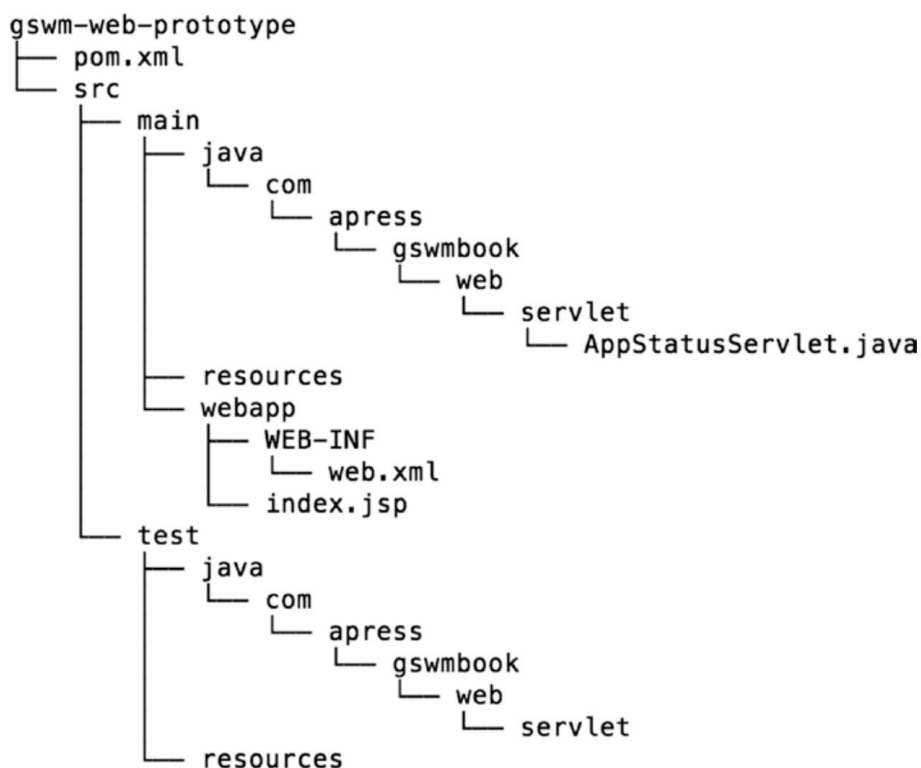


Рисунок 6-5. Сгенерированный прототип проекта

С помощью командной строки перейдите в папку проекта `gswm-web` прототип и выполните следующую команду:

Архетип mvn: создание из проекта

По завершении команды вы должны увидеть сообщение `Archetype created in target/generated-sources/archetype`. Недавно созданный архетип теперь находится в папке `gswm-web-prototype/target/generated-sources/archetype`.

Глава 6. Архетипы Maven

Следующим шагом является перемещение только что созданного архетипа в отдельную папку `gswm-web-archetype`, чтобы его можно было настроить перед публикацией. Для этого выполните следующие действия:

1. Создайте папку `gswm-web-archetype` в папке `C:`

`\apress\gswm-book\chapter6.`

2. Скопируйте каталог `pom.xml` и `src` и его файлы из каталога `C:`

`\apress\gswm-book\chapter6\gswm-web`

`prototype\target\generated-sources\archetype.`

папку в папку `gswm-web-archetype`.

Структура каталогов для `gswm-web-archetype` должна быть похожа на что показано на рисунке 6-6.

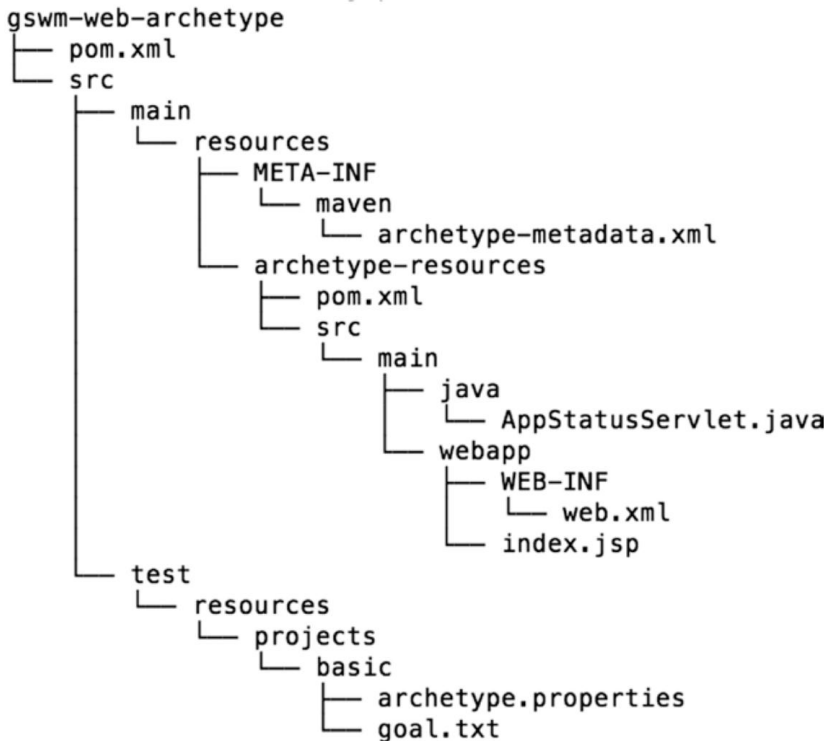


Рисунок 6-6. Структура проекта веб-архетипа

Начнем процесс модификации с файла `pom.xml`, расположенного в корне папки `gswm-web-archetype`. Измените идентификатор артефакта на «веб-архетип `gswm`» в файле `pom`. Далее мы изменим файл `pom.xml`, расположенный по адресу `gswm-web-archetype\src\main\resources\archetype-resources`. Измените `<finalName>` в файле `pom.xml` из `gswm-web` в `${artifactId}`. Во время создания проекта Maven заменит выражение `${artifactId}` предоставленным пользователем значением `ArtifactId`.

Когда проект создается из архетипа, Maven предлагает пользователю имя пакета. Он создаст каталоги, соответствующие пакету, в папке `src/main/java` только что созданного проекта. Затем он перемещает все содержимое папки `ресурсов/src/main/java` архетипа в этот пакет. Поскольку вам нужен `AppServlet` в подпакете `web.servlet`, создайте папку `web/servlet` и переместите `AppServlet.java` во вновь созданную папку. Новое расположение файла `AppServlet.java` показано на рис. 6-7.

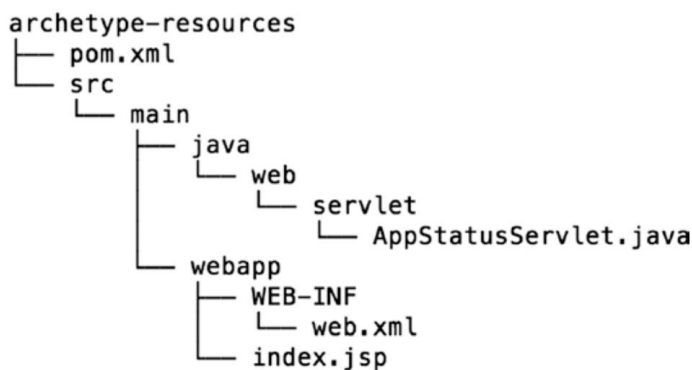


Рисунок 6-7. `AppServlet` в пакете `web.servlet`

Откройте `AppServlet.java` и измените имя пакета с пакет `${пакет}`; для упаковки `${package}.web.servlet`;

Глава 6. Архетипы Maven

Последним шагом в создании архетипа является выполнение следующего в командной строке внутри папки gswm-web-archetype:

mvn чистая установка

Использование архетипа

После установки архетипа самый простой способ создать из него проект — запустить следующую команду в C:\apress\gswm-book\chapter6:

Архетип mvn: сгенерировать -DarchetypeCatalog=local

Введите значения, показанные в листинге 6-10, для подсказок Maven, и вы увидите созданный тестовый проект.

Листинг 6-10. Создание нового проекта с использованием архетипа

```
C:\apress\gswm-book\chapter6>архетип mvn:генерировать
```

```
-DarchetypeCatalog=local
```

```
[INFO] Поиск проектов...
```

```
[ИНФОРМАЦИЯ]
```

```
[ИНФОРМАЦИЯ] -----
```

```
[INFO] Создание проекта-заглушки Maven (без POM) 1
```

```
[ИНФОРМАЦИЯ] -----
```

```
[INFO] Генерация проекта в интерактивном режиме
```

```
[INFO] Архетип не определен. Использование maven-archetype-quickstart
```

```
(org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
```

```
Выберите архетип:1: local -> com.apress.gswmbook:архетип gswm-web
```

```
(архетип gswm-web)
```

```
Выберите число или примените фильтр (формат: [groupId:]artifactId, содержит с  
учетом регистра): : 1
```

```
Определите значение для свойства 'groupId': : com.apress.gswmbook
```

```
Определить значение для свойства 'artifactId': : test-project
```

Определить значение свойства «версия»: 1.0-SNAPSHOT: :

Определить значение свойства «пакет»: com.apress.gswmbook: :

Подтвердите настройку свойств:

идентификатор группы: com.apress.gswmbook

ArtifactId: тестовый проект

версия: 1.0-СНИМОК

пакет: com.apress.gswmbook

Ю: :

проект

[ИНФОРМАЦИЯ] -----

[ИНФО] ДОСТИГАЙТЕ УСПЕХА

[ИНФОРМАЦИЯ] -----

Поскольку файл pom.xml для тестового проекта уже имеет встроенный плагин Jetty, запустите `mvn jetty:run` в командной строке в папке `C:\apress\gswm-book\chapter6\test-project`, чтобы запустить проект. . Откройте браузер и перейдите по адресу `http://localhost:8080/status`.
Вы увидите отображаемую строку ОК.

Резюме

Архетипы Maven — это шаблоны проектов, которые позволяют быстро запускать новые проекты. В этой главе использовались встроенные архетипы для создания расширенных проектов Maven, таких как веб-проекты и многомодульные проекты. Вы также рассмотрели создание и использование собственного архетипа.

В следующей главе вы изучите основы создания сайтов и создания документации и отчетов с помощью Maven.

ГЛАВА 7

Документация и отчетность

Документация и отчетность являются ключевыми аспектами любого проекта. Это особенно актуально для корпоративных проектов и проектов с открытым исходным кодом, когда над разработкой проекта сотрудничает множество людей. В этой главе рассматриваются некоторые инструменты и подключаемые модули Maven, которые упрощают публикацию и поддержку онлайн-документации.

В этой главе вы снова будете работать с Java-проектом gswm, созданным вами в предыдущих главах. Проект gswm также доступен в папке C:\apress\gswm-book\chapter7.

Использование жизненного цикла сайта

Как обсуждалось в главе 5, Maven обеспечивает жизненный цикл сайта, который можно использовать для создания документации проекта. Давайте запустим следующую команду из каталога gswm:

сайт mvn

Жизненный цикл сайта использует подключаемый модуль сайта Maven для создания сайта проекта. После выполнения этой команды в целевой папке проекта создается папка сайта. На рис. 7-1 показано содержимое папки сайта.

Глава 7 Документация и отчетность

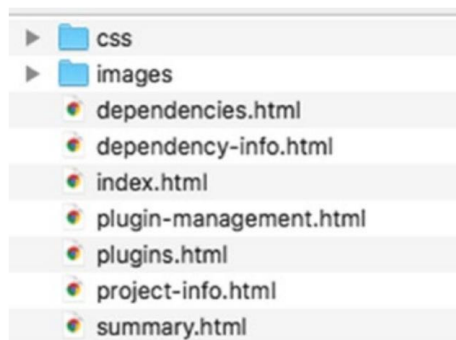


Рисунок 7-1. Сгенерированная папка сайта

Откройте файл `index.html` в браузере, чтобы просмотреть сгенерированный сайт.

Maven автоматически применяет к сайту скин по умолчанию и создает соответствующие изображения и файлы CSS. На рис. 7-2 показан сгенерированный файл `index.html`.

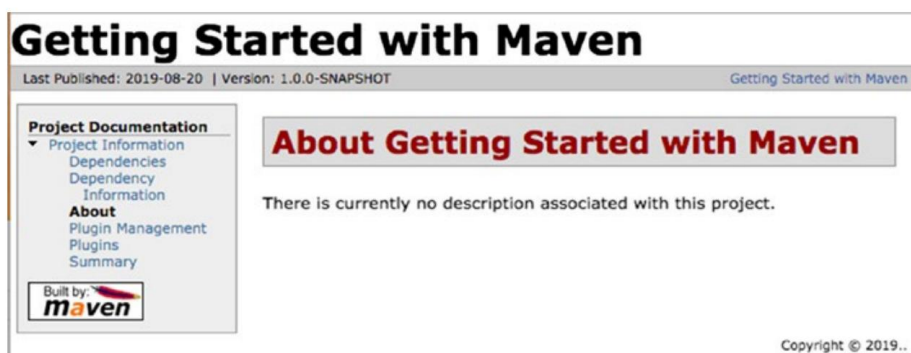


Рисунок 7-2. Сгенерированная индексная страница

Щелкнув ссылку «Зависимости» в нижней части левой панели навигации, вы перейдете на страницу «Зависимости проекта». Страница Project Dependencies предоставляет ценную информацию о прямых и транзитивных зависимостях проекта. Он также предоставляет информацию о лицензировании, связанную с этими зависимостями, как показано на рис. 7-3.

Project Dependencies

test

The following is a list of test dependencies for this project. These dependencies are only required to compile and run unit tests for the application:

GroupId	ArtifactId	Version	Type	License
junit	junit	4.11	jar	Common Public License Version 1.0

Project Transitive Dependencies

The following is a list of transitive dependencies for this project. Transitive dependencies are the dependencies of the project dependencies.

test

The following is a list of test dependencies for this project. These dependencies are only required to compile and run unit tests for the application:

GroupId	ArtifactId	Version	Type	License
org.hamcrest	hamcrest-core	1.3	jar	New BSD License

Project Dependency Graph

Рисунок 7-3. Страница зависимостей проекта

Maven позволяет добавлять информацию в файл `pom.xml`, чтобы сгенерированный сайт содержал полезную информацию. В листинге 7-1 показан обновленный файл `pom.xml`. Чтобы сайт успешно сгенерировался, мы явно объявляем последнюю версию `maven-site-plugin`.

Листинг 7-1. Файл `pom.xml` с информацией о проекте

```
<проект xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-экземпляр"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://
maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
```

Глава 7 Документация и отчетность

```

<groupId>com.apress.gswmbook</groupId>
<artifactId>gswm</artifactId> <version>1.0.0-
SNAPSHOT</version>

<packaging>банка</packaging>

<name>Начало работы с Maven</name> <url>http://
apress.com</url>

<description>
    Этот проект выступает в качестве начального проекта для книги «Введение
    в Maven» (http://www.apress.com/9781484208427), опубликованной Apress. </
    описание>

<mailingLists>
    <mailingList>
        <name>Список разработчиков GSWM</
        name> <subscribe>gswm-dev-subscribe@apress.com</subscribe>
        <unsubscribe>gswm-dev-unsubscribe@apress.com</unsubscribe> <post
        >developer@apress.com</post> </mailingList>

</списки рассылки>

<лицензии>
    <лицензия>
        <name>Лицензия Apache, версия 2.0</name>
        <url>http://www.apache.org/licenses/LICENSE-2.0.txt</url> </license>

</лицензии>

<!-- Разработчик и зависимость удалены для краткости --->
<сборка>

<плагины>
    <плагин>

```

```

<groupId>org.apache.maven.plugins</groupId>
<artifactId>плагин компилятора maven</artifactId>
<версия>3.8.1</версия>

<конфигурация>
  <источник>1.8</источник>

  <цель>1.8</цель>
</конфигурация>
</плагин>
<плагин>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-site-plugin</artifactId>
  <версия>3.8.2</версия>

</плагин>
</плагины>
</сборка>

</проект>

```

В листинге [7-1](#) мы используем элемент `description` для предоставления описания проекта. Элемент `mailingList` содержит информацию о различных списках рассылки, связанных с проектом, а элемент лицензии включает лицензию проекта. Имея обновленный файл `pom.xml`, давайте восстановим сайт, выполнив следующую команду:

мвн чистый сайт

Запустите файл `index.html` во вновь сгенерированном `target\` папка сайта. На рисунках [7-4A](#) и [7-4B](#) показаны новые страницы `About` и `Project License` соответственно. Обратите внимание, что Maven использует URL-адрес, объявленный в элементе лицензии, для загрузки текста лицензии и включения его в сгенерированный веб-сайт.

Глава 7 Документация и отчетность



Рисунок 7-4А. Создана страница «О нас»

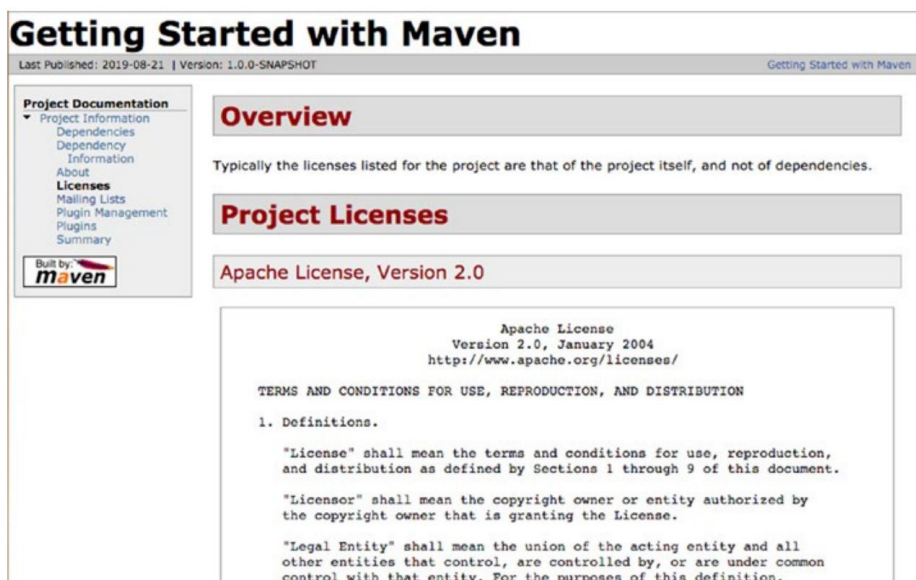


Рисунок 7-4Б. Страница сгенерированных лицензий проекта

Расширенная конфигурация сайта

В предыдущем разделе информация о проекте была указана в файле `pom`.

`xml` для использования Maven при создании сайта. Для более крупных проектов этот подход

приведет к раздутым и сложным в обслуживании файлам `pom.xml`.

Кроме того, предприятия обычно предпочитают использовать свою торговую марку и логотипы в

сгенерированный сайт, а не скин Maven по умолчанию. Чтобы решить эти проблемы, Maven позволяет указать содержимое и конфигурацию для создания сайта в папке с подходящим названием `src/site`. На рис. 7-5 показана структура каталогов для простой папки сайта.

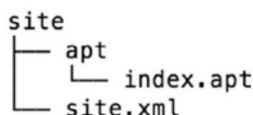


Рисунок 7-5. Структура каталогов папок сайта

Файл `site.xml`, также известный как дескриптор сайта, используется для настройки сгенерированного сайта. Мы рассмотрим этот файл через секунду.

Папка `apt` содержит контент сайта, написанный почти простым текстом (APT) . формат. Формат АРТ позволяет создавать документацию с синтаксисом, напоминающим обычный текст. Дополнительную информацию о формате АРТ можно найти на веб-сайте Maven (<http://maven.apache.org/doxia/>) . [ссылки/apt-format.html](http://maven.apache.org/doxia/)). Помимо АРТ, Maven поддерживает другие форматы, такие как FML, Xdoc и Markdown.

Maven предоставляет несколько архетипов, позволяющих автоматически генерировать структуру сайта. Чтобы обновить существующий проект `gswm`, выполните следующую команду в папке `C:\apress\gswm-book\chapter7\gswm`. При появлении запроса введите значения для `groupId`, артефакта и пакета.

Архетип `mvn`: сгенерировать -DarchetypeGroupId=org.apache.maven.

архетипы -DarchetypeArtifactId=maven-архетип-сайт-простой -DarchetypeVersion=1.4

Определите значение для свойства `'groupId':` : `com.apress.gswmbook`

Определить значение свойства `'artifactId':` : `gswm`

Определить значение свойства «версия»: `1.0-SNAPSHOT:` :
`1.0.0-СИМОК`

Определить значение свойства «пакет»: `com.apress.gswmbook:` :

<<Нажмите Enter>>

Глава 7 Документация и отчетность

После успешного выполнения команды вы увидите папку сайта, созданную в папке `gswm\src` с папками `site.xml` и `apt`. Начнем с добавления описания проекта в `index.apt`. Замените содержимое файла `index.apt` кодом из листинга 7-2.

Листинг 7-2. Содержимое файла `index.apt`

Начало работы с Maven

Адрес

08-10-2019

Этот проект выступает в качестве начального проекта для книги « Введение в Maven », опубликованной Apress. Для получения дополнительной информации посетите сайт Apress: www.apress.com.

Первые три раздела содержат название документа, автора и дату. Следующий блок текста содержит описание проекта. Запуск `mvn clean site` приводит к появлению новой страницы «О программе», как показано на рис. 7-6.

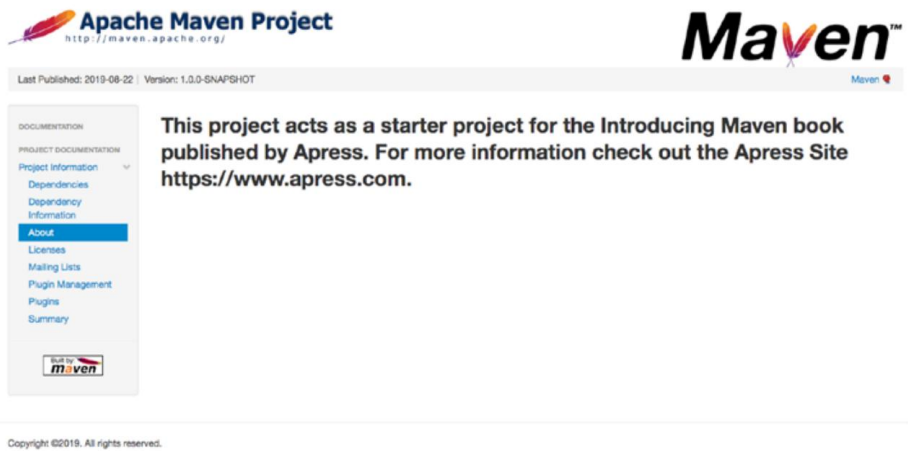


Рисунок 7-6. О странице с новым контентом

Файл `site.xml` позволяет настроить сгенерированный сайт, например изменить заголовок и переопределить стандартную навигацию и внешний вид.

Чтобы лучше понять возможности `site.xml`, давайте изменим сгенерированный логотип сайта. Статические активы, такие как изображения и HTML-файлы, размещаются на сайте/ папка ресурсов. Когда Maven создает сайт, он копирует ресурсы из папки ресурсов в корень сгенерированного сайта. Скопируйте логотип компании `company.png` из папки `C:\apress\gswm-book\chapter7` и поместите его в папку `gswm/src/site/resources/images`.

Замените файл `site.xml` содержимым листинга 7-3. Обратите внимание, что элемент `src` для логотипа включает относительный путь `images/company.png`. Элемент меню используется для создания ссылок на пользовательские веб-страницы/ контент/вики-страницы, которые вы хотите добавить на сайт.

Листинг 7-3. Содержимое файла `site.xml`

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<имя проекта="${artifactId}" xmlns="http://maven.apache.org/
УКРАШЕНИЕ/1.8.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema instance"
```

Глава 7 Документация и отчетность

```
xsi:schemaLocation="http://maven.apache.org/DECORATION/1.8.0 http://
maven.apache.org/xsd/decoration-1.8.0.xsd"> <bannerLeft>
```

```

<name>Apress</name>
<src>images/company.png</src>
<href>http://apress.com</href> </
bannerLeft>
```

```

<скин>
<groupId>org.apache.maven.skins</groupId>
<artifactId>maven-fluido-skin</artifactId>
<версия>1.7</версия>
</скин>
```

```

<тело>
<ссылки>
<item name="Maven" href="https://maven.apache.org/" /> </links>
```

```

<имя меню="Документация">
<item name="Сайт Apache" href="http://www.apache.org/" /> </menu>
```

```

<menu ref="отчеты" /> </
body> </project>
```

Запуск `mvn clean site` создает сайт с новым логотипом и дополнительным элементом навигации, как показано на рис. 7-7.

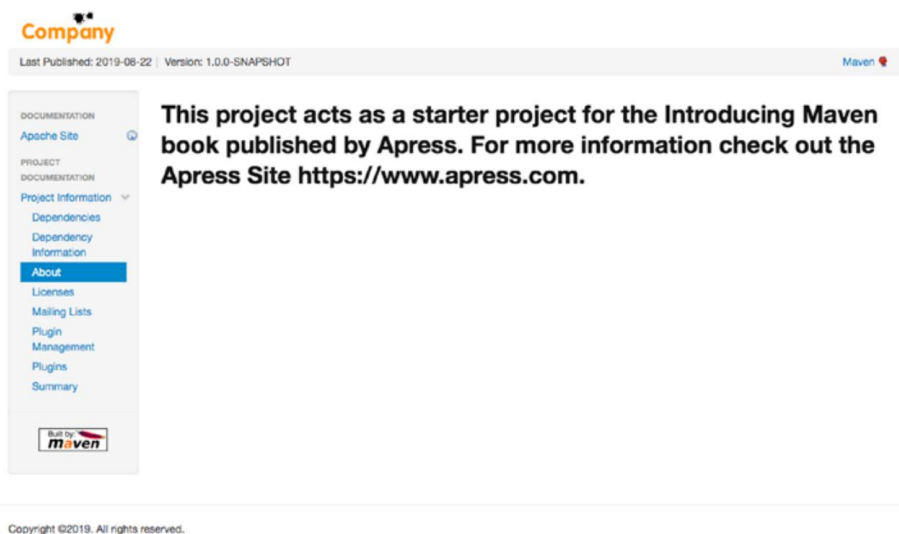


Рисунок 7-7. О странице с новым логотипом

Создание отчетов Javadoc

Javadoc является стандартом де-факто для документирования кода Java. Это помогает разработчикам понять, что делает класс или метод. Javadoc также выделяет устаревшие классы, методы или поля.

Maven предоставляет подключаемый модуль Javadoc, который использует инструмент Javadoc для создание Javadocs. Интеграция подключаемого модуля Javadoc просто включает его объявление в элементе отчета файла `pom.xml`, как показано в листинге 7-4. Плагины, объявленные в элементе отчетности `report`, выполняются во время создания сайта.

Листинг 7-4. Фрагмент `pom.xml` с подключаемым модулем Javadoc

```
<проект>
    <!--Содержимое удалено для краткости-->
<отчетность>
```

Глава 7 Документация и отчетность

```
<плагины>
  <плагин>
    <artifactId>плагин maven-javadoc</artifactId>

  </плагин>
</плагины>
</отчетность>
</проект>
```

Теперь, когда у вас настроен плагин Javadoc, давайте запустим mvn clean сайт для создания Javadoc. После успешного выполнения команды вы увидите папку apidocs, созданную в папке gswm /target/site. Запустите файл index.html на сайте и перейдите к Project Reports Javadoc.

На рис. 7-8 показан документ Javadoc, созданный для проекта gswm.

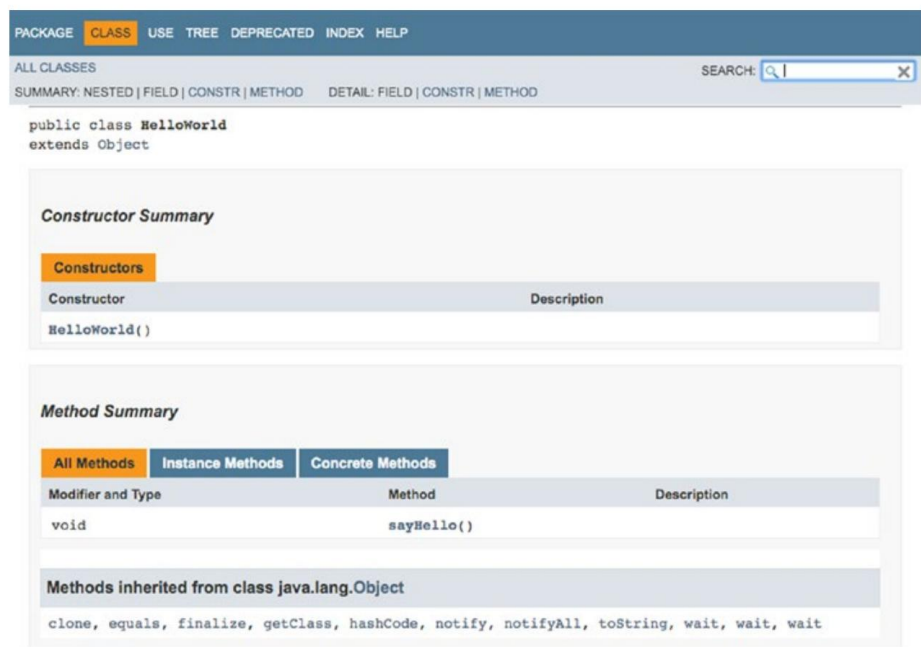


Рисунок 7-8. Сгенерированная страница Javadoc

Создание отчетов модульного тестирования

Разработка через тестирование стала сегодня нормой на предприятиях. Модульные тесты обеспечивают немедленную обратную связь с разработчиками и позволяют им создавать качественный код. Учитывая важность тестов, Maven выполняет все тесты для каждой сборки. Любой сбой теста приводит к неудачной сборке.

Maven предлагает подключаемый модуль Surefire, который обеспечивает единый интерфейс для выполнения тестов, созданных такими средами, как JUnit или TestNG. Он также генерирует результаты выполнения в различных форматах, таких как XML и HTML. Эти опубликованные результаты позволяют разработчикам быстро находить и исправлять неработающие тесты.

Плагин Surefire настраивается так же, как и плагин Javadoc, в разделе отчетов файла `pom`. В листинге 7-5 показана конфигурация подключаемого модуля Surefire.

Листинг 7-5. Фрагмент `pom.xml` с подключаемым модулем Surefire

```
<проект>
    <!--Содержимое удалено для краткости-->
    <отчетность>
        <плагины>
            <плагин>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-report-plugin</artifactId>
                <версия>2.17</версия>
            </плагин>
        </плагины>
    </отчетность>
</проект>
```

Теперь, когда Surefire настроен, давайте создадим сайт Maven, запустив Команда `mvn` очистить сайт. После успешного выполнения команды вы увидите папку Surefire Reports, сгенерированную в папке `gswm\target`.

Глава 7 Документация и отчетность

Он содержит результаты выполнения тестов в форматах XML и TXT. Одинаковый информация будет доступна в формате HTML в отчете surefire. html файл в папке сайта. Запустите файл index.html на сайте и перейдите в «Отчеты по проектам» «Отчет Surefire». Рисунок 7-9 показывает отчет Surefire для проекта gswm.

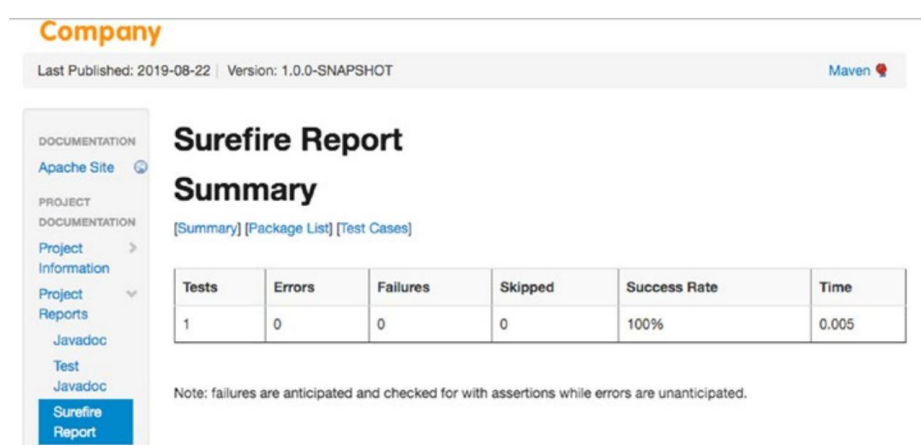


Рисунок 7-9. Сгенерированный отчет Surefire

Создание отчетов о покрытии кода

Покрытие кода — это показатель того, насколько исходный код проверяется автоматическими тестами. По сути, это дает представление о качестве ваших тестов. JaCoCo (с открытым исходным кодом) и Clover от Atlassian — два популярных инструмента покрытия кода для Java.

В этом разделе вы будете использовать JaCoCo для измерения кода этого проекта. покрытие. В листинге 7-6 показана конфигурация подключаемого модуля JaCoCo. Цель агента подготовки устанавливает свойство, указывающее на среду выполнения JaCoCo, которая передается в качестве аргумента виртуальной машины при выполнении модульных тестов. Цель отчета создает отчеты о покрытии кода после завершения выполнения модульного теста.

Листинг 7-6. Фрагмент pom.xml с подключаемым модулем JaCoCo

```

<project>
  <build>
    <plugins>
      <!--Содержимое удалено для краткости-->
      <plugin> <groupId>org.jacoco</
        groupId> <artifactId>jacoco-maven-
        plugin </artifactId> <version>0.8. 4</
        version> <executions> <execution>
          <id>jacoco-init</id> <goals>
            <goal>prepare-agent</goal> </goals>
          </execution> <execution>

          <id>jacoco-report</id>
          <phase>test</phase>
          <goals> <goal>report</
            goal> </goals> </execution>
          </executions> </plugin>
    </plugins> <построить>

</project>

```

Глава 7 Документация и отчетность

Теперь, когда плагин настроен, давайте сгенерируем сайт с помощью команды `mvn clean site`. После успешного выполнения команды JaCoCo создаст папку `jacoco` в папке `gswm\target\site`. Запустите отчет о покрытии кода, дважды щелкнув файл `index.html` в папке `jacoco`. Отчет должен быть похож на тот, что показан на рис. 7-10.

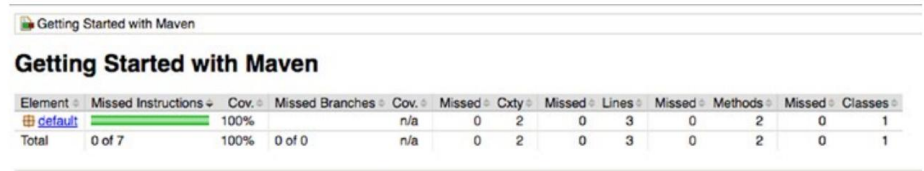


Рисунок 7-10. Сгенерированный отчет JaCoCo

Создание отчета SpotBugs

SpotBugs — это инструмент для обнаружения дефектов в коде Java. Он использует статический анализ для обнаружения шаблонов ошибок, таких как бесконечные рекурсивные циклы и разыменования нулевого указателя. В листинге 7-7 показана конфигурация SpotBugs.

Листинг 7-7. Фрагмент `pom.xml` с подключаемым модулем SpotBugs

```
<проект>
    <!--Содержимое удалено для краткости-->
    <отчетность>
        <плагины>
            <плагин>
                <groupId>com.github.spotbugs</groupId>
                <artifactId>spotbugs-maven-плагин</artifactId>
                <версия>3.1.12</версия>
            </плагин>
        </плагины>
    </отчетность>
</проект>
```

После создания сайта Maven откройте файл index.html под сайтом. папку и перейдите к Project Reports SpotBugs, чтобы просмотреть отчет SpotBugs. Он должен быть похож на тот, что показан на рисунке 7-11.



Рисунок 7-11. Сгенерированный отчет об обнаружении ошибок SpotBugs

Резюме

Возможности документации и отчетности, предоставляемые Maven, играют важную роль в создании удобного в сопровождении качественного программного обеспечения. В этой главе объяснялись основы использования жизненного цикла сайта и конфигурация, необходимая для создания документации. Вы также рассмотрели создание Javadocs, тестового покрытия и отчетов SpotBugs.

В следующей главе мы объясним, как интегрировать Maven с Nexus. и Гит. Вы также узнаете о процессе выпуска Maven.

ГЛАВА 8

Выпуск Мейвена

Maven предоставляет подключаемый модуль выпуска, который автоматизирует шаги, связанные с выпуском программного обеспечения. Прежде чем мы углубимся в процесс выпуска Maven, мы установим и настроим репозиторий Nexus и будем использовать Maven для публикации артефактов в Nexus.

Интеграция с Нексус

Менеджеры репозитория являются ключевой частью развертывания Maven на предприятиях. Менеджеры репозитория действуют как прокси общедоступных репозитория, облегчают совместное использование артефактов и совместную работу в команде, обеспечивают стабильность сборки и позволяют управлять артефактами, используемыми на предприятии.

Менеджер репозитория Sonatype Nexus — популярное программное обеспечение с открытым исходным кодом, что позволяет поддерживать внутренние репозитории и получать доступ к внешним репозиториям. Это позволяет группировать репозитории и получать к ним доступ через один URL-адрес. Это позволяет администратору репозитория добавлять и удалять новые репозитории в фоновом режиме, не требуя от разработчиков изменения конфигурации на своих компьютерах. Кроме того, он предоставляет возможности хостинга для сайтов, созданных с использованием возможностей поиска сайтов и артефактов Maven.

Прежде чем мы рассмотрим интеграцию Maven с Nexus, вам необходимо установить Nexus на вашем локальном компьютере. Nexus распространяется в виде архива и поставляется в комплекте с экземпляром Jetty. Загрузите дистрибутив Nexus (версия .zip для Windows) с веб-сайта Sonatype по адресу [https://help.](https://help.sonatype.com/repomanager3/download)

[sonatype.com/repomanager3/download](https://help.sonatype.com/repomanager3/download). На момент написания этой статьи

Глава 8. Релиз Maven

доступна версия 3.18.1-01 Nexus. Разархивируйте файл и поместите содержимое на свой компьютер. В этой книге предполагается, что содержимое находится в папке C:\tools\nexus.

Примечание . Большинство предприятий обычно имеют менеджеры репозитория, установленные и доступные на центральном сервере. Если у вас уже есть доступ к менеджеру репозитория, пропустите эту часть установки.

Запустите командную строку в режиме администратора и перейдите в папку bin , расположенную в папке C:\tools\nexus\nexus-3.18.1-01. Затем выполните команду `nexus /install Nexus_Repo_Manager`. Вы увидите сообщение об успешном завершении, как показано на рис. 8-1.

```
C:\tools\nexus\nexus-3.18.1-01\bin>nexus /install Nexus_Repo_Manager
Installed service 'Nexus_Repo_Manager'.
```

Рисунок 8-1. Сообщение об успешном завершении установки Nexus

Примечание . Для правильной работы Nexus 3.18 требуется JRE 8. Убедитесь, что на вашем локальном компьютере установлена версия 8 JDK/JRE. Также убедитесь, что JAVA_HOME указывает на версию 8 JDK.

В той же командной строке запустите команду `nexus start` для запуска Нексуса. На рис. 8-2 показан результат выполнения этой команды.

```
C:\tools\nexus\nexus-3.18.1-01\bin>nexus /start Nexus_Repo_Manager
Starting service 'Nexus_Repo_Manager'.
```

Рисунок 8-2. Запуск Нексуса

По умолчанию Nexus использует порт 8081. Запустите веб-браузер и перейдите на Nexus по адресу `http://localhost:8081/`. Это займет несколько минут, но в конечном итоге вы должны увидеть экран запуска Nexus, как показано на рис. 8-3.

Глава 8. Релиз Maven

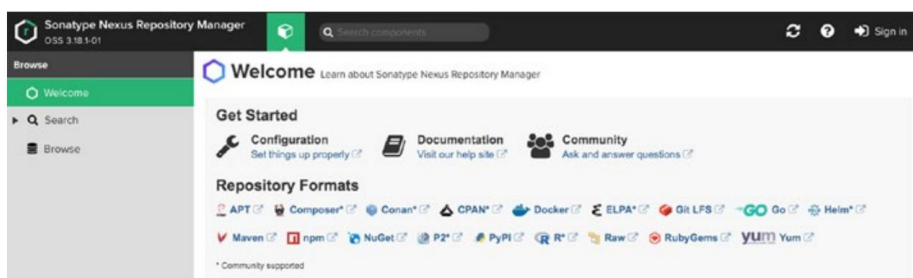


Рисунок 8-3. Экран запуска Нексуса

Нажмите ссылку «Войти» в правом верхнем углу, чтобы войти в Nexus. Вам будет представлено окно входа в систему, содержащее расположение файла с автоматически сгенерированным паролем администратора, как показано на рисунке 8-4.

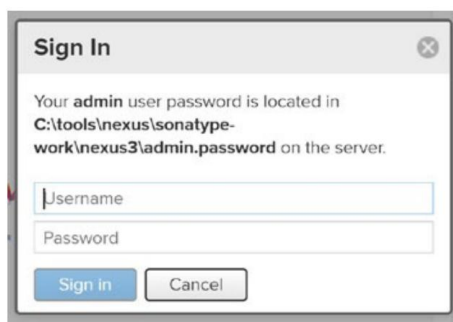
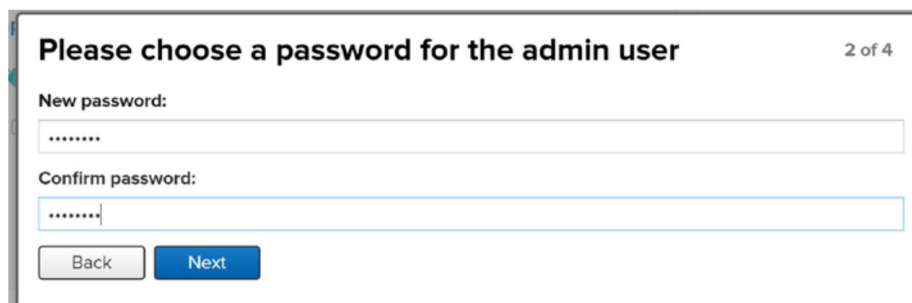


Рисунок 8-4. Модальное окно входа в Nexus

Войдите в Nexus с именем пользователя admin и паролем, скопированными из файла admin.password. Вам будет предложено изменить пароль, как показано на рисунке 8-5. Для упражнений в этой книге я изменил пароль на admin123.

Глава 8. Релиз Maven



Please choose a password for the admin user 2 of 4

New password:

Confirm password:

Back Next

Рисунок 8-5. Экран смены пароля Nexus

Теперь, когда Nexus установлен и запущен, давайте изменим проект gswm, расположенный в папке C:\apress\gswm-book\chapter8. Вы начнете с добавления элемента DistributionManagement в файл pom.xml, как показано в листинге 8-1. Этот элемент используется для предоставления информации репозитория о том, где будут развернуты артефакты проекта. Подэлемент репозитория указывает место, где будут развернуты выпущенные артефакты. Точно так же элемент snapshotRepository указывает место, где будут храниться SNAPSHOT-версии проекта.

Листинг 8-1. pom.xml с управлением распределением

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http:// /
maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
    <зависимости>
        <!-- Содержание удалено для краткости -->
    </зависимости>
    <управление распределением>
        <репозиторий>
            <id>nexusReleases</id>
            <name>Выпуски</name>
```

```

<url>http://localhost:8081/repository/maven-releases </url>

</репозиторий>
<моментальный репозиторий>
  <id>снимки связи</id>
  <name>Снимки</name>
  <url>http://localhost:8081/repository/maven snapshots/<
    url>
  </snapshotRepository>
</управление распределением>
<сборка>
  <!-- Содержание удалено для краткости -->
</сборка>
</проект>

```

Примечание . По умолчанию Nexus поставляется с выпусками и моментальными снимками. репозитории. По умолчанию артефакты SNAPSHOT будут храниться в репозитории снэпшотов, а артефакты релизов будут храниться в репозитории релизов.

Как и большинство менеджеров репозитория, развертывание в Nexus является защищенной операцией. Чтобы Maven мог взаимодействовать и развертывать артефакты на Nexus, вам необходимо предоставить пользователю правильные роли доступа в файле settings.xml. В листинге 8-2 показан файл settings.xml с информацией о сервере. Как видите, мы используем информацию администратора для подключения к Nexus. Обратите внимание, что идентификаторы, объявленные в теге сервера — nexusReleases и nexusSnapshots — должны совпадать с идентификаторами репозитория и snapshotRepository, объявленными в файле pom.xml. Замените содержимое файла settings.xml в папке C:\Users\<<USER_NAME>>\.m2 кодом из листинга 8-2.

Глава 8. Релиз Maven

Листинг 8-2. Файл Settings.xml с информацией о сервере

```
<?xml версия="1.0" кодировка="UTF-8" ?>
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0" xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http:// /
maven.apache.org/SETTINGS/1.0.0 http://maven.apache.org/xsd/settings-1.0.0.xsd">

<серверы>
  <сервер>
    <id>nexusReleases</id>
    <имя пользователя>администратор</имя пользователя>
    <пароль>admin123</пароль>
  </сервер>
  <сервер>
    <id>снимки связи</id>
    <имя пользователя>администратор</имя пользователя>
    <пароль>admin123</пароль>
  </сервер>
</серверы>
</настройки>
```

На этом этапы настройки для взаимодействия с Nexus завершены.

В командной строке запустите команду `mvn deploy` в каталоге `C:\apress\gswm-book\chapter8\gswm`. После успешного выполнения команды вы увидите артефакт SNAPSHOT в Nexus по адресу `http://localhost:8081/#browse/browse:maven-snapshots`, как показано на рисунке 8-6.

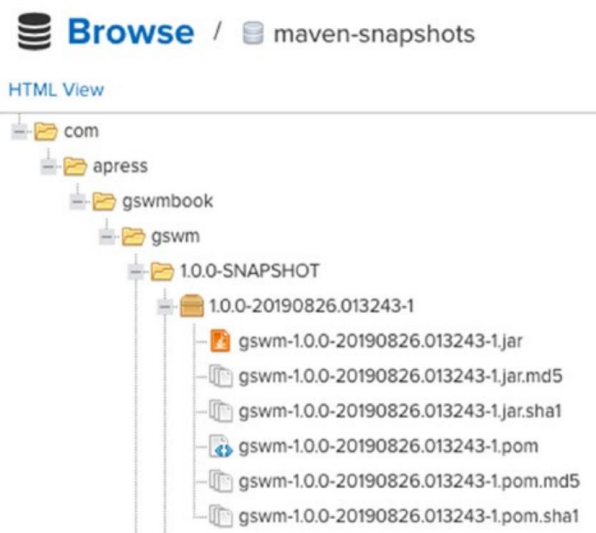


Рисунок 8-6. Артефакт SNAPSHOT под Nexus

Релиз проекта

Релиз проекта — сложный процесс, обычно включающий следующие этапы:

- Убедитесь в отсутствии незафиксированных изменений на локальной машине.
- Удалите SNAPSHOT из версии в файле `pom.xml`.
- Убедитесь, что проект не использует никаких зависимостей SNAPSHOT.
- Верните измененный файл `pom.xml` в исходный код-контроль.
- Создайте тег управления версиями исходного кода.

Глава 8. Релиз Maven

- Создайте новую версию артефакта и разверните ее в менеджере репозитория.
- Увеличьте версию в файле `pom.xml` и подготовьте для следующего цикла разработки.

У Maven есть подключаемый модуль выпуска, который предоставляет стандартный механизм для выполнения предыдущих шагов и выпуска артефактов проекта. Как видите, в процессе выпуска Maven активно взаимодействует с системой контроля версий. В этом разделе вы будете использовать Git в качестве системы управления версиями и GitHub в качестве удаленного сервера, на котором размещены репозитории. Типичное взаимодействие между Maven и GitHub показано на рис. 8-7. Выпуски Maven обычно выполняются на машине разработчика или сборки. Maven требует, чтобы на таких машинах был установлен клиент Git. Эти инструменты командной строки позволяют Maven взаимодействовать с GitHub и выполнять такие операции, как проверка кода, создание тегов и т. д.

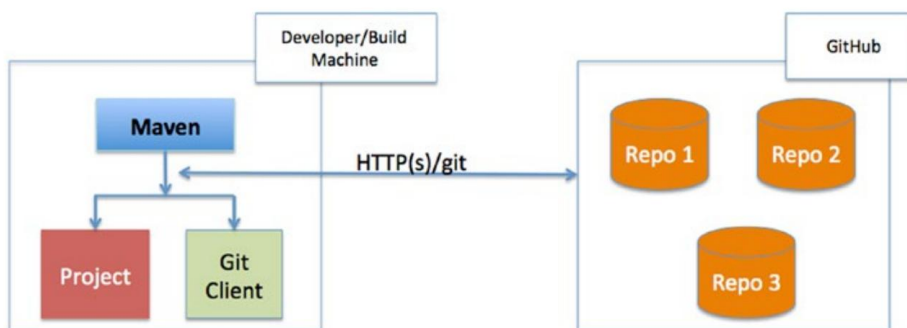


Рисунок 8-7. Взаимодействие между Maven и GitHub

Прежде чем мы углубимся в процесс выпуска Maven, вам нужно установить настроить среду, выполнив следующие шаги:

1. Установите клиент Git на свой локальный компьютер.
2. Создайте новый удаленный репозиторий на GitHub.
3. Зарегистрируйте проект, который вы будете использовать, в удаленном репозитории.

Установка Git-клиента

Существует несколько клиентов Git, упрощающих взаимодействие с репозиториями Git. К популярным относятся SourceTree (www.sourcetreeapp.com/) и GitHub Desktop (<https://desktop.github.com/>). В этой книге мы будем использовать клиент, поставляемый с дистрибутивом Git SCM. Перейдите по [адресу https://git-scm.com/downloads](https://git-scm.com/downloads) и загрузите версию дистрибутива Git для Windows. Дважды щелкните загруженный исполняемый файл и примите параметры установки по умолчанию. После завершения установки откройте новое окно командной строки и введите `git --version`. Вы должны увидеть сообщение, похожее на рисунок 8-8.

```
C:\>git --version
git version 2.23.0.windows.1
```

Рисунок 8-8. Git-версия

Создание репозитория GitHub

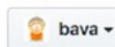
GitHub — это платформа для совместной разработки, которая позволяет бесплатно размещать общедоступные и частные репозитории Git. Прежде чем вы сможете создать новый репозиторий на GitHub, вам необходимо создать учетную запись на <https://github.com> / [присоединиться](#). После того, как вы вошли в GitHub, используя свои учетные данные, перейдите на <https://github.com/new> и создайте новый репозиторий, как показано на рис. 8-9.

Глава 8. Релиз Maven

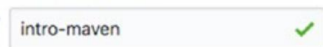
Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner



Repository name *



Great repository names are short and memorable. Need inspiration? How about **ubiquitous-giggle?**

Description (optional)

Repository to demo Maven Release Process



Public

Anyone can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.

Skip this step if you're importing an existing repository.

☐ Initialize this repository with a README

This will let you immediately clone the repository to your computer.

Add .gitignore: **None** ▼

Add a license: **None** ▼



Create repository

Рисунок 8-9. Новый репозиторий GitHub

Проверка исходного кода

Последним шагом в подготовке среды к выпуску Maven является проверка проекта gswm в папке C:\apress\gswm-book\chapter8\.

gswm во вновь созданный репозиторий удаления. С помощью командной строки перейдите в папку C:\apress\gswm-book\chapter8\gswm и последовательно выполните следующие команды. Убедитесь, что вы используете правильный пульт

Глава 8. Релиз Maven

URL-адрес репозитория, заменив свою учетную запись GitHub в следующем удаленном добавить команду:

```
git инициализировать

git добавить .

git commit -m "Первоначальная фиксация"

git удаленно добавить источник https://github.
com/<<your_git_hub_account>>/intro-maven.git

git push -u мастер происхождения
```

Команда Git push запросит у вас имя пользователя и пароль GitHub. Успешное выполнение команды push должно привести к выводу, показанному на рис. 8-10.

```
C:\apress\gswm-book\chapter8\gswm>git push -u origin master
Logon failed, use ctrl+c to cancel basic credential prompt.
Username for 'https://github.com': bava
Password for 'https://bava@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (10/10), 1.56 KiB | 799.00 KiB/s, done.
Total 10 (delta 0), reused 0 (delta 0)
To https://github.com/bava/intro-maven.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

Рисунок 8-10. Вывод первоначальной фиксации Git

С помощью браузера перейдите в удаленный репозиторий на GitHub, и вы увидите зарегистрированный код. Рисунок 8-11 показывает ожидаемый браузер экран.

Глава 8. Релиз Maven

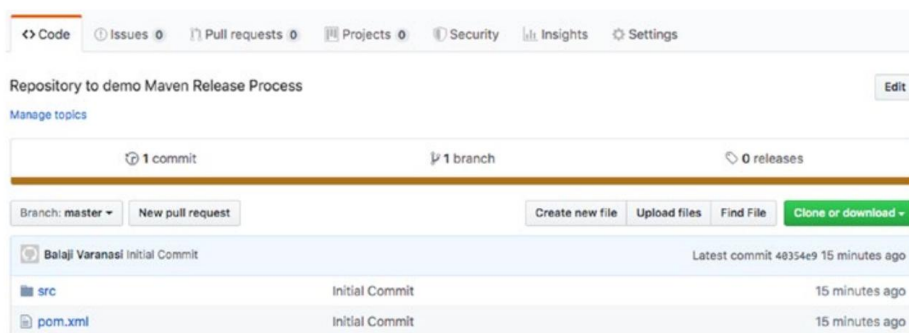


Рисунок 8-11. Проект зарегистрирован на GitHub

Предыдущие команды поместили код в основную ветку. на Гитхабе. Однако подключаемый модуль выпуска Maven взаимодействует с кодом в ветке выпуска. Итак, последний шаг в этой настройке — создать новую локальную ветку релиза и отправить ее на GitHub, выполнив следующие команды:

```
git checkout -b релиз
```

```
выпуск git push origin
```

Выпуск Мейвена

Выпуск артефакта с использованием процесса выпуска Maven требует достижения двух важных целей: подготовить и выполнить. Кроме того, подключаемый модуль выпуска обеспечивает чистую цель, которая пригодится, когда что-то пойдет не так.

Подготовить цель

Цель подготовки, как следует из названия, подготавливает проект к выпуску. В рамках этого этапа Maven выполняет следующие операции:

- `check-poms`: проверяет наличие в версии файла `pom.xml` SNAPSHOT.
- `scm-check-modifications`: проверяет наличие незафиксированных изменений.

Глава 8. Релиз Maven

- `check-dependency-snapshots`: Проверяет файл `pom` на предмет наличия каких-либо зависимостей `SNAPSHOT`. Для вашего проекта рекомендуется использовать выпущенные зависимости. Любые зависимости `SNAPSHOT`, найденные в `pom.xml` файл приведет к сбою выпуска.
- `map-release-versions`: когда подготовка выполняется в интерактивном режиме пользователю предлагается указать версию выпуска.
- `map-development-versions`: Когда подготовка выполняется в интерактивный режим, пользователю предлагается следующая версия разработки.
- `generate-release-poms`: создает файл `pom` выпуска.
- `scm-commit-release`: фиксирует выпуск файла `pom` к СКМ.
- `scm-tag`: создает тег выпуска для кода в SCM.
- `rewrite-poms-for-development`: файл `pom` обновлен для нового цикла разработки.
- `remove-release-poms`: удаляет файл `pom`, созданный для выпуска.
- `scm-commit-development`: отправляет файл `pom.xml` с версией разработки.
- `end-release`: завершает фазу подготовки выпускать.

Чтобы облегчить это, вы должны предоставить информацию SCM в файле проекта `pom.xml`. В листинге 8-3 показан фрагмент файла `pom.xml` с информацией SCM.

Глава 8. Релиз Maven

Листинг 8-3. pom.xml с информацией SCM

```
<проект>
  <modelVersion>4.0.0</modelVersion>

  <!-- Содержание удалено для краткости -->

  <scm>
    <соединение>scm:git:https://github.com/bava/intro-maven.
    гит </ соединение>
    <developerConnection>scm:git:https://github.com/bava/
    интро-maven.git</developerConnection>
    <url>https://github.com/bava/intro-maven</url>
  </scm>

  <!-- Содержание удалено для краткости -->
</проект>
```

После обновления файла pom.xml на локальном компьютере зафиксируйте измененный файл на GitHub, выполнив следующие команды:

```
git commit . -m "Добавлена информация SCM"
git push origin релиз
```

Чтобы Maven успешно взаимодействовал с GitHub, ему нужны учетные данные GitHub. Вы предоставляете эту информацию в settings.xml файл, как показано в листинге 8-4. Идентификатор элемента сервера объявляется как GitHub, так как он должен соответствовать имени хоста.

Листинг 8-4. Файл settings.xml с подробностями GitHub

```
<?xml версия="1.0" кодировка="UTF-8" ?>
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0 http://
maven.apache.org/xsd/settings-1.0.0.xsd">
```

```

<серверы>
  <сервер>
    <id>nexusReleases</id>

    <имя пользователя>администратор</имя пользователя>

    <пароль>admin123</пароль>
  </сервер>
  <сервер>
    <id>nexusSnapshots</id>
    <username>admin</username>

    <пароль>admin123</пароль>
  </сервер>
  <сервер>
    <id>github</id>
    <username>[your_github_account_name]</username>
    <password>[your_github_account_password]</password> </server>

</серверы>
</настройки>

```

Теперь у вас есть все настройки, необходимые для цели подготовки Maven. В листинге 8-5 показаны результаты выполнения цели `prepare`. Поскольку цель подготовки была запущена в интерактивном режиме, Maven предложит вам указать версию выпуска, тег выпуска или метку, а также новую версию разработки. Примите предложенные Maven значения по умолчанию, нажимая `Enter` для каждого приглашения.

Листинг 8-5. Maven подготовить команду

```
C:\apress\gswm-book\chapter8\gswm>mvn release:prepare [INFO]
```

```
Сканирование проектов...
```

```
[ИНФОРМАЦИЯ]
```

```
[ИНФОРМАЦИЯ] -----< com.apress.gswmbook:gswm >----- [ИНФОРМАЦИЯ]
```

```
Начало работы с Maven 1.0. 0-СИМОК
```

Глава 8. Релиз Maven

[INFO] --- maven-release-plugin:2.5.3:prepare (default-cli) @ gswm ---

[INFO] Проверка отсутствия локальных модификаций...

[INFO] Выполнение: cmd.exe /X /C "git rev-parse --show-toplevel"

[INFO] Рабочий каталог: C:\apress\gswm-book\chapter8\gswm

[INFO] Выполнение: cmd.exe /X /C "git status --porcelain".

Какова версия выпуска для «Начало работы с Maven»?

(com.apress.gswmbook:gswm) 1.0.0: :

Что такое тег или ярлык выпуска SCM для «Начало работы с

Maven»? (com.apress.gswmbook:gswm) gswm-1.0.0: :

Что представляет собой новая версия для разработки «Начало работы с Maven»? (com.apress.gswmbook:gswm) 1.0.1-СНИМОК: :

[INFO] Проверка измененных POM...

[INFO] Пометка выпуска меткой gswm-1.0.0...

[INFO] Выполнение: cmd.exe /X /C "git tag -FC:\Users\bavara\AppData\Local\Temp\maven-scm-73613791.commit gswm-1.0.0"

[INFO] Выполнение: cmd.exe /X /C "git push https://github.com/bava/intro-maven.git refs/tags/gswm-1.0.0"

[INFO] Подготовка к выпуску завершена.

[INFO] ДОСТИГАЙТЕ УСПЕХА

Обратите внимание, что команды Git выполняются как часть цели подготовки.

Успешное выполнение цели подготовки приведет к созданию тега

Git, как показано на рис. 8-12. Файл pom.xml в проекте gswm теперь будет иметь версию 1.0.1-SNAPSHOT.

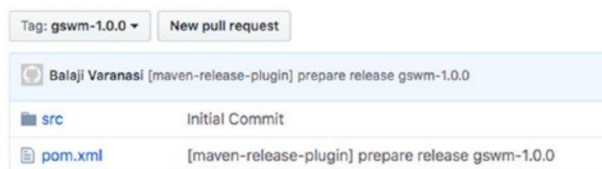


Рисунок 8-12. Тег Git, созданный при выполнении подготовки

Чистый гол

Цель подготовки выполняет множество действий и создает временные файлы, такие как `release.properties` и `pom.xml.releaseBackup`, как часть своего выполнения. После успешного завершения он очищает эти временные файлы.

Иногда цель подготовки может завершиться неудачей (например, невозможно подключиться к Git) и оставить проект в грязном состоянии. Вот тут-то и появляется цель чистой версии подключаемого модуля. Как следует из названия, он удаляет все временные файлы, созданные в ходе выполнения релиза.

Примечание Цель очистки подключаемого модуля выпуска должна использоваться только в случае сбоя цели подготовки.

Выполнить цель

Цель выполнения отвечает за проверку кода из вновь созданного тега, а также за сборку и развертывание выпущенного кода в удаленном репозитории.

Следующие этапы выполняются как часть выполнения цели:

- `verify-completed-prepare-phases`: подтверждает, что фаза подготовки была выполнена до запуска цели выполнения.

Глава 8. Релиз Maven

- checkout-project-from-scm: извлекает выпущенный код из тега SCM.
- run-perform-goal: выполняет цели, связанные с выполнять. Целью по умолчанию является развертывание.

Результат выполнения цели выполнения в проекте gswm показан на Листинг 8-6.

Листинг 8-6. Maven выполняет команду

```
C:\apress\gswm-book\chapter8\gswm>выпуск mvn:выполнить
[INFO] Поиск проектов...

[ИНФО] -----< com.apress.gswmbook:gswm >-----
[INFO] Начало работы с Maven 1.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] --- maven-release-plugin:2.5.3:perform (default-cli) @ gswm ---

[INFO] Проверка проекта для выполнения релиза ...
[INFO] Выполнение: cmd.exe /X /C "git clone --branch gswm-1.0.0 https://github.com/
bava/intro-maven.git C:\apress\gswm-book\
глава8\gswm\цель\оформить заказ"

[ИНФОРМАЦИЯ] Запуск выполнения целей в каталоге C:\apress\gswm-book\
глава8\gswm\цель\оформить заказ
[INFO] Выполнение целей "развернуть"...

[INFO] Строительная банка: C:\apress\gswm-book\chapter8\gswm\target\
проверка\цель\gswm-1.0.0-javadoc.jar

[INFO] --- maven-install-plugin:2.4:install (установка по умолчанию) @ gswm ---
```

Глава 8. Релиз Maven

[INFO] Установка C:\apress\gswm-book\chapter8\gswm\target\checkout\target\gswm-1.0.0.jar в C:\Users\bavara\.m2\репозиторий\com\apress\gswmbook\gswm\1.0.0\gswm-1.0.0.jar

[INFO] --- maven-deploy-plugin:2.7:deploy (развертывание по умолчанию) @gswm ---

[INFO] Загрузка в nexusReleases: http://localhost:8081/репозиторий/maven-релизы/com/apress/gswmbook/gswm/1.0.0/gswm-1.0.0.jar

[INFO] Загружено в nexusReleases: http://localhost:8081/репозиторий/maven-releases/com/apress/gswmbook/gswm/1.0.0/gswm-1.0.0.jar (2,4 КБ при 14 КБ/с)

[INFO] Загрузка в nexusReleases: http://localhost:8081/репозиторий/maven-релизы/com/apress/gswmbook/gswm/1.0.0/gswm-1.0.0.pom

[INFO] Загружено в nexusReleases: http://localhost:8081/репозиторий/maven-releases/com/apress/gswmbook/gswm/1.0.0/gswm-1.0.0-javadoc.jar (22 КБ при 84 КБ/с)

[ИНФО] ДОСТИГАЙТЕ УСПЕХА

На этом выпуск версии 1.0.0 проекта gswm завершен. Артефакт оказывается в менеджере репозитория Nexus, как показано на рис. 8-13.

Глава 8. Релиз Maven

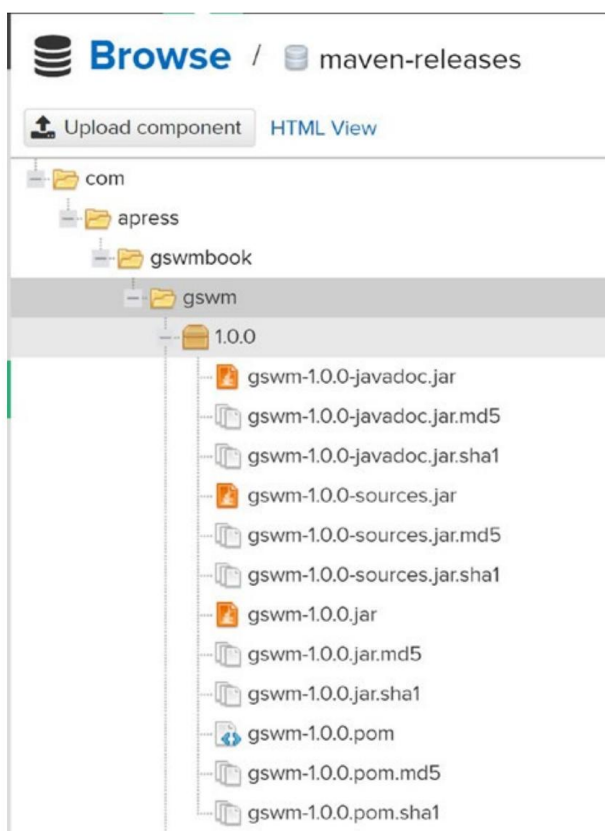


Рисунок 8-13. Нексус с выпущенным артефактом

Резюме

Внутренние менеджеры репозитория, такие как Nexus, позволяют предприятиям полностью внедрить Maven. Помимо того, что они служат в качестве прокси общедоступных репозитория, они обеспечивают совместное использование компонентов и управление ими. В этой главе мы рассмотрели интеграцию Maven с Nexus и познакомили вас с процессом развертывания артефакта в Nexus. Вы также узнали о процессе выпуска Maven и его различных фазах.

В следующей главе мы изучим концепции непрерывной интеграции (CI), а также установим и настроим Jenkins — популярный инструмент CI с открытым исходным кодом.

ГЛАВА 9

Непрерывный Интеграция

Непрерывная интеграция или CI — это передовая практика разработки программного обеспечения, при которой разработчики интегрируют изменения в свой код в общий репозиторий несколько раз в день. Каждое зафиксированное изменение приводило к автоматической сборке, которая компилировала код, запускала тесты и генерировала новую версию артефакта. О любых ошибках в процессе сборки будет немедленно сообщено команде разработчиков. Такая частая интеграция кода позволяет разработчикам выявлять и решать проблемы интеграции на ранних этапах цикла разработки.

Визуальное представление непрерывной интеграции вместе с задействованными компонентами показано на рис. 9-1. Поток CI начинается с того, что разработчик отправляет свои изменения в систему управления версиями, такую как Git или SVN. Сервер CI получает уведомление или отслеживает/опрашивает новые изменения кода и, обнаружив изменение, проверяет исходный код и запускает процесс сборки. При успешной сборке сервер CI может опубликовать артефакт в репозиторий или на тестовый сервер. В качестве последнего шага команде разработчиков отправляются уведомления о статусе сборки.

Глава 9 Непрерывная интеграция

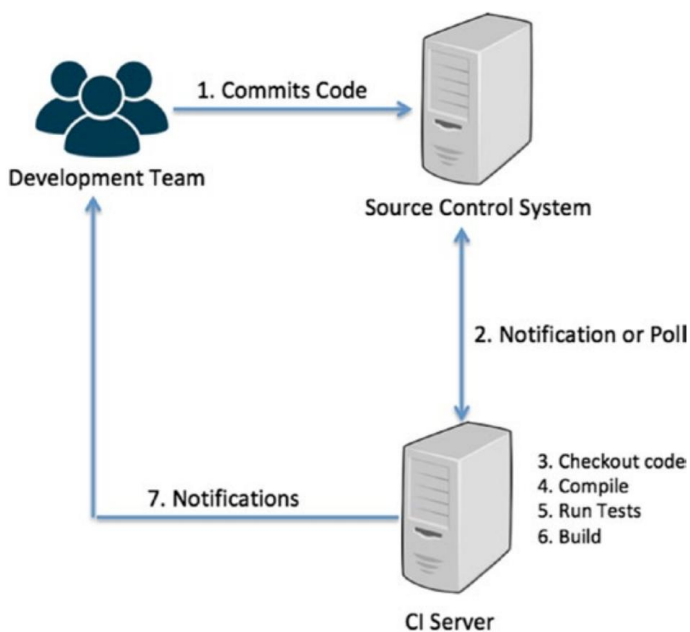


Рисунок 9-1. CI-компоненты

Jenkins — популярный CI-сервер с открытым исходным кодом, который хорошо интегрируется с Maven. Другие популярные серверы CI включают Bamboo, TeamCity и GitLab. В этой главе мы установим и настроим Jenkins для запуска автоматических сборок для проекта на основе Maven.

Установка Дженкинса

Jenkins распространяется в нескольких вариантах: собственные установщики, контейнеры Docker и исполняемый файл WAR. В этой книге мы будем использовать версию исполняемого файла WAR с долгосрочной поддержкой (LTS), которую вы можете скачать по адресу <https://jenkins.io/download/>. Сохраните загруженную версию по адресу `c:\tools\jenkins`.

После завершения загрузки с помощью командной строки перейдите в загруженную папку и выполните команду: `java -jar jenkins.war`. На

успешное выполнение команды, откройте браузер и перейдите по адресу <http://localhost:8080>. Вам будет предложено найти и ввести автоматически сгенерированный пароль администратора из файла «initialAdminPassword». На следующем экране выберите «Установить предлагаемые плагины» и дождитесь завершения установки плагина. На экране «Создать первого пользователя-администратора» введите «admin» в качестве имени пользователя и «admin123» в качестве пароля и заполните остальные данные в форме. По завершении настройки Jenkins вы должны увидеть панель инструментов Jenkins, аналогичную рисунку 9-2.

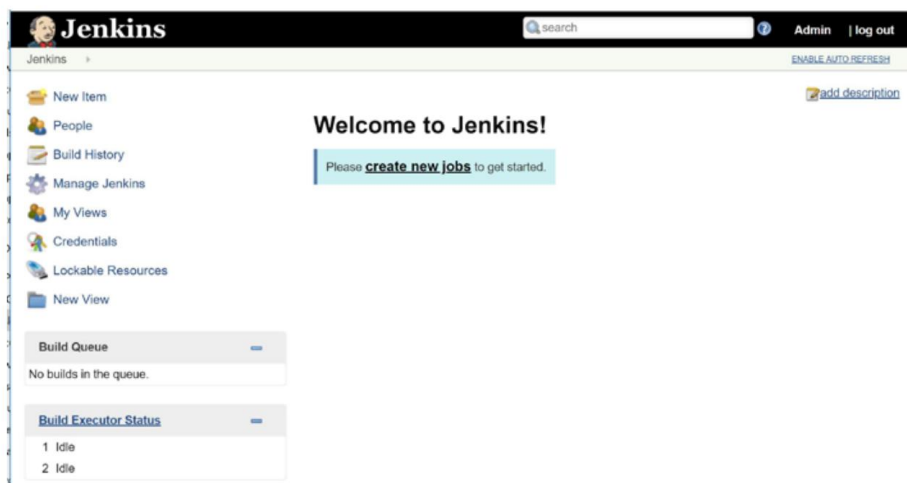


Рисунок 9-2. Дженкинс панель инструментов

Проект Мавен

Чтобы понять поддержку Jenkins для Maven, нам нужен пример проекта Maven на сервере управления исходным кодом. В этой главе мы будем использовать проект `gswm-jenkins`, размещенный на GitHub по адресу <https://github.com/bava/gswm-jenkins>. Чтобы вы могли следовать остальной части главы, вам нужно разветвить репозиторий `gswm-jenkins` под своей учетной записью. Вы можете сделать это, войдя в GitHub и нажав кнопку `fork`, как показано на рис. 9-3.

Глава 9 Непрерывная интеграция

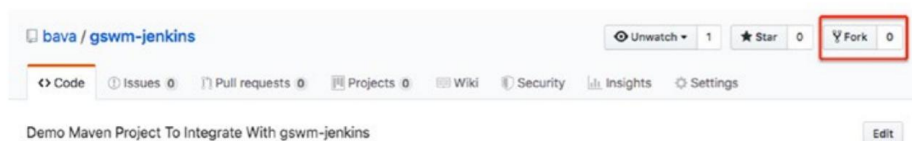


Рисунок 9-3. Вилка репозитория gswm-jenkins

Настройка Дженкинса

Чтобы начать настройку Jenkins, щелкните ссылку «Новый элемент» на панели инструментов. На экране «Новый элемент» выберите проект Freestyle и введите имя «gswm-jenkins-integration», как показано на рис. 9-4.

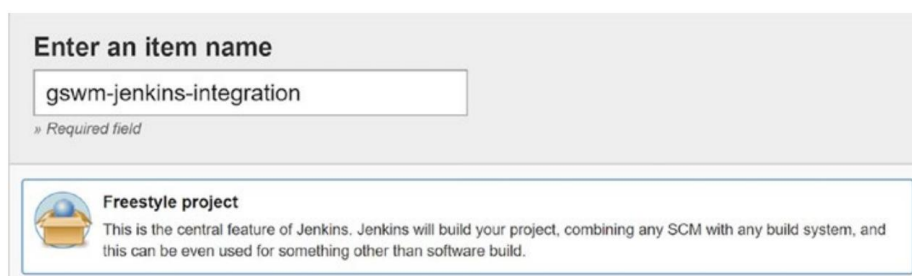


Рисунок 9-4. Экран нового элемента

На следующем экране в разделе «Общие» установите флажок «Проект GitHub» и введите URL-адрес проекта. Это должен быть URL-адрес вашего разветвленного проекта в вашей учетной записи GitHub.



Рисунок 9-5. Новый элемент - Общий раздел

В разделе «Управление исходным кодом» выберите переключатель «Git» и введите URL-адрес вашего репозитория GitHub, как показано на рисунке 9-6. Это URL-адрес клона GitHub, который вы можете найти, нажав «Клонировать или загрузить» под именем репозитория.

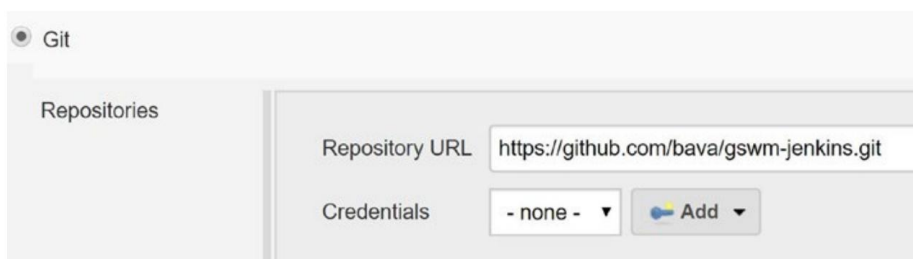


Рисунок 9-6. Новый раздел управления исходным кодом элемента

Чтобы Jenkins мог проверить ваш код, вам необходимо предоставить свои учетные данные GitHub. Для этого нажмите кнопку «Добавить» рядом с «Учетными данными» и введите свое имя пользователя и пароль, как показано на рис. 9-7.

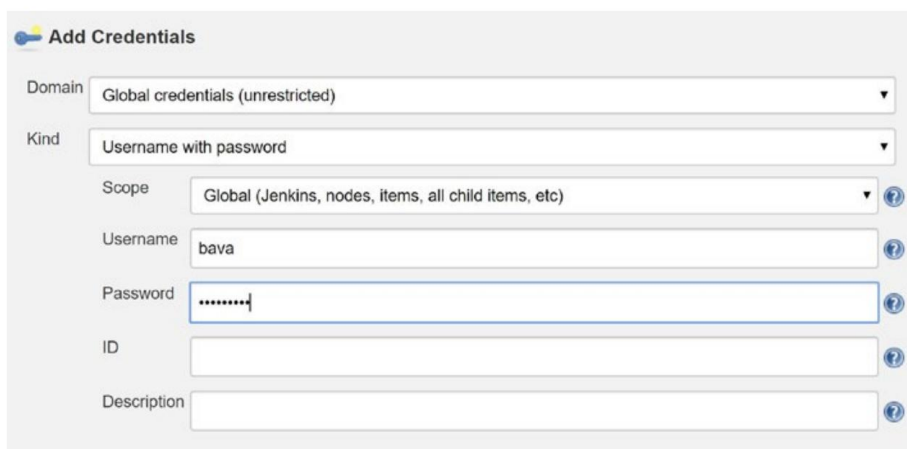


Рисунок 9-7. Ввод учетных данных GitHub

Глава 9 Непрерывная интеграция

В разделе Build Triggers выберите опцию «Poll SCM» и введите «H/15 * * * * |» в качестве значения, как показано на рисунке 9-8. Это указывает на то, что Jenkins необходимо опрашивать репозиторий GitHub на наличие изменений каждые 15 минут.

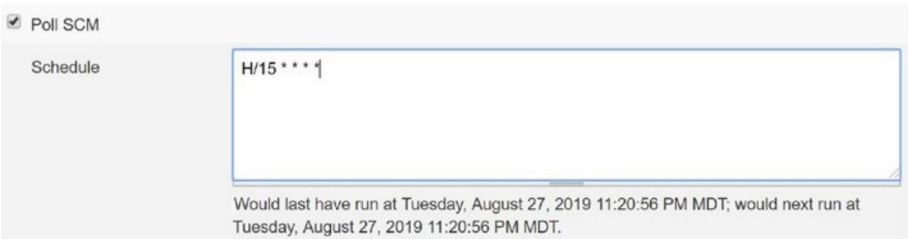


Рисунок 9-8. Построить расписание триггерного опроса

В разделе «Сборка» нажмите «Добавить шаг сборки» и выберите «Вызывать цели Maven верхнего уровня». Введите «чистая установка» в качестве цели, как показано на рис. 9-9.

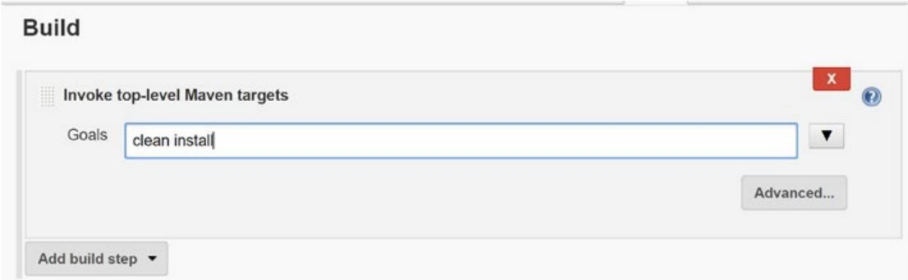


Рисунок 9-9. Шаг сборки для Maven

Наконец, в разделе «Действия после сборки» нажмите «Добавить действие после сборки» и выберите «Архивировать артефакты». Введите `target/*.jar` в качестве значения Files to Archive, как показано на рисунке 9-10.



Рисунок 9-10. Раздел архивных артефактов

Нажмите кнопку «Добавить действие после сборки» еще раз и выберите «Опубликовать отчет о результатах теста JUnit». Введите «target/surefire-reports/*.xml» в качестве значения XML отчета о тестировании, как показано на рисунке 9-11. Щелкните Сохранить, чтобы сохранить конфигурацию.

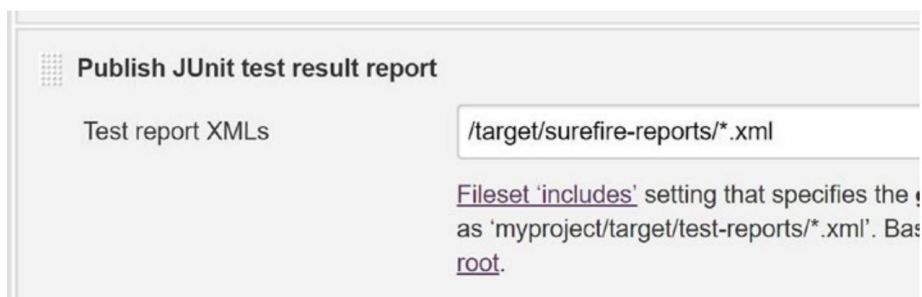


Рисунок 9-11. Опубликовать результаты JUnit

Запуск задания сборки

Теперь у нас есть все необходимое, чтобы Дженкинс построил наш проект. На странице задания проекта нажмите ссылку «Построить сейчас», чтобы запустить новую сборку. Это запустит новую сборку с числовым номером, доступ к которому вы можете получить в разделе «История сборки» в левом нижнем углу страницы. Щелкните стрелку раскрывающегося списка рядом с номером сборки и выберите «Консольный вывод». Это приведет вас к экрану вывода, подобному рисунку 9-12.

Глава 9 Непрерывная интеграция

```

[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ gswm-jenkins ---
[INFO] Building jar: C:\Users\bavara\.jenkins\workspace\gswm-jenkins-
integration\target\gswm-jenkins-1.0.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ gswm-jenkins ---
[INFO] Installing C:\Users\bavara\.jenkins\workspace\gswm-jenkins-integration\target\gswm-
jenkins-1.0.0-SNAPSHOT.jar to C:\Users\bavara\.m2\repository\com\apress\gswmbook\gswm-
jenkins\1.0.0-SNAPSHOT\gswm-jenkins-1.0.0-SNAPSHOT.jar
[INFO] Installing C:\Users\bavara\.jenkins\workspace\gswm-jenkins-integration\pom.xml to
C:\Users\bavara\.m2\repository\com\apress\gswmbook\gswm-jenkins\1.0.0-SNAPSHOT\gswm-
jenkins-1.0.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.685 s
[INFO] Finished at: 2019-08-27T23:25:36-06:00
[INFO] -----
Finished: SUCCESS

```

Рисунок 9-12. Экран вывода консоли задания

После успешного завершения задания вы увидите построенный артефакт на странице проекта, как показано на рисунке 9-13.



Рисунок 9-13. Страница проекта Jenkins — Build Artifact

Результаты теста прогона также доступны на странице проекта в разделе «Последний результат теста».

Резюме

В этой главе вы узнали о непрерывной интеграции и настроили Jenkins для взаимодействия с проектом Maven.

Это обсуждение подводит нас к концу нашего путешествия. На протяжении всей книги вы изучали ключевые концепции Maven. Мы надеемся, что вы будете использовать свои новые знания Maven для автоматизации и улучшения существующих процессов сборки и разработки программного обеспечения.

Индекс

А, Б

Муравей + Плющ

Встроенные

архетипы, [69](#), [70](#)

создание

Файл Java

AppServlet,

[82](#), [85](#) gswm-web-archetype, [84](#)

gswm-web-prototype, [80](#), [83](#) pom.xml,

[80](#), [81](#) структура проекта, [82](#), [83](#)

Servlet 4.0, [80](#) определено, [69](#)

тестовый проект, [86](#), [87](#)

С

Покрытие кода, [102–104](#)

Компоненты непрерывной

интеграции (CI), [127](#), [128](#)

Артефакты

архива Дженкинса,

триггер сборки [132](#),

[конфигурация](#) [132](#), экран

предметов, [130](#)

Учетные данные GitHub, [131](#), [132](#)

Проект GitHub, [130](#)

установка, [128](#), [129](#)

JUnit тест, [133](#)

Maven, [129](#)

запускает задание сборки, [133](#), [134](#)

Соглашение о конфигурации

(CoC), [4](#), [5](#)

Д

Область зависимости, [32](#), [33](#)

Добавление установки зависимостей,

репозиторий, руководство [34](#),

руководство [33](#)

Архитектура управления

зависимостями, [24](#)

идентификация зависимостей,

[28](#), [29](#)

Maven Central, [24](#)

менеджера репозитория, [25](#), [26](#)

Репозитории Spring/JBoss, [27](#), [28](#)

транзитивные зависимости, [29–31](#)

Э, Ф

Архив предприятия (AP), [28](#)

Корпоративные JavaBeans (EJB), [74](#)

Расширяемая разметка

язык (XML), [5](#)

ИНДЕКС

Г, Н

Глобальные настройки,

[16](#) целей, Maven Lifecycle

clean, [54](#) компиляции,

[53](#), [54](#) подключаемых

модуля справки, [55](#)

подключаемых модулей,

[54](#) файла pom.xml, [55](#), [56](#)

Gradle, [7](#) групп, артефактов и

zversion (GAV) , [28](#)

я

Интегрированные среды

разработки (IDE), [1](#)

Дж, К

Комплект для разработки Java (JDK), [11](#)

Джавадок, [99](#), [100](#)

Java Enterprise Edition (JEE), [74](#)

Среда выполнения Java (JRE), [11](#)

Обычная старая Java Java

Объект (ПОЖО), [61](#)

Виртуальная машина Java (JVM), [14](#)

Дженкинс, [128](#), [129](#)

Юнит/ТестНГ, [101](#)

Л

Встроенный

жизненный цикл,

[57](#), [58](#) целей, [58](#), [59](#)

фазы, [57-59](#)

Проект WAR, [60](#)

упаковочных элементов, [60](#)

М, Н, О

Maven, [129](#) Ant

+ Ivy, [5](#) файлов

Ant build.xml, [6](#) компиляций,

[5](#) листингов Ivy, [6](#) целей, [5](#)

задач, [5](#) архетипов, [4](#)

загрузки архива, [12](#)

ошибок артефактов, [19](#)

CoC, [4](#), [5](#) управление

зависимостями, [2](#) структуры

каталогов , [2](#) Doxia, [10](#) Gradle,

[7](#) файл build.gradle по

умолчанию, [7](#) использование

инструмента сборки, [8](#) DSL, [7](#) команда

справки, [15](#) IDE, [21](#) установка

содержимое каталога, [12](#)

Мас, [13](#)

тестов, [14](#)

окон, [13](#)

с открытым исходным

кодом, [4](#) плагина, [3](#)

прокси, настройка, [19](#)

- защита паролей, [20](#) файлов
- settings.xml, [16–18](#) каркасных
- файлов settings.xml, [17](#) систем
- контроля версий /управления
 - кодом (SCM), [9](#) инструментов
- поддержки, [3](#) вагона, [9](#) Maven
- Doxia, [10](#) Maven Old Java Object
- (MOJO), [61](#) Maven выпускать
- чистый гол, [123](#)
- Установка клиента Git, [115](#)
- Репозитории GitHub, [115](#), [116](#)
- взаимодействие с GitHub, [114](#)
- Нексус
 - дистрибуцияУправление
 - элемент, [110](#)
 - установка, [108](#)
 - режим входа в
 - систему, [109](#) менеджеры
 - репозитория, [107](#) информация о сервере, [115](#)
 - Артефакт SNAPSHOT, [112](#)
- выполнить команду цели, [124](#), [125](#)
- Нексус, [126](#)
- фаз, [123](#)
- команды
 - подготовки цели,
 - [121](#) выполнение, [122](#)
 - Детали GitHub, [120](#)
 - операций, [118](#)
 - Информация SCM, [119](#)
- проверка исходного кода, [116–118](#)
- свойство maven.test.skip, [61](#)
- команда mkdir gswm, [39](#)
- многомодульный проект,
 - интерактивный режим
 - параметр, [77](#)
- Команда mvn package, [79](#)
- Родительский файл pom.xml, [76](#), [77](#) сервисный проект, [76](#)
- структура, [74](#) визуальное
- представление, [75](#) веб-модуль, [78](#), [79](#)
- команда пакета mvn, [58](#)
- Плагины P, Q, R Apache
- Commons Lang, [62](#) Проект Maven
- Java, [61](#), [62](#) команда mvn install, [65](#) pom.xml, [62–64](#) цель systeminfo, [66–68](#) SystemInfoMojo, [64](#), [66](#)
- SystemInfoPlugin, [61](#)
- файл pom.xml, [24](#), [40](#), [41](#)
- неявные свойства, [50](#)
- пользовательских
- свойства, [50](#), [51](#)
- Здание
- проекта
 - Класс HelloWorld, [42](#)
 - упакованных файла JAR, [44](#)
 - Пакет Maven, [43](#)
 - структура проекта, [43](#)

ИНДЕКС

Тестирование

проекта

(продолжение) HelloWorldTest,

[47](#), [49](#) Зависимость JUnit, [44](#), [46](#)

структура, [47](#) целевая папка, [49](#)

Команда дерева, [46](#)

организационные компоненты,

[38](#) создание, проект, [39](#), [40](#) каталоги,

[39](#) структура, [37](#)

С

Архетипы

конфигурации

жизненного цикла

сайта, [95](#) структура каталогов,

[94](#), [95](#) содержимое файла, [96](#), [97](#)

логотип, [98](#) содержимое, [89](#)

зависимости, [90](#) элемент описания,

[92](#), [93](#) сгенерированный веб-сайт, [93](#)

индексная страница, [90](#)

SNAPSHOT квал., [42](#)

Контроль версий/управление кодом

(СКМ) системы, [9](#), [38](#)

SpotBugs, [104](#), [105](#)

Т, У, В

Разработка через тестирование

Конфигурация

плагины Surefire, [101](#)

Верный отчет, [101](#), [102](#)

Транзитивные зависимости, [29](#)

посредничество в зависимости, [29](#)

JUnit, [31](#)

подключаемый модуль дерева, [30](#)

Ш, Х, И, Я

Архив веб-приложений (WAR), [5](#)

Браузер веб-

проектов, [73](#), [74](#)

входа, информация, [71](#)

Плагин Jetty, [72](#), [73](#)

Команда запуска причала, [73](#)

maven-archetype-webapp, [71](#)

Командование причала,

строение [73](#), [71](#), [72](#)